

CSCI103: Data Engineering

통합 강의 노트

Contents

Chapter 1

용어 정리

강의명: CSCI103 Lecture 1: 데이터 엔지니어링 소개

주차: Lecture 1

교수명: UIUC Student

목적: 본 문서는 CSCI103 Lecture 1 강의 자료를 통합하여 작성된 시험 대비용 LaTeX 노트입니다. 데이터 엔지니어링의 기본 개념, 빅데이터의 특성, 데이터 플랫폼의 진화 과정, 그리고 클라우드 기반 실행 환경인 Databricks 사용법을 다룹니다. 강의 목표는 비즈니스 문제 해결을 위한 데이터 엔지니어링의 중요성을 이해하고, 실제 분석 환경에서 데이터를 효과적으로 다루는 기술을 습득하는 것입니다. 주요 평가 포인트는 과제, 퀴즈, 최종 그룹 프로젝트 발표이며, 적극적인 참여가 중요합니다. 이 문서는 초심자도 완벽히 이해할 수 있도록 상세한 설명과 예시를 포함합니다.

용어 정리

다음은 강의에서 소개된 주요 용어들을 쉽게 설명한 표입니다.

용어	쉬운 설명	원어	비고
데이터 엔지니어링	원시 데이터를 가치 있는 통찰로 바꾸는 과정	Data Engineering	데이터 파이프라인 구축 및 관리
빅데이터	기존 도구로 처리하기 어려운 대규모 데이터	Big Data	5V(Volume, Velocity, Variety, Veracity, Value) 특성
Databricks	빅데이터 처리 및 머신러닝을 위한 통합 플랫폼	Databricks	Spark 기반, 레이크하우스 아키텍처
데이터 페르소나	데이터 관련 업무를 수행하는 다양한 역할	Data Personas	엔지니어, 과학자, 분석가 등
분산 컴퓨팅	여러 컴퓨터가 협력하여 작업을 처리하는 방식	Distributed Computing	빅데이터 처리에 필수적
머신러닝 (ML)	데이터 학습을 통해 예측/결정하는 인공지능 분야	Machine Learning	데이터 엔지니어링의 주요 결과물
ETL	데이터 추출, 변환, 적재 과정	Extract, Transform, Load	데이터 파이프라인의 핵심
데이터 파이프라인	데이터가 흐르고 처리되는 자동화된 경로	Data Pipeline	원시 데이터 → 정제 데이터
레이크하우스	데이터 레이크와 데이터 웨어하우스의 장점 결합	Lakehouse	Databricks의 핵심 아키텍처
ACID	데이터베이스 트랜잭션의 4가지 속성	Atomicity, Consistency, Isolation, Durability	관계형 데이터베이스의 신뢰성 보장
BASE	NoSQL 데이터베이스의 3가지 속성	Basically Available, Soft state, Eventual consistency	유연성 및 확장성 중시
CAP 이론	분산 시스템의 3가지 속성 중 2가지만 만족 가능	Consistency, Availability, Partition tolerance	시스템 설계 시 트레이드오프
OLTP	실시간 트랜잭션 처리 시스템	Online Transactional Processing	은행 거래, 온라인 쇼핑 등
OLAP	대규모 데이터 분석 시스템	Online Analytical Processing	보고서, 비즈니스 인텔리전스
Hadoop	대규모 데이터 분산 처리를 위한 오픈소스 프레임워크	Hadoop	데이터 레이크 시대의 시작
Spark	Hadoop의 한계를 극복한 인메모리 분산 처리 엔진	Spark	빠른 속도, 다양한 기능 지원
RDD	Spark의 기본 데이터 추상화 단위	Resilient Distributed Dataset	불변성, 분산 처리
데이터프레임	Spark의 구조화된 데이터 처리 API	DataFrame	RDD보다 사용하기 쉬운 추상화
IaaS	인프라를 서비스로 제공하는 클라우드 모델	Infrastructure as a Service	가상 머신, 스토리지 등
PaaS	플랫폼을 서비스로 제공하는 클라우드 모델	Platform as a Service	개발 환경, 데이터베이스 등
SaaS	소프트웨어를 서비스로 제공하는 클라우드 모델	Software as a Service	Gmail, Salesforce, Databricks 등
Unity Catalog	Databricks의 통합 데이터 거버넌스 솔루션	Unity Catalog	데이터 접근 제어, 메타데이터 관리

핵심 개념 · 원리

1. 데이터 엔지니어링의 정의와 중요성

핵심 요약

데이터 엔지니어링은 원시 데이터를 수집, 변환, 관리하여 비즈니스 통찰력을 얻고 머신러닝 모델을 구축하는 데 사용할 수 있도록 준비하는 과정입니다. 이는 현대 비즈니스에서 데이터의 가치를 극대화하는 핵심적인 역할입니다.

왜 필요한가? 오늘날 대부분의 비즈니스는 데이터를 기반으로 운영됩니다. 하지만 데이터는 종종 여러 소스에 흩어져 있고, 정제되지 않은 '날것'의 형태입니다. 이러한 원시 데이터는 그 자체로는 가치가 낮습니다. 데이터 엔지니어링은 이 원시 데이터를 분석 가능한 형태로 가공하고, 신뢰할 수 있는 파이프라인을 통해 지속적으로 제공함으로써 비즈니스 의사결정과 혁신을 가능하게 합니다.

직관적 비유: 데이터 엔지니어는 마치 정수기 필터와 배관공을 합쳐놓은 것과 같습니다. 수도꼭지에서 나오는 흙탕물(원시 데이터)을 바로 마실 수는 없습니다. 데이터 엔지니어는 이 흙탕물을 깨끗하게 정수(변환)하고, 각 가정(분석가, 데이터 과학자)에 필요한 형태로 안정적으로 공급(파이프라인)하는 역할을 합니다. 그래야 사람들이 깨끗한 물(통찰력)을 마시고 건강(비즈니스 성장)해질 수 있습니다.

기술적 설명: 데이터 엔지니어링은 소프트웨어 엔지니어링, 데이터 과학, 비즈니스 인텔리전스(BI)의 교차점에 있습니다. 주요 업무는 다음과 같습니다:

- **데이터 구조 및 분산 컴퓨팅:** 대규모 데이터를 효율적으로 저장하고 처리하기 위한 구조를 설계하고 분산 시스템을 활용합니다.
- **프로그래밍 및 도구:** Python, Scala, SQL 등 다양한 언어와 Spark, Databricks 같은 도구를 사용하여 데이터를 처리합니다.
- **ETL 및 데이터 파이프라인 구축:** 데이터를 추출(Extract), 변환(Transform), 적재(Load)하는 과정을 자동화하는 파이프라인을 설계하고 구현합니다.
- **소프트웨어 엔지니어링 원칙 적용:** 설계, 개발, 웹/모바일 앱, DevOps, 서비스 배포 등 소프트웨어 개발의 모범 사례를 따릅니다.

- **데이터 과학 및 비즈니스 인텔리전스 이해:** 머신러닝 모델링, 알고리즘, 비즈니스 요구사항을 이해하여 데이터가 최종적으로 어떻게 활용될지 고려합니다.

2. 데이터 관련 페르소나 (Data Personas)

데이터를 다루는 다양한 역할들이 있으며, 이들은 서로 협력하여 데이터의 가치를 창출합니다. 데이터 엔지니어는 이 모든 역할과 긴밀하게 연결되어 있습니다.

페르소나	주요 역할	데이터 엔지니어와의 관계
데이터 엔지니어	데이터 파이프라인 구축 및 유지보수, 데이터 변환 및 준비	모든 역할의 기반 데이터 제공
BI 분석가	정제된 데이터를 분석하여 대시보드 및 보고서 생성	엔지니어가 준비한 데이터를 소비
데이터 과학자/ML 실무자	머신러닝 모델 개발, 피쳐 엔지니어링, 데이터 기반 예측	엔지니어가 준비한 고품질 데이터를 활용하여 모델 학습
DevOps/MLOps	데이터 파이프라인 및 ML 모델 배포 및 자동화 지원	엔지니어가 구축한 파이프라인의 운영 효율성 담당
데이터 리더	데이터 전략 수립, IT 조직의 데이터 방향성 제시	엔지니어에게 비즈니스 요구사항 전달, 데이터 활용 가이드

핵심 정보

최종 프로젝트는 그룹 프로젝트로 진행되며, 각 팀원은 이러한 데이터 페르소나 중 하나를 맡아 프로젝트의 특정 측면을 발표하게 됩니다. 이는 실제 업무 환경을 반영한 것입니다.

3. 빅데이터의 특성: 5V

오늘날 대부분의 데이터셋은 '빅데이터'로 분류되며, 그 특성은 5가지 'V'로 설명됩니다.

특성	설명	예시
Volume (규모)	데이터의 물리적 크기 (테라바이트, 페타바이트 이상)	수십억 건의 고객 거래 기록, 수천 시간의 비디오 데이터
Velocity (속도)	데이터가 생성되고 처리되는 속도 (실시간, 배치)	주식 시장 데이터 (초당 수천 건), IoT 센서 데이터 (지속적인 스트림)
Variety (다양성)	데이터의 형태 (정형, 반정형, 비정형)	관계형 데이터베이스(정형), JSON/XML(반정형), 오디오/비디오/이미지(비정형)
Veracity (정확성)	데이터의 신뢰성 및 정확성	잘못된 센서 값, 중복된 고객 정보, 편향된 설문 응답
Value (가치)	데이터를 통해 얻을 수 있는 잠재적 비즈니스 가치	고객 행동 분석을 통한 맞춤형 추천, 질병 예측 모델

예제

5V 예시: 소셜 미디어 데이터

- **Volume:** 매일 수십 페타바이트의 게시물, 좋아요, 댓글이 생성됩니다.
- **Velocity:** 사용자들이 실시간으로 콘텐츠를 생성하고 상호작용합니다.
- **Variety:** 텍스트(게시물), 이미지, 비디오(비정형), 사용자 프로필(정형), API 응답(반정형) 등 다양한 형태입니다.
- **Veracity:** 가짜 뉴스, 봇 활동, 스팸 등으로 인해 데이터의 신뢰성이 떨어질 수 있습니다.
- **Value:** 사용자 관심사 분석, 트렌드 파악, 광고 타겟팅, 여론 분석 등 막대한 비즈니스 가치를 가집니다.

4. 데이터 분류 기준

데이터는 다양한 기준에 따라 분류될 수 있으며, 이는 데이터 처리 전략을 결정하는 데 중요합니다.

분류 기준	설명	예시
크기	• 빅데이터: 테라바이트 이상, 분산 컴퓨팅 필요 • 수십억 건의 웹 로그 (빅데이터)	
구조	• 정형 데이터 (Structured): 스키마가 명확 • 반정형 데이터 (Semi-structured): 스키마가 데이터 내에 포함 • 비정형 데이터 (Unstructured): 스키마 없음 • JSON, XML (반정형) • 이미지, 오디오, 비디오, 문서 (비정형)	
속도	• 배치 (Batch): 일정 시간 동안 모아서 일괄 처리 • 스트리밍 (Streaming): 데이터가 생성되는 즉시 연속적으로 처리 • 마이크로 배치 (Micro-batch): 스트리밍과 배치 중간 형태 • 일일 재고 보고서 (배치) • 주식 시세, IoT 센서 데이터 (스트리밍)	
변경 빈도	• 거의 변경 없음: 매우 드물게 업데이트 • 가끔 변경: 주기적으로 업데이트 • 자주 변경: 빈번하게 업데이트 • 매우 자주 변경: 지속적으로 업데이트 • 인구 조사 데이터 (거의 변경 없음) • 공장 기계 상태 (가끔 변경) • 소셜 미디어 상호작용 (자주 변경) • 날씨 데이터 (매우 자주 변경)	

5. 데이터 온도와 처리 속도

데이터는 그 중요성과 긴급성에 따라 '온도' 개념으로 분류되며, 이에 따라 적절한 처리 속도가 요구됩니다.

데이터 온도	설명	처리 속도
Hot Data (뜨거운 데이터)	최근에 생성되었고, 빠르게 변화하며, 즉각적인 분석이 필요한 데이터	실시간 (Real-time): 데이터 도착 즉시 처리
Warm Data (따뜻한 데이터)	비교적 최근 데이터지만, 즉각적인 처리가 필수는 아닌 데이터	인터랙티브 (Interactive): 필요 시 즉시 쿼리 및 분석
Cold Data (차가운 데이터)	오래되었고, 거의 변경되지 않으며, 보관 및 가끔의 분석에 사용되는 데이터	배치 (Batch): 일정 시간 모아서 일괄 처리

예제

데이터 온도 및 처리 속도 예시:

- **Hot Data (실시간)**: 온라인 사기 탐지 시스템. 거래가 발생하는 즉시 분석하여 사기 여부를 판단해야 합니다. 5분, 15분 지연은 큰 손실로 이어질 수 있습니다.
- **Warm Data (인터랙티브)**: 고객 서비스 상담원이 고객의 최근 1주일 구매 내역을 조회하여 상담에 활용하는 경우. 즉각적인 응답이 필요하지만, 초 단위의 실시간은 아닙니다.
- **Cold Data (배치)**: 연간 재무 보고서 작성. 지난 1년간의 모든 거래 데이터를 밤새 일괄 처리하여 다음 날 아침에 보고서를 생성합니다.

주의

비즈니스에서 항상 '실시간' 처리를 요구하지만, 모든 데이터가 실시간으로 처리될 필요는 없습니다. 실시간 처리는 비용이 많이 들기 때문에, 해당 통찰이 정말로 '실행 가능(actionable)'하고 비즈니스 가치가 높은지 신중하게 판단해야 합니다.

6. 데이터 가치 증대 과정 (Refinement)

원시 데이터는 여러 단계를 거쳐 정제되고 변환될수록 비즈니스 가치가 높아집니다.

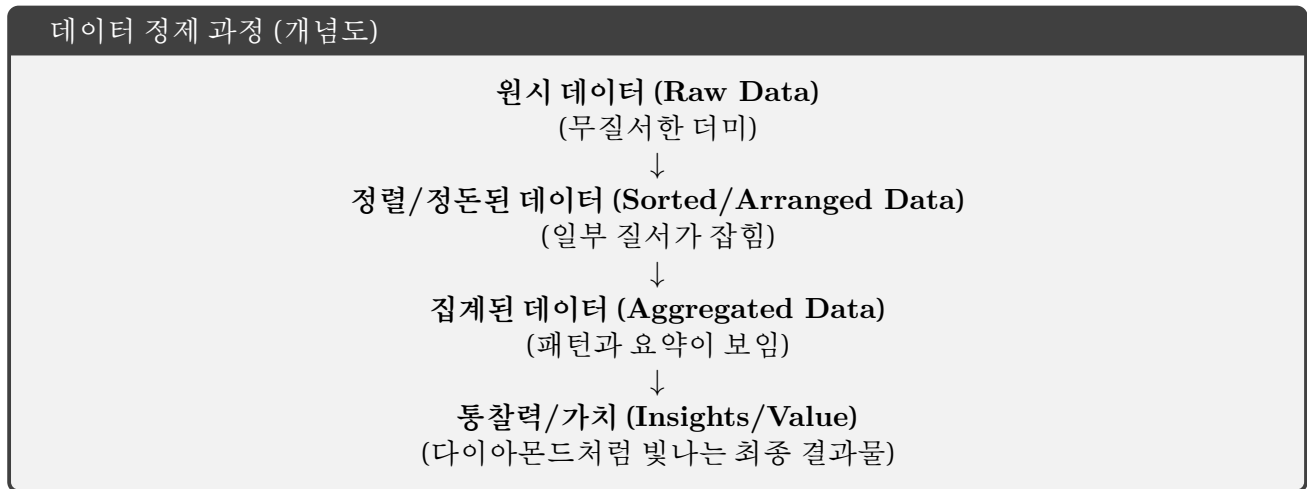


Figure 1.1: 원시 데이터에서 가치 있는 통찰로의 변환 과정

- **원시 데이터 (Raw Data):** 소스 시스템에서 직접 추출된, 가공되지 않은 데이터. (예: 흙탕물)
- **정제 및 변환 (Cleansing & Transformation):** 데이터의 오류를 수정하고, 형식을 표준화하며, 필요한 형태로 가공합니다. (예: 정수 필터링)
- **집계 (Aggregation):** 데이터를 요약하고 그룹화하여 패턴을 찾습니다. (예: 물을 병에 담아 분류)
- **통찰력 (Insights):** 정제된 데이터를 분석하여 비즈니스 의사결정에 도움이 되는 의미 있는 정보를 도출합니다. (예: 깨끗한 생수)

핵심 정보

Databricks에서는 이러한 데이터 정제 과정을 메달리온 아키텍처(Medallion Architecture)로 설명합니다.

- **Bronze (브론즈):** 원시 데이터 (최소한의 정제)
- **Silver (실버):** 정제되고 필터링된 데이터 (비즈니스 규칙 적용)
- **Gold (골드):** 집계되고 분석 준비가 완료된 데이터 (최종 사용자용)

이 아키텍처는 데이터의 품질과 신뢰성을 단계적으로 높여줍니다.

7. 데이터 플랫폼의 진화

데이터를 다루는 기술 스택은 시대에 따라 계속 발전해 왔지만, 핵심적인 사용 사례(use cases)와 데이터 자체의 가치는 변하지 않습니다.

시대	주요 기술/개념	특징	한계/변화 요인
1960년대	DBMS (데이터베이스 관리 시스템)	플랫 파일에서 관계형 데이터베이스로 전환	데이터 볼륨 증가, 비정형 데이터 미지원
1980년대	데이터 웨어하우스 (Data Warehouse)	차원 모델링, 데이터 마트, MPP (대규모 병렬 처리)	고비용, 독점 시스템, BI 중심, 비정형 데이터 미지원
2000년대	웹 및 비정형 데이터	웹의 부상, 오디오/비디오/이미지, 메타데이터 중요성 증가	SQL의 한계, NoSQL 등장, 스트리밍 초기 단계
2010년대 초	Hadoop (하둡)	오픈소스, 저비용 분산 처리, 데이터 레이크 개념 도입	데이터 품질/신뢰성 문제, 복잡한 관리
2010년대 중반	Spark (스파크)	Hadoop의 한계 극복, 인메모리 처리, 빠른 속도, 클라우드 채택	클라우드 컴퓨팅 (AWS, Azure, GCP) 확산, 스토리지/컴퓨트 분리
2020년대	레이크하우스 (Lakehouse)	데이터 레이크와 데이터 웨어하우스의 장점 결합	데이터 메시, 데이터 페브릭 등 새로운 아키텍처 등장

핵심 정보

기술 부채 (Tech Debt): 오래된 기술 스택을 계속 사용하면 유지보수 비용이 증가하고 새로운 기능을 도입하기 어려워지는 현상입니다. 비즈니스는 결국 '충알을 물고(bite the bullet)' 더 현대적인 스택으로 전환해야 합니다.

8. SQL 기반 시스템 vs NoSQL 기반 시스템

데이터베이스는 크게 SQL(관계형)과 NoSQL(비관계형)로 나눌 수 있으며, 각각 다른 특성과 사용 사례를 가집니다.

8.1. SQL 기반 시스템 (ACID)

핵심 요약

SQL 기반 시스템은 관계형 데이터베이스의 핵심으로, 데이터의 일관성과 신뢰성을 최우선으로 합니다. **ACID** 속성을 통해 트랜잭션의 무결성을 보장합니다.

ACID 속성:

- **Atomicity (원자성):** 트랜잭션 내의 모든 작업은 완전히 성공하거나, 아니면 완전히 실패하여 이전 상태로 되돌려집니다 (All or Nothing).

예제

예시: 은행 계좌 이체. A 계좌에서 돈을 인출하고 B 계좌로 입금하는 두 작업이 모두 성공해야 합니다. 만약 인출만 되고 입금이 실패하면, 전체 트랜잭션은 롤백되어 A 계좌의 돈도 다시 돌아옵니다.

- **Consistency (일관성):** 트랜잭션이 성공적으로 완료되면 데이터베이스는 항상 유효한 상태를 유지합니다. 데이터베이스 규칙(제약 조건)을 위반하지 않습니다.

예제

예시: 재고 관리 시스템. 상품이 판매되면 재고 수량이 정확히 감소해야 합니다. 10개 팔렸는데 9개만 감소하면 일관성이 깨집니다.

- **Isolation (격리성):** 여러 트랜잭션이 동시에 실행될 때, 각 트랜잭션은 다른 트랜잭션의 영향을 받지 않고 독립적으로 실행되는 것처럼 보입니다.

예제

예시: 두 고객이 동시에 같은 상품을 구매하려고 할 때, 한 고객의 구매가 완료될 때까지 다른 고객은 해당 상품의 재고 상태를 변경할 수 없습니다.

- **Durability (영속성):** 트랜잭션이 성공적으로 완료되면, 그 결과는 시스템 오류(정전 등)에도 불구하고 영구적으로 저장됩니다.

예제

예시: 결제 완료 후 시스템이 다운되더라도, 결제 기록은 사라지지 않고 유지됩니다.

주요 사용 사례:

- 고도로 구조화된 데이터 (정형 데이터)
- 예측 가능한 입력 및 출력
- 금융 시스템, 회계 장부, 재고 관리 등 데이터의 정확성과 일관성이 매우 중요한 경우

8.2. NoSQL 기반 시스템 (BASE)**핵심 요약**

NoSQL 기반 시스템은 대규모 분산 환경에서 유연성, 확장성,고가용성을 중시합니다. **BASE** 속성을 통해 일관성보다는 가용성과 성능에 초점을 맞춥니다.

BASE 속성:

- **Basically Available (기본적 가용성):** 시스템의 일부 노드에 장애가 발생하더라도 전체 시스템은 계속해서 작동하며 요청에 응답합니다.

예제

예시: 소셜 미디어 서비스. 일부 서버가 다운되어도 사용자는 계속해서 게시물을 보거나 올릴 수 있습니다.

- **Soft state (유연한 상태):** 시스템의 상태가 즉시 일관되지 않을 수 있습니다. 여러 노드에 데이터가 분산되어 저장될 때, 모든 노드가 동시에 최신 상태를 반영하지 못할 수 있습니다.
- **Eventual consistency (최종 일관성):** 데이터가 모든 노드에 즉시 전파되지 않더라도, 충분한 시간이 지나면 모든 노드의 데이터가 최종적으로 일관된 상태가 됩니다.

예제

예시: 온라인 쇼핑몰의 '좋아요' 카운트. 사용자가 '좋아요'를 눌렀을 때, 즉시 모든 사용자에게 카운트가 반영되지 않을 수 있지만, 잠시 후에는 정확한 카운트가 표시됩니다. 몇 초의 지연은 허용됩니다.

주요 사용 사례:

- 스키마가 자주 변경되거나 유연한 데이터 구조가 필요한 시나리오
- 대규모 웹 서비스, 소셜 미디어, IoT 데이터, 실시간 분석 등 고가용성과 확장성이 중요한 경우

8.3. CAP 이론 (Consistency, Availability, Partition tolerance)

핵심 요약

CAP 이론은 분산 컴퓨팅 시스템이 일관성(Consistency), 가용성(Availability), 분할 내성(Partition tolerance) 세 가지 속성 중 동시에 두 가지만 만족할 수 있다는 이론입니다. 시스템 설계 시 어떤 속성을 우선할지 선택해야 합니다.

CAP 속성:

- **Consistency (일관성):** 모든 클라이언트가 동일한 데이터를 보게 됩니다. 데이터가 업데이트되면 모든 노드에서 즉시 반영됩니다.
- **Availability (가용성):** 모든 요청에 대해 시스템이 항상 응답을 반환합니다. (응답이 성공적이지 않을 수 있어도)
- **Partition tolerance (분할 내성):** 네트워크 분할(일부 노드 간 통신 단절)이 발생하더라도 시스템이 계속 작동합니다.

CAP 이론에 따른 시스템 분류:

- **CP 시스템 (Consistency + Partition tolerance):** 네트워크 분할 시 일관성을 유지하기 위해 가용성을 포기합니다. (예: 일부 관계형 데이터베이스, ZooKeeper)
- **AP 시스템 (Availability + Partition tolerance):** 네트워크 분할 시 가용성을 유지하기 위해 일관성을 포기합니다. (예: NoSQL 데이터베이스 - Cassandra, DynamoDB)
- **CA 시스템 (Consistency + Availability):** 분할 내성을 포기합니다. 단일 노드 시스템이나 네트워크 분할이 발생하지 않는 환경에서만 가능합니다. (예: 전통적인 단일 서버 관계형 데이터베이스)

주의

실제로는 세 가지 속성을 모두 완벽하게 달성하기는 매우 어렵습니다. 특히 데이터 규모가 커질수록 데이터 일관성을 유지하는 것은 큰 도전 과제입니다.

9. OLTP 시스템 vs OLAP 시스템

데이터베이스 시스템은 그 목적에 따라 크게 두 가지로 나뉩니다.

구분	OLTP (Online Transactional Processing)	OLAP (Online Analytical Processing)
목적	실시간 트랜잭션 처리	대규모 데이터 분석 및 보고
특징	• 짧고 빈번한 트랜잭션 (읽기, 쓰기, 업데이트, 삭제) • 복잡하고 장기 실행되는 쿼리	
기술 예시	• SQL 기반: Oracle, SQL Server, MySQL • NoSQL 기반: MongoDB (문서), Cassandra (컬럼), DynamoDB (키-값) • 중기: Hadoop • 현대: Databricks, Snowflake (클라우드 데이터 플랫폼)	• 초기: 데이터 웨어하우스, 데이터 마트
사용 사례	• 은행의 계좌 거래 • 항공권 예약 시스템 • 고객 행동 패턴 분석 • 시장 트렌드 예측	• 온라인 상점의 주문 처리 • 월별 판매 보고서 생성

OLTP 및 OLAP 시스템의 데이터 흐름 (개념도)



Figure 1.2: OLTP와 OLAP 시스템 간의 데이터 흐름

10. 현대 데이터 플랫폼의 진화 과정

현대 데이터 플랫폼은 다양한 데이터 소스, 유형, 처리 방식을 통합하여 비즈니스 사용 사례를 지원하도록 발전해 왔습니다.

영역	과거	현재
데이터 소스	적은 수의 정형 데이터 소스	더 많은 데이터 소스, 고객에 대한 전체적인 시야
데이터 유형	주로 정형 데이터 (행/열, 스키마 명확)	정형, 반정형 (JSON/XML), 비정형 (텍스트, 이미지, 비디오)
데이터 수집	배치 처리 (일괄)	배치, 실시간 스트리밍, 마이크로 배치
데이터 처리 (ETL)	데이터 웨어하우스 (구형)	현대 데이터 플랫폼 (Snowflake, Databricks)
사용 사례	보고서 (Reporting)	보고서, 임시 쿼리 (EDA), 머신러닝 (ML)

11. Hadoop과 Spark의 비교

Hadoop은 데이터 레이크 시대의 시작을 알렸지만, Spark는 그 한계를 극복하고 현대 빅데이터 처리의 핵심 엔진이 되었습니다.

11.1. Hadoop의 특징

핵심 요약

Hadoop은 대규모 데이터를 저렴한 비용으로 분산 처리하기 위해 설계된 오픈소스 프레임워크입니다. HDFS(분산 파일 시스템)와 MapReduce(분산 처리 모델)가 핵심입니다.

장점:

- **저비용:** 값싼 범용 하드웨어(commodity grade hardware)를 사용하여 고가의 독점 서버를 대체할 수 있습니다.
- **신뢰성:** 데이터의 3중 복제(triple replication)를 통해 데이터 손실을 방지합니다.

- **수평적 확장성:** 필요에 따라 노드를 추가하여 용량과 성능을 쉽게 확장할 수 있습니다.
- **다양한 데이터 유형 지원:** 정형, 반정형, 비정형 데이터를 모두 저장하고 처리할 수 있습니다.
- **오픈소스 생태계:** Hive, Presto 등 다양한 오픈소스 프로젝트가 Hadoop을 기반으로 발전했습니다.

한계:

- **복잡한 관리:** NameNode, YARN 등 여러 구성 요소를 관리하기 어렵습니다.
- **데이터 품질/신뢰성 문제:** 데이터 레이크의 유연성 때문에 데이터 거버넌스가 어려워질 수 있습니다.
- **느린 처리 속도:** MapReduce는 각 단계마다 디스크 I/O를 수행하여 느립니다.
- **잦은 업데이트에 부적합:** '한 번 쓰고 여러 번 읽기(write once, read many)'에 최적화되어 잦은 데이터 업데이트에는 비효율적입니다.

11.2. Spark의 특징

핵심 요약

Spark는 Hadoop의 MapReduce를 대체하는 빠르고 범용적인 분산 처리 엔진입니다. 인메모리(in-memory) 컴퓨팅과 DAG(Directed Acyclic Graph)를 활용하여 MapReduce보다 훨씬 빠른 속도를 제공합니다.

장점:

- **빠른 속도:** 데이터를 메모리에 캐싱하고 DAG를 사용하여 디스크 I/O를 최소화함으로써 Hadoop MapReduce보다 100배 이상 빠릅니다.
- **통합 분석 엔진:** 배치 처리, 스트리밍, SQL 쿼리, 머신러닝, 그래프 처리 등 다양한 워크로드를 하나의 엔진에서 지원합니다.
- **다국어 지원:** Scala(원래 언어), Python, Java, R 등 다양한 프로그래밍 언어를 지원합니다.
- **쉬운 배포:** Hadoop과 독립적으로 실행 가능하며, 다양한 클러스터 관리자(YARN, Mesos, Kubernetes)와 클라우드 환경에서 쉽게 배포됩니다.
- **수평적 확장성:** Hadoop과 마찬가지로 노드를 추가하여 쉽게 확장할 수 있습니다.

Spark 아키텍처 (Hadoop과 유사):

- **Driver Program (드라이버 프로그램):** Spark 애플리케이션의 메인 함수를 실행하고, 클러스터의 Worker 노드에 작업을 분배합니다.
- **Cluster Manager (클러스터 관리자):** Worker 노드를 관리하고 리소스를 할당합니다.
- **Worker Nodes (워커 노드):** Driver로부터 작업을 받아 실행하며, Executor를 통해 CPU와 메모리를 사용합니다.
- **Executor (실행자):** Worker 노드에서 실제 작업을 수행하는 프로세스입니다.

11.3. Hadoop vs Spark 비교

특성	Hadoop (MapReduce)	Spark
처리 방식	디스크 기반 (각 Map/Reduce 단계마다 디스크 I/O)	인메모리 기반 (DAG를 통해 디스크 I/O 최소화)
속도	느림	빠름 (MapReduce 대비 100배 이상)
데이터 흐름	선형 (Map → Shuffle → Reduce)	DAG (Directed Acyclic Graph) 기반의 유연한 처리
데이터 재사용	각 작업 후 디스크에 저장, 재사용 어려움	메모리에 RDD/DataFrame 캐싱, 재사용 용이
지원 언어	Java (주로)	Scala, Python, Java, R 등 다국어 지원
사용 사례	대규모 배치 처리, 데이터 웨어하우징	배치, 스트리밍, SQL, ML, 그래프 처리 등 범용
복잡성	여러 컴포넌트 관리 (HDFS, YARN, MapReduce)	단일 통합 엔진, 상대적으로 단순한 관리

핵심 정보

RDD (Resilient Distributed Dataset): Spark의 기본 데이터 추상화 단위입니다. 불변(immutable)하며, 분산된 데이터 컬렉션을 나타냅니다. **DataFrame (데이터프레임):** RDD 위에 구축된 고수준 추상화로, 구조화된 데이터를 다루기 쉽고 최적화된 성능을 제공합니다. SQL과 유사한 방식으로 데이터를 조작할 수 있습니다.

12. 클라우드 컴퓨팅 모델 (IaaS, PaaS, SaaS)

클라우드 서비스는 제공하는 추상화 수준에 따라 세 가지 모델로 나뉩니다.

모델	설명	관리 책임	예시
IaaS (Infrastructure as a Service)	가상 머신, 스토리지, 네트워크 등 기본 인프라 제공	사용자가 OS, 미들웨어, 애플리케이션 관리	AWS EC2, Azure Virtual Machines
PaaS (Platform as a Service)	애플리케이션 개발 및 배포를 위한 플랫폼 제공	사용자가 애플리케이션만 관리, 인프라/OS는 클라우드 제공자 관리	AWS Elastic Beanstalk, Azure App Service
SaaS (Software as a Service)	완제품 소프트웨어를 서비스 형태로 제공	클라우드 제공자가 모든 것을 관리, 사용자는 소프트웨어 사용	Gmail, Salesforce, Databricks

예제

Databricks는 SaaS의 좋은 예시입니다. Databricks는 AWS, Azure, GCP와 같은 클라우드 인프라 위에 구축된 소프트웨어 서비스입니다. 사용자는 Databricks 플랫폼에 접속하여 데이터 파이프라인을 구축하고 분석을 수행할 수 있으며, 하부 인프라(서버, OS 등) 관리에 대해 걱정할 필요가 없습니다. 이는 사용자가 핵심 비즈니스 로직에 집중할 수 있게 해줍니다.

13. 빅데이터 생태계의 주요 도전 과제

빅데이터를 효과적으로 활용하는 데에는 여러 가지 어려움이 따릅니다.

- **데이터 품질 및 신선도 (Data Quality & Staleness):**
 - 데이터가 부정확하거나 불완전하면 잘못된 통찰로 이어집니다.
 - 데이터가 너무 오래되면 그 가치가 떨어집니다 (데이터 온도 개념).
- **데이터 사일로 (Data Silos):**
 - 조직 내 여러 부서가 각자의 데이터를 독립적으로 관리하여 통합된 시야를 확보하기 어렵습니다.
 - 서로 다른 기술 스택을 사용하여 데이터 공유 및 활용이 어렵습니다.
- **파편화된 도구 (Fragmented Tools):**
 - 다양한 클라우드 서비스, BI 도구, ML 프레임워크가 혼재하여 호환성 문제가 발생하고 생산성을 저해합니다.
- **기술 부족 (Shortage of Skills):**

- 데이터를 다루고 최신 도구를 활용할 수 있는 숙련된 인력이 부족합니다.

이러한 도전 과제들은 결국 부정확한 통찰과 지연된 결과로 이어져 데이터의 가치를 떨어뜨립니다.

14. 빅데이터 플랫폼을 위한 모범 사례

효과적인 빅데이터 플랫폼을 구축하고 운영하기 위한 몇 가지 핵심 원칙입니다.

- **결합도 낮은 시스템 구축 (Decoupled System):**
 - 스토리지와 컴퓨팅 자원을 분리하여 각각 독립적으로 확장하고 관리할 수 있도록 합니다. (예: S3와 Spark)
- **서비스 지향적 설계 (Service-Oriented):**
 - 재사용 가능한 모듈 형태로 파이프라인이나 애플리케이션을 구축하여 특정 기능에 집중하도록 합니다.
- **클라우드 스토리지 및 개방형 형식 활용 (Cloud Storage & Open Format):**
 - 클라우드의 저렴하고 확장 가능한 스토리지를 활용하고, Parquet, Delta Lake와 같은 개방형 데이터 형식을 사용하여 특정 벤더에 종속되는 것을 피합니다 (Vendor Lock-in 방지).
- **사용 사례에 맞는 도구 선택 (Right Tool for the Right Job):**
 - 모든 문제에 하나의 도구가 최선일 수는 없습니다. 지연 시간, 처리량, 접근 패턴 등 사용 사례의 요구사항을 고려하여 최적의 도구를 선택합니다.
- **로그 중심 설계 패턴 (Log-centric Design Patterns):**
 - 변경 불가능한(immutable) 로그를 사용하여 데이터 변경 이력을 추적하고, 필요 시 데이터를 재처리(replay)할 수 있도록 합니다. (메달리온 아키텍처의 Bronze 계층의 중요성)
- **소비자 요구에 따른 다양한 데이터 뷰 (Multiple Views of Data):**
 - 동일한 원시 데이터로부터 다양한 소비자(보고서, ML 모델, 애플리케이션)의 요구에 맞춰 여러 형태의 뷰를 제공합니다. (예: 마스킹된 데이터, 집계된 데이터)
- **구축 vs 구매 결정 (Build vs Buy):**
 - 시간, 비용, 내부 전문성, 시장 경쟁 상황 등을 고려하여 솔루션을 직접 구축할지(Build) 외부 솔루션을 구매할지(Buy) 결정합니다.

15. 기술 투자에 대한 비즈니스 정당성

기술 투자는 단순히 비용이 아니라 비즈니스 성장을 위한 혁신 센터로 간주되어야 합니다.

- **비즈니스 정당성 필수:** 새로운 기술을 도입할 때는 항상 명확한 비즈니스 목표와 정당성이 있어야 합니다. 단순히 '새로운 기술이라서' 도입하는 것은 실패할 가능성이 높습니다.
- **ROI (투자 수익률) 측정:** 모든 기술 투자는 측정 가능하고 가시적인 ROI를 가져와야 합니다. 핵심 성과 지표(KPI)를 설정하고 이를 통해 투자의 효과를 입증해야 합니다. **Capex vs Opex:**
 - **Capex (Capital Expenditure):** 자본 지출. 온프레미스 서버 구매와 같이 초기 대규모 투자가 필요합니다.
 - **Opex (Operational Expenditure):** 운영 지출. 클라우드 서비스 사용료와 같이 사용한 만큼 지불하는 방식입니다. 클라우드는 Opex 모델을 통해 초기 비용 부담을 줄여줍니다.
- **데이터 거버넌스 및 보호:** 데이터는 중요한 자산이므로, 부적절한 접근과 유출로부터 보호되어야 합니다. Unity Catalog와 같은 도구를 사용하여 접근 권한을 세밀하게 제어해야 합니다.

- **기술적 관점과 비즈니스 관점의 통합:** 현재 상태(Current State)와 목표 상태(Desired State)를 명확히 하고, 기술적 해결책이 비즈니스 문제를 어떻게 해결할지 설명해야 합니다.
 - **MEDDPICC 프레임워크 (Databricks 활용):**
 - **Metrics** (측정 지표): KPI
 - **Economic Buyer** (경제적 구매자): 누가 비용을 지불할 것인가?
 - **Decision Criteria** (결정 기준): 지연 시간, 비용, 복원력 등
 - **Decision Process** (결정 과정): 누구와 협력해야 하는가?
 - **Pain** (문제점): 현재 해결되지 않는 비즈니스 문제
 - **Implicate** (영향): 문제 해결 시 비즈니스에 미칠 긍정적 영향
 - **Champion** (옹호자): 프로젝트를 지지하는 내부 인물
 - **Competition** (경쟁): 경쟁사 대비 우위 확보
- 이 프레임워크는 기술 투자의 비즈니스 가치를 명확히 하는 데 도움이 됩니다.

절차/방법: Databricks Free Edition 시작하기 (Lab 00)

Lab 00은 Databricks Free Edition 환경에서 기본적인 데이터 처리 작업을 수행하는 방법을 소개합니다.

1. Databricks Free Edition 계정 생성 및 접속

1. Databricks Free Edition 웹사이트에 접속하여 계정을 생성합니다. (일반 Trial 계정과 혼동하지 않도록 주의)
2. 로그인 후, 화면 좌측 상단에 "Databricks Free Edition" 문구가 표시되는지 확인합니다. 이 문구가 없으면 Trial 계정일 가능성이 높습니다.

주의

Trial 계정 vs Free Edition 계정: Databricks는 두 가지 유형의 무료 계정을 제공합니다.

- **Trial Account (평가판 계정):** 14일 동안 모든 기능을 사용할 수 있지만, 이후에는 유료 전환이 필요합니다.
- **Free Edition Account (무료 에디션 계정):** 학술적 목적으로 영구적으로 무료이며, 일부 기능 제한이 있습니다. 이 수업에서는 Free Edition을 사용합니다.

만약 "Databricks Free Edition" 문구가 보이지 않는다면, Trial 계정일 수 있으므로 다시 Free Edition 계정을 생성해야 합니다.

2. Lab 00 노트북 가져오기 (Import)

1. 제공된 Google Drive 링크에서 Lab 00 zip 파일을 다운로드합니다.
2. Databricks Workspace 좌측 패널에서 **Workspace**를 클릭합니다.
3. 본인의 사용자 이름 아래에 폴더를 생성합니다 (예: labs).
 - 사용자 이름 클릭 → **Create** → **Folder** → 이름 입력 (예: labs)
4. 생성된 폴더 안에서 마우스 오른쪽 버튼 클릭 → **Import**를 선택합니다.

5. 다운로드한 zip 파일을 드래그 앤 드롭하여 가져옵니다. Databricks가 자동으로 압축을 해제하고 노트북 파일들을 생성합니다.
6. 가져오기가 완료되면 lab00 폴더 안에 readme, initialize, intro_basics, read_write, spark_intro 5개의 노트북이 보일 것입니다.

3. 컴퓨트(Compute) 연결 및 노트북 실행

Databricks Free Edition은 서버리스(Serverless) 환경이므로, 별도의 클러스터를 생성할 필요 없이 컴퓨트 자원에 연결하기만 하면 됩니다.

1. initialize 노트북을 엽니다.
2. 노트북 상단의 **Connect** 버튼을 클릭하고 **Serverless**를 선택합니다.
3. 연결이 완료되면, 첫 번째 셀을 실행합니다. (셀 좌측의 실행 버튼 클릭 또는 Ctrl+Enter)
4. 첫 번째 셀은 카탈로그, 스키마, 볼륨을 생성하는 명령을 포함합니다.

```

1 CREATE CATALOG IF NOT EXISTS csci103_catalog;
2 USE CATALOG csci103_catalog;
3
4 CREATE SCHEMA IF NOT EXISTS lab00;
5 USE SCHEMA lab00;
6
7 CREATE VOLUME IF NOT EXISTS lab00_input;
8 CREATE VOLUME IF NOT EXISTS lab00_output;
9

```

Listing 1.1: initialize 노트북의 첫 번째 셀 예시

5. 실행이 성공하면, 좌측 패널의 **Catalog** 아이콘을 클릭하여 생성된 csci103_catalog와 그 아래의 lab00 스키마, 그리고 lab00_input, lab00_output 볼륨을 확인할 수 있습니다.

핵심 정보

Databricks의 메타데이터 계층 구조:

- **Catalog (카탈로그):** 최상위 메타데이터 컨테이너. 데이터베이스 서버와 유사합니다.
- **Schema (스키마):** 카탈로그 아래의 논리적 그룹. 전통적인 데이터베이스의 스키마 또는 데이터베이스와 유사합니다.
- **Table (테이블):** 스키마 아래에 저장되는 실제 데이터 테이블.
- **Volume (볼륨):** 클라우드 스토리지 내의 특정 경로를 가리키는 논리적 단위. 파일 시스템 경로를 추상화합니다.

4. Databricks 노트북 기본 사용법

- **셀 실행:** 셀 좌측의 실행 버튼 또는 Ctrl+Enter.
- **언어 전환 (Magic Commands):**
 - %python: Python 코드 실행 (기본값)
 - %sql: SQL 쿼리 실행

- %sh: 셸(Shell) 명령어 실행 (예: ls, top)
- %fs: Databricks 파일 시스템(DBFS) 명령어 실행 (예: ls, cp)
- %md: Markdown 텍스트 작성
enditemize
- 데이터셋 탐색:
 - * %fs ls /databricks-datasets: Databricks에서 제공하는 공개 데이터셋 목록 확인.
 - * dbutils.fs.ls("경로"): Python에서 파일 시스템 목록 확인.
 - * display(dbutils.fs.ls("경로")): 결과를 표 형태로 예쁘게 출력.

예제

다양한 언어 셸 예시:

```

1 print("Hello from Python!")
2 my_variable = "Python variable"
3 print(my_variable)

```

Listing 1.2: Python 셸 예시

```

1 %sql
2 SELECT "Hello from SQL!" AS greeting;
3 SELECT current_user() AS current_user;

```

Listing 1.3: SQL 셸 예시

```

1 %sh
2 ls -l /databricks-datasets/

```

Listing 1.4: Shell 셸 예시

```

1 %md
2 # Markdown 제목
3 \textbf굵은{ 글씨와} 기울임* 글씨를* 사용할 수 있습니다.
4 - 목록 1
5 - 목록 2

```

Listing 1.5: Markdown 셸 예시

5. 데이터프레임 생성 및 조작

Spark의 핵심은 데이터프레임을 사용하여 데이터를 처리하는 것입니다.

1. 데이터 읽기:

```

1 file_path = "/databricks-datasets/bikeSharing/data-001/201508_trip_data.
   csv"
2 df = spark.read.format("csv") \
3     .option("header", "true") \
4     .option("inferSchema", "true") \
5     .load(file_path)
6 df.printSchema()

```

```

7 df.show(5)
8

```

Listing 1.6: CSV 파일 읽기 예시

- `format("csv")`: 파일 형식 지정.
- `option("header", "true")`: 첫 줄을 헤더로 인식.
- `option("inferSchema", "true")`: Spark가 데이터 타입을 자동으로 추론. (정확한 스키마를 알고 있다면 수동으로 정의하는 것이 더 좋습니다.)
- `printSchema()`: 데이터프레임의 스키마(컬럼 이름과 타입) 출력.
- `show(n)`: 데이터프레임의 상위 n개 행 출력.

2. 스키마 수동 정의:

```

1 from pyspark.sql.types import StructType, StructField, StringType,
  IntegerType
2
3 custom_schema = StructType([
4     StructField("trip_id", IntegerType(), True),
5     StructField("duration", IntegerType(), True),
6     StructField("start_date", StringType(), True),
7     # ... 다른 컬럼들
8 ])
9 df_custom = spark.read.format("csv") \
10     .option("header", "true") \
11     .schema(custom_schema) \
12     .load(file_path)
13 df_custom.printSchema()
14

```

Listing 1.7: 스키마 수동 정의 예시

- `schema(custom_schema)`: Spark의 스키마 추론 대신 사용자가 정의한 스키마를 적용.

3. 데이터 쓰기 (다양한 형식):

```

1 output_path_parquet = "/Volumes/csci103_catalog/lab00/lab00_output/
  bike_data_parquet"
2 df.write.format("parquet").mode("overwrite").save(output_path_parquet)
3

```

Listing 1.8: 데이터를 Parquet 형식으로 쓰기 예시

- `mode("overwrite")`: 이미 파일이 존재하면 덮어씁니다. ("append", "ignore", "errorifexists" 등)
- `save(경로)`: 지정된 경로에 데이터를 저장합니다.
- **Delta Lake 형식으로 쓰기**: Databricks의 기본이자 권장 형식입니다.

```

1 output_path_delta = "/Volumes/csci103_catalog/lab00/lab00_output/
  bike_data_delta"
2 df.write.format("delta").mode("overwrite").save(output_path_delta)
3

```

Listing 1.9: 데이터를 Delta 형식으로 쓰기 예시

4. 테이블 생성 및 등록 (Unity Catalog):

```

1 %sql
2 CREATE TABLE IF NOT EXISTS bike_trips_delta
3 USING DELTA
4 LOCATION '/Volumes/csci103_catalog/lab00/lab00_output/bike_data_delta';
5
6 -- 또는 데이터프레임을 사용하여 테이블로 저장
7 -- df.write.format("delta").mode("overwrite").saveAsTable("
   bike_trips_df_table")
8

```

Listing 1.10: Delta 테이블 생성 및 등록 예시

- CREATE TABLE ... USING DELTA LOCATION ...: 기존 Delta 형식으로 저장된 데이터를 Unity Catalog에 테이블로 등록합니다.
- saveAsTable("테이블_이름"): 데이터프레임을 직접 Unity Catalog에 테이블로 등록합니다. 이 경우 Databricks가 스토리지 경로를 관리합니다 (Managed Table).
- 테이블이 등록되면 **Catalog Explorer**에서 해당 테이블의 스키마, 히스토리, 권한 등을 확인할 수 있습니다.

5. 컬럼 추가/삭제/변환:

```

1 from pyspark.sql.functions import col, lit
2
3 # 새로운 컬럼 추가
4 df_with_new_col = df.withColumn("new_column", lit("some_value"))
5
6 # 기존 컬럼 이름 변경
7 df_renamed = df_with_new_col.withColumnRenamed("duration", "
   trip_duration_seconds")
8
9 # 컬럼 삭제
10 df_dropped = df_renamed.drop("start_station_id", "end_station_id")
11
12 df_dropped.printSchema()
13

```

Listing 1.11: 데이터프레임 컬럼 조작 예시

6. 사용자 정의 함수 (UDF) 사용:

```

1 from pyspark.sql.functions import udf
2 from pyspark.sql.types import IntegerType
3
4 # 을 duration 분 단위로 변환하는 UDF 정의
5 def seconds_to_minutes(seconds):
6     return seconds // 60
7
8 seconds_to_minutes_udf = udf(seconds_to_minutes, IntegerType())
9
10 # 를 UDF 사용하여 새로운 컬럼 생성

```

```

11 df_with_minutes = df.withColumn("trip_duration_minutes",
    seconds_to_minutes_udf(col("duration")))
12 df_with_minutes.select("duration", "trip_duration_minutes").show(5)
13

```

Listing 1.12: 사용자 정의 함수 (UDF) 예시

실습/코드/오류와 해결

1. Databricks UI 탐색 팁

- **좌측 패널:** Workspace (노트북, 폴더), Catalog (데이터, 테이블, 볼륨), Compute (클러스터 관리), Workflow (작업 스케줄링) 등 주요 기능에 접근합니다.
- **노트북 상단:** 언어 선택 (Python, SQL 등), 컴퓨트 연결/해제, 셀 실행 옵션 (모든 셀 실행, 선택 셀 실행 등).
- **셀 출력:** 셀 실행 후 결과는 셀 아래에 표시됩니다. 표, 그래프, 텍스트 등 다양한 형태로 나타납니다.
- **쿼리 프로파일 (Query Profile):** Spark 쿼리 실행 후 셀 하단의 'Performance' 탭을 클릭하면 쿼리 실행 계획, 병목 현상 등을 시각적으로 확인할 수 있습니다. 이는 Spark UI의 Databricks 버전입니다.

2. Lab 00에서 발생할 수 있는 일반적인 오류 및 해결책

증상	원인 가설	수정 방법
Notebook failed to load - 네트워크 문제 또는 Databricks 서비스 일시적 오류 - 인터넷 연결 확인 후 새로고침 또는 잠시 후 재시도 Permission denied 또는 Catalog not found - 다른 사용자의 카탈로그/스키마에 접근 시도 - Free Edition이 아닌 Trial 계정 사용 - 본인의 카탈로그/스키마를 사용하거나, 다른 사용자에게 권한 요청 - Free Edition 계정으로 다시 생성 및 로그인 Command failed 또는 Error in SQL statement - 컴퓨트(Serverless)에 연결되지 않음 - 데이터 경로 오류 - 노트북 상단의 Connect 버튼을 클릭하여 Serverless 컴퓨트에 연결 - 파일 경로나 올바른지 확인 (특히 /Volumes/catalog/schema/volume 형식) DBFS writing data to DBFS is not supported - dbutils.fs.put() 대신 df.write.save() 또는 df.write.saveAsTable() 사용.	- Free Edition이 아닌 Trial 계정 사용 - Databricks Free Edition 계정으로 다시 생성 및 로그인 - initialize 노트북을 실행하지 않음 - initialize 노트북을 먼저 실행하여 카탈로그/스키마/볼륨 생성 - 구문 오류 (SQL, Python 등) - 코드의 오타, 문법 오류 확인	/Volumes/csci103_catalog/lab00/lab00_output/bike_data_delta와 같은 /Volumes 경로를 통해 데이터를 저장합니다.

3. Spark의 실행 계획 확인 (explain())

Spark는 코드를 즉시 실행하지 않고, 최적화된 실행 계획을 세운 후 실행합니다. `explain()` 함수를 사용하여 이 계획을 확인할 수 있습니다.

```

1 # 이전 예시에서 생성된 데이터프레임 df_with_minutes 사용
2 df_with_minutes.explain(extended=True)

```

Listing 1.13: Spark 실행 계획 확인 예시

- **explain():** Spark가 데이터프레임 작업을 어떻게 수행할지 논리적/물리적 실행 계획을 보여줍니다.
- **extended=True:** 더 상세한 정보를 포함합니다.
- **Photon:** Databricks의 고성능 쿼리 엔진으로, Spark 작업을 가속화합니다. `explain()` 결과에서 Photon이 지원하는 작업과 지원하지 않는 작업을 확인할 수 있습니다.

체크리스트

1. 과제 제출 전 점검표

- 마감 기한 준수: 모든 과제는 마감일 자정까지 제출해야 합니다. (업로드 문제 대비 최소 5분 전 제출 권장)
- 지연 정책 확인: 지연 제출 시 하루당 2점 감점, 최대 10점 감점됩니다. (늦더라도 제출하는 것이 중요)
- 개별 제출: 모든 과제는 개별 제출입니다. (그룹 프로젝트는 별도 명시)
- 노트북 내보내기: 완료된 노트북을 Canvas에 업로드할 수 있도록 내보내기 (export) 했는지 확인합니다.
- 초기 시작: 과제는 가능한 한 빨리 시작하여 충분한 학습 시간을 확보합니다.
- 질문 활용: 막히는 부분이 있다면 Slack, Office Hours를 통해 질문합니다. 마감 직전까지 기다리지 않습니다.

2. 학습 점검표

- 데이터 엔지니어링의 정의와 중요성을 설명할 수 있는가?
- 빅데이터의 5V 특성을 각각 설명하고 예시를 들 수 있는가?
- 데이터 페르소나(엔지니어, 과학자, 분석가 등)의 역할을 구분할 수 있는가?
- SQL(ACID)과 NoSQL(BASE) 시스템의 차이점과 사용 사례를 비교할 수 있는가?
- CAP 이론을 설명하고, 시스템 설계 시의 트레이드오프를 이해하는가?
- OLTP와 OLAP 시스템의 목적과 특징을 구분할 수 있는가?
- Hadoop과 Spark의 주요 차이점(속도, 처리 방식 등)을 설명할 수 있는가?
- IaaS, PaaS, SaaS 클라우드 모델의 차이점을 설명하고 예시를 들 수 있는가?
- Databricks Free Edition 환경에서 노트북 가져오기, 컴퓨터 연결, 셀 실행을 할 수 있는가?
- Databricks에서 카탈로그, 스키마, 볼륨을 생성하고 탐색할 수 있는가?
- Spark 데이터프레임을 사용하여 CSV, JSON, Parquet, Delta 파일을 읽고 쓸 수 있는가?
- Spark 데이터프레임에서 컬럼 추가/삭제/변환 및 UDF를 사용할 수 있는가?

FAQ (자주 묻는 질문)

- Q: 최종 발표는 개인 발표인가요, 그룹 발표인가요? A: 최종 발표는 그룹 프로젝트의 일환으로 진행되는 그룹 발표입니다. 각 팀원은 데이터 엔지니어링, 데이터 과학 등 특정 페르소나 역할을 맡아 프로젝트의 해당 측면을 발표하게 됩니다. 이는 실제 업무 환경을 반영한 것입니다.
- Q: 강의 섹션(Section)에도 반드시 참석해야 하나요? A: 섹션은 과제 지원 및 강의 자료 복습에 중점을 둡니다. 라이브 참석이 권장되지만, 시간대 문제 등으로 참석이 어렵다면 녹화본이 제공되므로 반드시 시청해야 합니다. 특히 과제 리뷰가 있는 주에는 시청이 필수적입니다.
- Q: 교재는 구매해야 하나요? A: 두 권의 교재(*Simplifying Data Engineering and Analytics with Delta, Spark, the Definitive Guide*) 모두 하버드 도서관을 통해 전자책 형태로 무료로 이용할 수 있습니다. 온라인 구매도 가능하지만 필수는 아닙니다.

- **Q: Databricks Free Edition 계정 생성 시 오류가 발생했습니다. A:** "Notebook failed to load" 와 같은 오류는 주로 Databricks Free Edition이 아닌 일반 Trial 계정을 생성했을 때 발생합니다. Free Edition 계정 생성 링크를 다시 확인하고, 계정 생성 후 Databricks UI 좌측 상단에 "Databricks Free Edition" 문구가 표시되는지 꼭 확인하세요.

빠르게 훑어보기 (1페이지 요약)

핵심 요약

CSCI103 Lecture 1 핵심 요약

데이터 엔지니어링: 원시 데이터를 가치 있는 통찰로 변환하는 핵심 역할. (수집 → 변환 → 저장 → 제공)

빅데이터 5V: **Volume** (규모), **Velocity** (속도), **Variety** (다양성), **Veracity** (정확성), **Value** (가치).

데이터 페르소나: 엔지니어, 과학자, 분석가, DevOps, 리더 등 다양한 역할의 협업.

데이터 온도: Hot (실시간), Warm (인터랙티브), Cold (배치) 데이터에 따라 처리 속도 결정.

데이터 플랫폼 진화: DBMS → DW → Hadoop → Spark → 레이크하우스.

SQL (ACID) vs NoSQL (BASE):

- * **SQL:** 일관성, 신뢰성 중시 (금융, 거래). ACID (원자성, 일관성, 격리성, 영속성).
- * **NoSQL:** 유연성, 확장성, 가용성 중시 (웹, 소셜). BASE (기본적 가용성, 유연한 상태, 최종 일관성).

CAP 이론: 분산 시스템은 일관성(C), 가용성(A), 분할 내성(P) 중 두 가지만 만족 가능.

OLTP vs OLAP:

- * **OLTP:** 실시간 트랜잭션 처리 (예: 은행 거래).
- * **OLAP:** 대규모 데이터 분석 및 보고 (예: 판매 보고서).

Hadoop vs Spark:

- * **Hadoop:** 디스크 기반, 느림, 데이터 레이크 시작.
- * **Spark:** 인메모리 기반, 빠름, 범용 분석 엔진 (배치, 스트리밍, ML, SQL).

클라우드 모델: IaaS (인프라), PaaS (플랫폼), SaaS (소프트웨어). Databricks는 SaaS.

Databricks Lab 00:

- * Free Edition 계정 생성 및 접속 확인.
- * 노트북 가져오기, Serverless 컴퓨트 연결.
- * 카탈로그, 스키마, 볼륨 생성 (initialize 노트북).
- * %sql, %python, %sh, %fs, %md 등 매직 명령어 활용.
- * Spark 데이터프레임으로 데이터 읽기/쓰기 (CSV, JSON, Parquet, Delta).
- * CREATE TABLE 또는 saveAsTable()로 테이블 등록.
- * 컬럼 조작, UDF 사용, explain()으로 실행 계획 확인.

모범 사례: 스토리지/컴퓨트 분리, 오픈 형식, 사용 사례에 맞는 도구, 로그 중심 설계.

비즈니스 정당성: 기술 투자는 ROI를 통해 비즈니스 가치를 입증해야 함 (MEDDPICC).

부록: 강의 운영 및 평가

1. 강의 운영 방식

- **강의 (Lectures):** 주 1회 진행되며, 이론 학습 (전반부)과 실습 (후반부)으로 구성됩니다. 라이브 Zoom 세션 또는 녹화본 시청이 가능합니다.
- **섹션 (Sections):** 목요일 오후 6시 7시 (미국 동부 시간)에 진행됩니다. 과제 지원, 강의 자료 복습, 과제 풀이 등에 중점을 둡니다. 녹화본이 제공됩니다.
- **오피스 아워 (Office Hours):** 교수진과의 1:1 세션으로, 과제나 학습에 어려움이 있을 때 도움을 받을 수 있습니다. Cali 링크를 통해 쉽게 예약 가능합니다.
- **주요 소통 채널:**
 - * **Canvas:** 강의 자료 (과제, 슬라이드, 강의 계획서, 일정), 녹화본 링크가 게시됩니다.
 - * **Slack:** 질문, 토론, 그룹 프로젝트 협업 등 상호작용적인 소통을 위한 채널입니다.

2. 학생 기대 사항

- **적극적인 참여 및 질문:** 궁금한 점은 주저하지 말고 질문합니다.
- **개방적인 학습 태도:** 새로운 아이디어와 기술에 대해 배우려는 의지를 가집니다.
- **예의 바르고 전문적인 태도:** 교수진 및 다른 학생들에게 항상 예의를 지키고 존중합니다.
- **그룹 프로젝트 참여:** 팀원들과 적극적으로 협력하고, 자신의 역할을 다하여 최대한의 학습 효과를 얻습니다. 팀원들과 일찍 소통하여 관계를 구축하는 것이 중요합니다.

3. 평가 및 채점 기준

평가 항목	설명
과제 (Assignments)	총 4개의 과제가 주어집니다. 강의 내용과 연관된 실습 위주입니다.
사례 연구 (Case Studies)	총 2개의 사례 연구가 있습니다.
퀴즈 (Quizzes)	총 2개의 퀴즈가 있습니다.
최종 발표 (Final Presentation)	그룹 프로젝트의 결과물을 발표합니다.
참여도 (Participation)	강의 중 질문, Slack 활동, 그룹 프로젝트 참여도 등이 반영됩니다.

주의

과제 마감 정책:

- 모든 과제는 마감일 자정 (midnight) 까지 제출해야 합니다.
- 업로드 문제에 대비하여 마감 시간 최소 5분 전에는 제출하는 것이 좋습니다.
- 지연 제출 시 하루당 2점 감점되며, 과제당 최대 10점까지 감점됩니다. (늦더라도 제출하면 부분 점수를 받을 수 있습니다.)
- 가능한 한 빨리 과제를 시작하여 충분한 시간을 가지고 학습하고 제출하는 것이 가장 좋은 방법입니다.

4. 학습 로드맵 (초심자 권장)

1. **기초 다지기:** 데이터 엔지니어링의 정의, 빅데이터 5V, 데이터 페르소나 등 핵심 개념을 먼저 이해합니다.
 2. **환경 설정:** Databricks Free Edition 계정을 성공적으로 생성하고 Lab 00 노트북을 실행하여 기본적인 환경에 익숙해집니다.
 3. **데이터 처리 기본:** Spark 데이터프레임을 사용하여 데이터 읽기, 쓰기, 간단한 변환 작업을 반복적으로 실습합니다.
 4. **개념 심화:** 데이터 플랫폼의 진화, SQL/NoSQL, OLTP/OLAP, CAP 이론 등 이론적 배경을 학습하여 '왜' 이러한 기술이 필요한지 이해합니다.
 5. **과제 해결:** 각 과제를 통해 배운 이론과 실습 지식을 적용하고, 막히는 부분은 적극적으로 질문하여 해결합니다.
 6. **프로젝트 준비:** 그룹 프로젝트를 통해 실제 비즈니스 문제를 해결하는 과정을 경험하고, 다양한 데이터 페르소나의 역할을 수행해봅니다.
-

Chapter 2

용어 정리

강의명: CSCI103

주차: Lecture 2

교수명: UIUC Student

목적: 이 문서는 CSCI103 Lecture 2 강의 자료를 통합하여 작성된 시험 대비용 LaTeX 노트입니다. 데이터 모델링, 메타데이터, 데이터 형식, 압축, 프로파일링 등 핵심 개념을 다룹니다. 특히, OLTP와 OLAP 시스템의 차이, 다양한 데이터 모델링 기법(스타, 스노우플레이크, 데이터 볼트), 그리고 데이터브릭스 환경에서의 실습(Lab 01) 과정을 상세히 설명합니다. 비전공자도 쉽게 이해할 수 있도록 배경 설명, 비유, 예시를 풍부하게 포함하며, 과제 및 시험 준비에 필요한 모든 정보를 제공합니다.

2.1 용어 정리

다음 표는 강의에서 다루는 주요 용어들을 쉽게 설명하고 원어와 함께 정리한 것입니다.

2.2 핵심 개념 및 원리

2.2.1 데이터의 복잡성과 모델링의 필요성

매일 2.5 쿼틸리언(10^{18}) 바이트의 데이터가 생성되며, 이는 계속 증가하고 있습니다. 문서, 소셜 미디어, 클라우드, 웹사이트, IoT 기기, 데이터베이스, 사용자 프로파일 등 다양한 소스에서 데이터가 발생합니다. 이처럼 방대하고 다양한 데이터를 효과적으로 관리하고 활용하기 위해서는 데이터 모델링이 필수적입니다.

데이터 모델링이란?

데이터 모델링은 데이터를 비즈니스 프로세스 관점에서 조직화하고, 데이터 간의 논리적 관계를 정의하여 비즈니스를 지원하는 과정입니다. 이는 데이터의 구조를 시각적으로 표현하여 이해를 돕고, 데이터 저장 및 검색 효율성을 극대화하는 데 목적이 있습니다.

Table 2.1: 주요 용어 정리

용어 (Term)	쉬운 설명 (Easy Explanation)	원어 (Original)	비고 (Notes)
ACID	데이터베이스 트랜잭션의 안정성을 보장하는 네 가지 속성	Atomicity, Consistency, Isolation, Durability	주로 관계형 데이터베이스와 연관
BASE	분산 시스템의 가용성과 확장성을 강조하는 속성	Basically Available, Soft state, Eventual consistency	NoSQL 시스템에서 주로 사용
CAP 이론	분산 시스템에서 일관성, 가용성, 분할 내성 중 두 가지만 선택 가능	Consistency, Availability, Partition tolerance	관계형 DB는 C, NoSQL은 A 선호
IaaS	가상 서버, 스토리지 등 IT 인프라를 서비스로 제공	Infrastructure as a Service	클라우드 컴퓨팅 모델
PaaS	개발 환경 및 플랫폼을 서비스로 제공	Platform as a Service	클라우드 컴퓨팅 모델
SaaS	소프트웨어 애플리케이션을 서비스로 제공	Software as a Service	클라우드 컴퓨팅 모델
레이크하우스	데이터 웨어하우스와 데이터 레이크의 장점을 결합한 아키텍처	Lakehouse	데이터브릭스의 핵심 개념
DAG	작업의 의존성을 방향성 있는 그래프로 표현	Directed Acyclic Graph	스파크 작업 스케줄링에 사용
ETL	데이터를 추출, 변환, 로드하는 과정	Extract, Transform, Load	데이터 통합의 기본 과정
스파크 (Spark)	대규모 데이터 처리를 위한 분산 컴퓨팅 프레임워크	Spark	하둡보다 100배 빠름 (메모리 사용)
폴리글랏 (Polyglot)	여러 프로그래밍 언어를 지원하는 능력	Polyglot	스파크가 여러 언어(Python, Scala 등) 지원
5 Vs of Big Data	빅데이터의 특징을 나타내는 다섯 가지 요소	Volume, Velocity, Veracity, Variety, Value	빅데이터의 본질 이해에 중요
Veracity	데이터의 정확성, 신뢰성, 유효성	Veracity	데이터 품질과 직결
개념적 데이터 모델	비즈니스 관점에서 데이터의 큰 그림을 정의	Conceptual Data Model	비즈니스 사용자 이해에 초점
논리적 데이터 모델	데이터 구조, 관계, 속성 유형을 상세히 정의	Logical Data Model	기술적 구현 전 단계
물리적 데이터 모델	실제 데이터 저장 방식에 맞춰 모델을 정의	Physical Data Model	특정 데이터베이스 기술에 의존
NoSQL	관계형 데이터베이스 외의 다양한 데이터 저장 방식	Not only SQL	유연한 스키마, 수평 확장성
메타데이터	데이터에 대한 데이터	Metadata	데이터 이해, 관리, 활용에 필수
데이터 카탈로그	메타데이터를 저장하고 관리하는 기술적 리소스	Data Catalog	데이터 검색, 거버넌스 지원
OLTP	온라인 트랜잭션 처리 시스템	Online Transaction Processing	실시간 트랜잭션, 빠른 읽기/쓰기
OLAP	온라인 분석 처리 시스템	Online Analytical Processing	대규모 데이터 분석, 복잡한 쿼리
스타 스키마	중앙의 팩트 테이블과 주변의 차원 테이블로 구성	Star Schema	분석 쿼리 성능 최적화
스노우플레이크 스키마	차원 테이블이 다른 차원 테이블을 참조하는 계층 구조	Snowflake Schema	데이터 정규화 수준 높음, 복잡성 증가
데이터 볼트	데이터 웨어하우스 모델링 기법 (허브, 링크, 위성)	Data Vault	변경 관리 및 감사에 강점
데이터 컨포먼스	데이터의 일관성과 표준화를 보장하는 과정	Data Conformance	분석 품질 향상에 필수
델타 레이크	파케이 파일 위에 ACID 트랜잭션 등을 추가한 프로토콜	Delta Lake	데이터브릭스의 핵심 데이터 형식
데이터 압축	정보 인코딩에 더 적은 비트를 사용하는 과정	Data Compression	저장 공간 절약, I/O 대역폭 감소
데이터 프로파일링	데이터의 품질, 구조, 관계를 분석하는 과정	Data Profiling	데이터 품질 문제 식별
RMSE	예측값과 실제값 간의 평균 오차 크기를 측정하는 지표	Root Mean Square Scaled Error	모델 성능 평가 지표

데이터 모델링의 3단계

데이터 모델링은 일반적으로 세 가지 추상화 수준으로 진행됩니다.

1. 개념적 데이터 모델 (Conceptual Data Model):

- **한 줄 요약:** 비즈니스 사용자가 이해할 수 있는 수준에서 데이터의 큰 그림과 핵심 비즈니스 개념을 정의합니다.
- **직관적 예시:** 은행 업무를 예로 들면, '고객', '계좌', '거래'와 같은 엔티티(개체)와 이들 간의 관계를 정의하는 것입니다. 고객이 계좌를 가지고 있고, 계좌에서 거래가 발생한다는 식입니다.
- **기술적 설명:** 특정 기술에 종속되지 않고, 비즈니스 요구사항을 기반으로 엔티티, 속성, 관계를 식별합니다. 이는 데이터의 의미(시맨틱)를 정의하는 데 중점을 둡니다.

2. 논리적 데이터 모델 (Logical Data Model):

- **한 줄 요약:** 개념적 모델을 더 구체화하여 데이터의 구조, 관계, 속성 유형을 상세하게 정의합니다.
- **직관적 예시:** '고객' 엔티티에 '고객 이름(문자열)', '고객 번호(정수)'와 같은 속성을 추가하고, 각 속성의 데이터 타입을 명시합니다. '고객'과 '주소'가 1대 다 관계를 가질 수 있음을 정의합니다.
- **기술적 설명:** 특정 데이터베이스 관리 시스템(DBMS)에 독립적이지만, 관계형 모델의 테이블, 컬럼, 기본 키, 외래 키, 관계 등의 개념을 사용하여 데이터를 표현합니다.

3. 물리적 데이터 모델 (Physical Data Model):

- **한 줄 요약:** 논리적 모델을 실제 데이터 저장 기술(예: 관계형 데이터베이스)에 맞춰 구체적으로 구현하는 모델입니다.
- **직관적 예시:** '고객'과 '주소'의 다대다 관계를 처리하기 위해 '고객-주소 연결'이라는 중간 테이블(조인 테이블)을 추가하고, 각 테이블의 인덱스, 파티셔닝 전략 등을 정의합니다. 주소 유형을 나타내는 '사용 코드'와 같은 추가 필드를 포함할 수 있습니다.
- **기술적 설명:** 특정 DBMS의 특징(데이터 타입, 인덱스, 제약 조건, 저장 방식 등)을 반영하여 테이블, 컬럼, 인덱스, 뷰 등을 정의합니다. 성능 최적화를 위한 물리적 설계 요소가 포함됩니다.

모델 단계	설명 및 예시
개념적 모델	비즈니스 관점: 고객이 제품을 구매하고, 고객은 주소를 가짐. 예시: 고객(이름, 주소) – 구매 → 제품(이름, 가격)
논리적 모델	구조 및 관계 정의: 고객 이름(문자열), 고객 번호(정수) 등 속성 타입 명시. 예시: 고객(고객번호:PK, 고객이름:VARCHAR, 주소ID:FK) – 1:N → 주소(주소ID:PK, 상세주소:VARCHAR)
물리적 모델	저장 기술 구현: 관계형 DB 테이블, 인덱스, 조인 테이블 추가. 예시: CUSTOMER 테이블(고객번호:PK, 이름), PRODUCT 테이블(제품ID:PK, 이름), CUSTOMER_ADDRESS 조인 테이블(고객번호:FK, 주소ID:FK, 사용코드:CHAR(2))

Figure 2.1: 데이터 모델링 3단계 요약

데이터 모델링의 이점

- **데이터 가치 극대화:** 데이터의 속성과 관계를 이해하여 최대한의 가치를 얻을 수 있습니다.
- **시각화:** 데이터 구조를 시각적으로 표현하여 이해를 돕고, 불일치나 오류를 쉽게 발견하고 수정할 수 있습니다.
- **공통 어휘 및 문서화:** 조직 내 모든 구성원이 데이터를 동일한 방식으로 참조할 수 있는 공통 언어를 제공하여 혼란을 방지합니다.
- **조기 오류 수정:** 모델링 단계에서 문제를 식별하고 수정함으로써, 개발 후반부에 발생하는 비용을 절감합니다.
- **효율적인 데이터 저장 및 검색:** 데이터를 가장 효율적인 방식으로 저장하고 검색할 수 있도록 설계합니다.

2.2.2 NoSQL 데이터 모델링

전통적인 관계형 데이터베이스(SQL)와 달리 NoSQL 데이터베이스는 다양한 데이터 저장 방식을 제공하며, 이에 따라 데이터 모델링 방식에도 차이가 있습니다.

NoSQL 데이터 저장 유형

- **키-값 저장소 (Key-Value Stores):** 가장 단순한 형태로, 고유한 키와 해당 키에 연결된 값으로 데이터를 저장합니다. (예: Amazon S3, Redis)
- **빅 테이블 (Big Tables):** 수백만 또는 수십억 개의 행을 저장할 수 있는 대규모 분산 데이터베이스입니다. (예: Google Bigtable, Apache Cassandra, HBase, DynamoDB)
- **문서 데이터베이스 (Document Databases):** 데이터를 JSON, BSON, XML과 같은 문서 형태로 저장합니다. (예: MongoDB, CouchDB)
- **그래프 데이터베이스 (Graph Databases):** 노드(개체)와 엣지(관계)를 사용하여 데이터를 저장하며, 복잡한 관계 분석에 적합합니다. (예: Neo4j)
- **전체 텍스트 검색 (Full-Text Search):** 텍스트 데이터의 효율적인 검색을 위해 사용됩니다. (예: Elasticsearch, Apache Solr)

NoSQL 모델링의 특징

- **스키마 유연성 (Schema Flexibility):** NoSQL은 '스키마 없음 (schema-less)' 또는 '스키마 자유 (schema-free)'라고 불리지만, 이는 스키마를 고려하지 않아도 된다는 의미가 아닙니다. 오히려 더 많은 유연성을 제공하므로 모델링에 대한 깊은 고민이 필요합니다. 관계형 DB처럼 데이터를 로드하기 전에 엄격한 스키마를 정의할 필요가 없습니다.
- **질문 중심 사고 (Question-Oriented Thinking):** 관계형 DB가 '내가 가진 데이터로 무엇을 답할 수 있는가?'에 초점을 맞춘다면, NoSQL은 '비즈니스가 어떤 질문을 하고 싶어 하는가?'에 초점을 맞춰 데이터를 모델링합니다.
- **비정규화 (Denormalization):** NoSQL 시스템은 대부분 조인 (Join)을 효율적으로 지원하지 않으므로, 데이터 중복을 허용하는 비정규화 (denormalization)를 선호합니다. 이는 읽기 성능을 향상시키는 데 도움이 됩니다.
- **와이드 테이블 (Wide Tables):** 수백만 또는 수십억 개의 컬럼을 가질 수 있는 와이드 테이블을 지원하여, 필요한 모든 데이터를 단일 테이블에 저장할 수 있습니다.
- **데이터 임베딩 vs. 참조 (Embedding vs. Referencing):**
 - * **임베딩 (Embedding):** 관련 데이터를 하나의 문서나 레코드 안에 직접 포함시키는 방식입니다. 단일 쿼리로 모든 정보를 가져올 수 있어 읽기 성능이 매우 빠릅니다.
 - * **참조 (Referencing):** 관계형 DB처럼 다른 테이블의 ID를 참조하는 방식입니다. 여러 쿼리가 필요할 수 있어 임베딩보다 느릴 수 있습니다.

데이터 임베딩 예시

주문 정보에 제품 정보를 직접 포함하는 경우:

```
{
  "order_id": "00001",
  "customer_id": "C123",
  "products": [
    { "product_id": "P001", "name": "Laptop", "price": 1200 },
    { "product_id": "P002", "name": "Mouse", "price": 25 }
  ],
  "order_date": "2023-10-26"
}
```

이 경우, 주문 ID로 한 번의 쿼리만으로 주문과 관련된 모든 제품 정보를 가져올 수 있습니다.

데이터 참조 예시

주문 정보에서 제품 ID만 참조하고 제품 정보는 별도 테이블에 저장하는 경우:

```
// Order Table
{
  "order_id": "00001",
  "customer_id": "C123",
  "product_ids": ["P001", "P002"],
  "order_date": "2023-10-26"
}
```

```
// Product Table
{ "product_id": "P001", "name": "Laptop", "price": 1200 }
{ "product_id": "P002", "name": "Mouse", "price": 25 }
```

이 경우, 주문 정보를 가져온 후 다시 제품 ID를 사용하여 제품 테이블에서 정보를 가져와야 하므로 최소 두 번의 쿼리가 필요합니다.

NoSQL 모델링 규칙

- **비정규화 (Denormalization):** 읽기 성능 향상을 위해 데이터 중복을 허용합니다.
- **조인 최소화 (Minimal Joins):** 대부분의 NoSQL 시스템은 조인 지원이 없거나 제한적입니다.
- **데이터 마트 (Data Marts):** 스타 스키마 또는 스노우플레이크 스키마를 활용합니다.
- **그래프 데이터 저장소 (Graph Data Stores):** 소셜 플랫폼처럼 복잡한 관계를 표현할 때 노드(vertices)와 엣지(edges)를 사용합니다.
- **인메모리 데이터 저장소 (In-Memory Data Stores):** 빠른 키-값 접근이 필요할 때 사용합니다.

2.2.3 메타데이터 (Metadata)

메타데이터란?

메타데이터는 '데이터에 대한 데이터'를 의미합니다. 이는 데이터의 의미, 출처, 사용 방법, 소유자, 생성 시점 등 데이터를 설명하는 모든 정보를 포함합니다. 메타데이터는 조직이 정보 자산을 이해하고 관리하며, 데이터로부터 더 큰 비즈니스 가치를 얻는 데 필수적입니다.

메타데이터가 답하는 질문들

메타데이터는 다음과 같은 중요한 질문들에 답하여 데이터의 가치를 높입니다.

- **누가 (Who):** 데이터를 생성, 관리, 사용, 소유, 규제하는가?
- **무엇을 (What):** 데이터의 비즈니스 정의, 사용 규칙, 보안 수준, 약어, 명명 표준은 무엇인가?
- **어디서 (Where):** 데이터는 어디에 저장되고, 어디서 오며, 어디서 사용되는가? 백업 위치, 지역적 제한은? **왜 (Why):** 데이터를 저장하는 목적과 용도는 무엇이며, 비즈니스 동기는 무엇인가?
- **언제 (When):** 데이터는 언제 생성/업데이트되었고, 얼마나 오래 저장되어야 하며, 언제 삭제되어야 하는가?
- **어떻게 (How):** 데이터는 어떻게 포맷되었고, 몇 개의 데이터베이스/소스에 저장되어 있는가?

메타데이터의 유형

- **비즈니스 메타데이터 (Business Metadata):** 데이터의 비즈니스적 의미를 정의합니다. (예: 온톨로지, 비즈니스 규칙, KPI)
- **기술적 메타데이터 (Technical Metadata):** 데이터의 기술적 구조와 특성을 정의합니다. (예: 데이터 모델, 데이터 계보(lineage), 저장 방식)
- **운영 메타데이터 (Operational Metadata):** 데이터의 운영적 측면을 설명합니다. (예: SLA, 데이터 신선도, 사용 패턴)

데이터 카탈로그 (Data Catalog)

데이터 카탈로그는 메타데이터를 저장하고 관리하는 기술적 리소스입니다. 이를 통해 데이터 검색, 데이터 거버넌스, 개인 정보 보호 및 위험 관리, 데이터 가치 평가 등을 지원합니다. 데이터브릭스에도 카탈로그 기능이 내장되어 있습니다.

2.2.4 OLTP 시스템 vs. OLAP 시스템

데이터베이스 시스템은 크게 두 가지 유형으로 나눌 수 있습니다.

Table 2.2: OLTP와 OLAP 시스템 비교

특징	OLTP (Online Transaction Processing)	OLAP (Online Analytical Processing)
목적	일상적인 비즈니스 운영, 트랜잭션 처리	의사 결정 지원, 데이터 분석
데이터 소스	운영 데이터베이스 (예: ERP, CRM)	OLTP 시스템에서 추출된 데이터
데이터 특성	현재 데이터, 상세하고 원자적	이력 데이터, 요약 및 집계
데이터 모델	정규화된 ER 다이어그램 (엔티티-관계 모델)	비정규화된 차원 모델 (스타/스노우플레이크 스키마)
주요 작업	빠른 읽기/쓰기/업데이트/삭제 (CRUD)	복잡한 쿼리, 대규모 데이터 스캔
사용자	최종 사용자 (고객, 직원)	데이터 분석가, 비즈니스 사용자
성능 지표	트랜잭션 처리량 (Transaction Throughput)	쿼리 응답 시간 (Query Response Time)
데이터 크기	기가바이트 (GB) 수준	테라바이트 (TB) 이상
업데이트 빈도	지속적인 업데이트	배치 업데이트 (주기적)
예시	은행 거래 시스템, 온라인 쇼핑몰 주문 시스템	데이터 웨어하우스, 비즈니스 인텔리전스 대시보드

ER 다이어그램 (Entity-Relationship Diagram)

ER 다이어그램은 OLTP 시스템에서 관계형 데이터베이스를 모델링하는 데 사용됩니다. 엔티티 (개체), 속성 (attributes), 엔티티 간의 관계를 시각적으로 표현하며, 데이터의 기술적이고 구체적인 정의 (데이터 타입, 기본 키, 외래 키 등)에 중점을 둡니다.

차원 모델링 (Dimensional Modeling): 스타 스키마 vs. 스노우플레이크 스키마

OLAP 시스템에서는 분석 효율성을 위해 차원 모델링을 사용하며, 주로 스타 스키마와 스노우플레이크 스키마가 있습니다.

1. 스타 스키마 (Star Schema):

- **한 줄 요약:** 중앙에 하나의 '팩트 테이블 (Fact Table)'이 있고, 그 주위를 여러 '차원 테이블 (Dimension Table)'이 둘러싸는 형태입니다.
- **직관적 예시:** '판매'라는 팩트 테이블 (거래 금액, 수량 등)이 있고, '고객', '제품', '시간', '지점'과 같은 차원 테이블이 이 판매 팩트와 연결됩니다.
- **기술적 설명:** 팩트 테이블은 측정 가능한 수치 (메트릭)를 포함하고, 차원 테이블은 팩트의 맥락 (누가, 무엇을, 언제, 어디서)을 제공합니다. 조인 수가 적어 쿼리 성능이 빠르지만, 데이터 중복이

발생할 수 있습니다(비정규화).

2. 스노우플레이크 스키마 (Snowflake Schema):

- **한 줄 요약:** 스타 스키마의 차원 테이블이 다시 다른 차원 테이블을 참조하여 계층 구조를 형성하는 형태입니다.
- **직관적 예시:** '판매' 팩트 테이블이 '제품' 차원 테이블과 연결되고, '제품' 차원 테이블은 다시 '제품 카테고리' 차원 테이블과 연결되는 식입니다.
- **기술적 설명:** 스타 스키마보다 더 정규화되어 데이터 중복이 적지만, 더 많은 조인이 필요하여 쿼리 성능이 저하될 수 있습니다. 모델의 복잡성이 증가합니다.

Table 2.3: 스타 스키마와 스노우플레이크 스키마 비교

특징	스타 스키마 (Star Schema)	스노우플레이크 스키마 (Snowflake Schema)
구조	팩트 테이블 주위에 차원 테이블이 직접 연결	차원 테이블이 다른 차원 테이블을 참조하여 계층 형성
정규화 수준	낮음 (비정규화)	높음 (더 정규화됨)
데이터 중복	높음	낮음
조인 수	적음	많음
쿼리 성능	빠름 (단순한 조인)	느림 (복잡한 조인)
구현 복잡성	낮음	높음
저장 공간	더 많이 필요 (중복 데이터)	더 적게 필요
OLAP 큐브	큐브 처리 속도 빠름	큐브 처리 속도 느림

2.2.5 데이터 웨어하우스 모델링 기법

현대 데이터 웨어하우스 모델링에는 크게 세 가지 접근 방식이 있습니다.

1. 빌 인몬 (Bill Inmon) - 탑다운 접근 (Top-Down Approach):

- **한 줄 요약:** 데이터 웨어하우스를 먼저 3차 정규화(3NF)된 형태로 구축하고, 그 후에 특정 비즈니스 기능에 맞는 데이터 마트(Data Mart)를 파생시키는 방식입니다.
- **특징:** 데이터 웨어하우스 자체가 고도로 정규화되어 데이터 중복을 최소화하고 데이터 무결성을 보장합니다. '단일 진실의 원천(Single Source of Truth)'을 강조합니다. 구현 및 유지보수가 복잡하고 쿼리 성능이 느릴 수 있습니다.

2. 랄프 킴볼 (Ralph Kimball) - 바텀업 접근 (Bottom-Up Approach):

- **한 줄 요약:** 개별 데이터 마트를 먼저 차원 모델(스타/스노우플레이크 스키마)로 구축하고, 이들을 통합하여 데이터 웨어하우스를 형성하는 방식입니다.
- **특징:** 보고 및 분석에 최적화되어 이해하기 쉽고 쿼리 성능이 빠릅니다. 데이터 마트가 비정규화되어 데이터 중복이 발생할 수 있으며, '단일 진실의 원천'을 유지하기 어려울 수 있습니다.

3. 댄 린스테트 (Dan Linstedt) - 데이터 볼트 (Data Vault):

- **한 줄 요약:** 인몬과 킴볼 모델의 단점을 보완하기 위해 고안된 모델링 기법으로, 허브(Hubs), 링크(Links), 위성(Satellites)이라는 새로운 개념을 도입합니다.
- **개념:**
 - * **허브 (Hubs):** 비즈니스의 핵심 개념(예: 고객, 제품, 주문)을 나타내며, 고유한 비즈니스 키를 저장합니다.
 - * **링크 (Links):** 허브 간의 관계를 나타냅니다.

- * **위성 (Satellites):** 허브나 링크의 속성(예: 고객 이름, 주소, 제품 가격)과 해당 속성의 변경 이력을 저장합니다.
- **특징:** 변경 관리에 매우 유연하고, 감사 추적성이 뛰어나며, 데이터 로딩 속도가 빠릅니다. 복잡성이 증가할 수 있습니다. 원시 데이터(Raw Vault)와 비즈니스 규칙이 적용된 데이터(Business Vault)를 분리하여 관리합니다.

Table 2.4: 데이터 웨어하우스 모델링 기법 비교

특징	인몬 (Inmon)	킴볼 (Kimball)	데 이 터 볼 트 (Data Vault)
접근 방식	탑다운 (Top-Down)	바텀업 (Bottom-Up)	하이브리드
정규화	고도로 정규화 (3NF)	비정규화 (차원 모델)	정규화와 비정규화 혼합
핵심 구조	정규화된 데이터 웨어하우스	차원 모델 기반 데이터 마트	허브, 링크, 위성
장점	단일 진실의 원천, 데이터 무결성	보고/분석 최적화, 이해 용이	변경 유연성, 감사 추적성, 빠른 로딩
단점	복잡한 구현/유지보수, 느린 쿼리	단일 진실의 원천 유지 어려움	모델링 복잡성 증가
주요 용도	대규모 엔터프라이즈 데이터 통합	보고 및 분석	운영 유연성, 새로운 소스 추가 용이

2.2.6 데이터 아키텍처 레이어 (Bronze, Silver, Gold)

데이터 레이크하우스 아키텍처에서는 데이터를 처리하고 정제하는 과정에 따라 세 가지 계층으로 나눕니다.

1. 브론즈 레이어 (Bronze Layer):

- **한 줄 요약:** 원본 데이터를 거의 변경 없이 저장하는 초기 수집 영역입니다.
- **특징:** 엔터프라이즈 애플리케이션, OLTP 데이터베이스 등 다양한 소스에서 CSV, JSON, Parquet, Delta 등 여러 형식의 데이터가 들어옵니다. 최소한의 유효성 검사(예: 타임스탬프 추가, 파일명 기록)만 수행하며, 원본 데이터의 무결성을 유지합니다.

2. 실버 레이어 (Silver Layer):

- **한 줄 요약:** 브론즈 레이어의 데이터를 정제하고, 변환하며, 다른 데이터 소스와 조인하여 비즈니스 규칙을 적용하는 영역입니다.
- **특징:** 데이터 정제(클렌징), 정규화, 중복 제거, 데이터 유형 변환 등이 이루어집니다. 데이터 볼트의 Raw Vault와 Business Vault 개념이 이 단계에 해당할 수 있습니다. 데이터 컨포먼스(Data Conformance)를 통해 데이터의 일관성을 확보하기 시작합니다.

3. 골드 레이어 (Gold Layer):

- **한 줄 요약:** 비즈니스 사용자의 분석 및 보고 요구사항에 맞춰 최적화된 형태로 데이터를 집계하고 요약하는 최종 영역입니다.
- **특징:** 스타 스키마, 데이터 마트, 특수 뷰(View) 등이 생성됩니다. 다양한 유형의 소비자가 쉽게 접근하고 분석할 수 있도록 데이터를 롤업(Roll-up)하고 집계합니다. 보고서, 대시보드, ML 모델 학습 등에 직접 사용됩니다.

데이터 컨포먼스 (Data Conformance)

데이터 컨포먼스는 서로 다른 소스에서 들어온 데이터를 일관된 형식과 의미로 표준화하는 과정입니다. 예를 들어, 우편번호가 '12345' 또는 '123-45'로 입력될 수 있지만, 분석을 위해 '12345'와 같은 5자리 숫자로 통일하는 것입니다. 성별 필드가 'm', 'f', 'male', 'female', '0', '1' 등으로 다양하게 표현될 수 있을 때, 이를 'M'과 'F'로 통일하는 작업도 포함됩니다. 이는 최종 분석의 정확성과 신뢰성을 높이는 데 필수적입니다.

2.2.7 도메인 주도 설계 (Domain-Driven Design, DDD)

특정 산업 분야(예: 제조, 보험)에서는 데이터 모델을 처음부터 구축하기보다, 해당 산업의 표준화된 엔티티-관계 모델 템플릿을 활용하고 조직의 특정 요구사항에 맞춰 조정하는 도메인 주도 설계를 사용합니다. Open Management Group (OMG)과 같은 기관에서 이러한 템플릿을 제공합니다.

2.2.8 데이터 형식 선택

데이터를 저장하는 형식(Format)은 파이프라인의 성능, 비용, 유연성에 큰 영향을 미치므로 신중하게 선택해야 합니다.

데이터 형식 선택 시 고려사항

- 도구 지원: 모든 도구가 모든 데이터 형식을 지원하는 것은 아닙니다.
- 저장 및 메모리 제약: 파일 크기와 메모리 사용량은 중요한 고려 사항입니다.
- 분할 가능성 (Splittable): 대용량 파일이 여러 노드에 분산되어 병렬 처리될 수 있는지 여부입니다. 분할 불가능한 파일은 분산 컴퓨팅의 이점을 활용할 수 없습니다.
- 컬럼 수 및 사용량: 저장되는 컬럼 수와 분석에 사용되는 컬럼 수를 고려해야 합니다. `SELECT *` 쿼리는 불필요한 데이터를 읽어 성능을 저하시킬 수 있습니다.
- 데이터 변경 빈도: 데이터가 얼마나 자주 변경되는지(추가, 업데이트)에 따라 적합한 형식이 달라 집니다.

데이터 형식 분류

1. 텍스트 형식 (Text Formats):

- 예시: CSV (Comma Separated Values), JSON (JavaScript Object Notation)
- 특징: 사람이 읽기 쉽습니다. 하지만 공간 효율성이 낮고, 데이터 타입이나 구조 검증 기능이 부족하며, 메타데이터가 없거나 중복되어 저장될 수 있습니다. 압축 효율도 낮고, 네이티브 인덱싱이나 검색 기능이 없습니다.

2. 바이너리 형식 (Binary Formats):

- 예시: Parquet, Avro, ORC
- 특징: 기계가 읽기 쉽고, 공간 효율성이 높으며, 데이터 타입 및 스키마 정보를 내장합니다. 압축 및 인덱싱에 유리합니다.

행 기반 vs. 컬럼 기반 형식

- 행 기반 저장소 (Row-Based Stores):

- * **예시:** CSV, JSON, Avro
- * **특징:** 데이터를 행 단위로 저장합니다. 데이터 쓰기 속도가 빠르므로 OLTP 시스템(트랜잭션 처리)에 적합합니다. 데이터 정규화가 업데이트 효율성을 높입니다.
- **컬럼 기반 저장소 (Column-Based Stores):**
 - * **예시:** Parquet, ORC
 - * **특징:** 데이터를 컬럼 단위로 저장합니다. 특정 컬럼만 읽을 때 I/O 효율성이 높아 대규모 데이터 집계 및 분석(OLAP 시스템)에 적합합니다. 쿼리 속도가 빠릅니다. 비정규화된 데이터 구조에 유리합니다.

Table 2.5: 데이터 형식별 속성 비교

형식	컬럼 저장	행 지향	압축 가능	분할 가능	사람이 읽기 가능	바이너리	복잡성	스키마 진화 지원
CSV	No	Yes	No	Yes	Yes	No	Simple	No
JSON	No	Yes	No	Yes	Yes	No	Medium	Yes
Avro	No	Yes	Yes	Yes	No	Yes	Medium	Yes
Parquet	Yes	No	Yes	Yes	No	Yes	Complex	Yes
ORC	Yes	No	Yes	Yes	No	Yes	Complex	Yes

델타 레이크 (Delta Lake)

델타 레이크는 Parquet 파일 형식 위에 구축된 오픈 소스 스토리지 레이어(프로토콜)입니다. Parquet 파일의 장점(컬럼 기반, 압축, 분할 가능)을 유지하면서 다음과 같은 추가 기능을 제공합니다.

- **ACID 트랜잭션 (ACID Transactions):** 분산 컴퓨팅 환경에서 데이터 무결성과 신뢰성을 보장합니다.
- **스키마 진화 (Schema Evolution):** 데이터 스키마 변경에 유연하게 대응합니다.
- **시간 여행 (Time Travel):** 데이터의 과거 버전에 접근하여 특정 시점으로 롤백하거나 변경 이력을 추적할 수 있습니다.
- **업서트 (Upsert) 작업 간소화:** 기존 행을 업데이트하거나 새 행을 삽입하는 MERGE 작업을 단일 명령으로 처리할 수 있습니다.
- **배치 및 스트리밍 통합:** 배치 처리와 스트리밍 처리를 동일한 테이블에서 지원하여 파이프라인 복잡성을 줄입니다.
- **트랜잭션 로그 (Transaction Log):** 모든 변경 사항을 기록하여 테이블의 상태를 재구성하고, 데이터 일관성을 유지합니다. (`_delta_log` 디렉토리에 저장)

2.2.9 데이터 압축 (Data Compression)

데이터 압축은 정보를 인코딩할 때 원본 표현보다 더 적은 비트를 사용하는 과정입니다.

압축의 중요성

- **디스크 공간 절약:** 저장 비용을 줄입니다.
- **I/O 대역폭 감소:** 네트워크를 통해 전송되는 데이터 양이 줄어들어 데이터 전송 속도가 빨라집니다.
- **쿼리 성능 향상:** 더 적은 데이터를 읽으므로 쿼리 실행 시간이 단축됩니다.

압축 알고리즘 유형

- 손실 압축 (Lossy Compression): 일부 데이터 손실을 허용하여 압축률을 높입니다. (예: 이미지, 비디오)
- 무손실 압축 (Lossless Compression): 원본 데이터를 완벽하게 복원할 수 있습니다. 데이터베이스와 같이 정확성이 중요한 경우에 사용됩니다.

코덱 (Codec)

코덱은 압축기(compressor)와 압축 해제기(decompressor)의 조합입니다. 압축 알고리즘은 압축률과 압축/압축 해제 속도 사이에서 트레이드오프를 가집니다.

Table 2.6: 인기 있는 압축 코덱 비교

코덱	파일 확장자	분할 가능	압축률	압축 속도
Gzip	.gz	No	높음	중간
Snappy	.snappy	Yes	중간	매우 빠름
LZO	.lzo	Yes	중간	빠름
Zstandard (Zstd)	.zst	Yes	매우 높음	빠름
Brotli	.br	No	매우 높음	중간

2.2.10 데이터 프로파일링 (Data Profiling)

데이터 프로파일링은 데이터의 품질, 구조, 관계를 분석하고 이해하는 과정입니다. 데이터 품질 문제(오래된 데이터, 부정확한 데이터)는 비즈니스에 막대한 비용을 초래하므로, 데이터 파이프라인의 소유자와 소비자 모두 데이터 품질을 이해하는 것이 중요합니다.

데이터 프로파일링의 이점

- 신뢰성 확보: 데이터의 신뢰도를 높여 의사 결정의 기반을 강화합니다.
- 더 나은 의사 결정: 정확하고 신뢰할 수 있는 데이터를 바탕으로 더 나은 비즈니스 결정을 내릴 수 있습니다.
- 조직화된 데이터: 데이터의 일관성과 구조를 개선하여 관리를 용이하게 합니다.

데이터 프로파일링 기법

- 구조적 발견 (Structural Discovery): 데이터가 일관되고 올바른 형식으로 되어 있는지 확인합니다. (예: 스키마 일치 여부, 기본 통계량 - 최소, 최대, 평균, 표준 편차, 결측치 수)
- 콘텐츠 발견 (Content Discovery): 데이터 값 자체의 품질을 평가합니다. (예: Null 값 비율, 유효하지 않은 범위의 값, 잘못된 형식의 값)
- 관계 발견 (Relationship Discovery): 서로 다른 데이터 세트 간의 연결을 식별합니다. (예: 퍼지 로직을 사용한 엔티티 해상도, 데이터 병합)
- 데이터 상관관계 (Data Correlation): 데이터 포인트 간의 통계적 관계를 분석합니다. (예: 단변량 및 다변량 분석, 요인 분석, 클러스터 분석, 분산 분석)

- **데이터 시각화 (Data Visualization):** 시각적 도구를 사용하여 데이터의 패턴, 이상치(outliers), 분포 등을 파악합니다. (예: 대시보드, 차트)

스파크 데이터 프로파일링 예시

스파크 데이터프레임에서 `describe()` 또는 `summary()` 명령을 사용하여 데이터 세트의 통계적 세부 정보를 빠르게 확인할 수 있습니다.

```
1 df.describe().show()
2 df.summary().show()
```

Listing 2.1: Spark DataFrame 요약 통계

2.2.11 데이터 엔지니어링 모범 사례

- **요구사항 이해 및 소비 패턴 모델링:** 데이터 사용자의 요구사항을 명확히 이해하고, 그들의 데이터 소비 패턴에 맞춰 데이터를 모델링합니다.
- **데이터 품질 가드레일 설정:** 데이터 품질을 보장하기 위한 기준과 프로세스를 수립합니다.
- **민감 데이터 이해 및 분류:** 개인 정보 등 민감한 데이터를 식별하고 적절하게 분류하여 보호합니다.
- **메타데이터 관리:** 메타데이터를 최신 상태로 유지하고, 테이블 및 컬럼에 대한 명확한 설명과 주석을 제공합니다.
- **올바른 파일 형식 선택:** 데이터의 특성과 사용 목적에 맞는 최적의 파일 형식을 선택합니다.
- **데이터 압축 및 유형 일치:** 데이터를 압축하고, 데이터 유형에 맞는 압축 방식을 선택합니다.
- **데이터 중복 제거:** 분석 시스템에서도 중복 제거는 중요합니다. 특히 보고 및 집계 시 정확한 계산을 위해 필요합니다.

2.3 실습: Lab 01 - 주택 가격 예측

Lab 01은 데이터브릭스(Databricks) 환경에서 주택 가격을 예측하는 머신러닝 워크플로우를 실습합니다. 이 실습은 데이터 과학 프로젝트의 일반적인 단계를 따릅니다.

2.3.1 실습 목표

- 데이터브릭스 워크스페이스에서 노트북을 가져오고 실행하는 방법 이해
- 데이터 로드, 탐색, 전처리, 모델링, 평가의 일반적인 데이터 과학 워크플로우 학습
- Spark DataFrame과 Pandas DataFrame 간의 전환 및 Scikit-learn 라이브러리 사용
- Delta Lake 테이블 생성 및 활용
- 데이터 시각화를 통한 데이터 이해 및 모델 분석
- 선형 회귀 모델을 사용하여 주택 가격 예측 및 RMSSE 지표로 모델 성능 평가

2.3.2 데이터브릭스 환경 설정 및 노트북 가져오기

1. **노트북 다운로드:** 제공된 Google Drive 링크에서 Lab 01 zip 파일을 다운로드합니다.
2. **워크스페이스 접속:** 데이터브릭스 워크스페이스에 로그인합니다.

3. 노트북 가져오기:

- Workspace -> Home (또는 원하는 폴더)로 이동합니다.
- 폴더를 생성(Labs 등)하고 해당 폴더 안에서 Import 버튼을 클릭합니다.
- 다운로드한 zip 파일을 업로드하여 가져옵니다.

4. 노트북 확인: 가져온 lab01 폴더 안에 pispark.mml과 scikitlearn 두 개의 노트북이 있는지 확인합니다.

중요: 서버리스 환경 제약사항

데이터브릭스 무료 에디션(Free Edition)은 서버리스(Serverless) 컴퓨팅 환경을 사용합니다. 현재 서버리스 환경에서는 Spark ML (분산 학습 라이브러리)이 지원되지 않습니다. 따라서 pispark.mml 노트북은 실행되지 않거나 오류가 발생할 수 있습니다. 반드시 **scikitlearn** 버전의 노트북을 사용해야 합니다. scikitlearn은 단일 노드 머신러닝 라이브러리를 사용하며, PySpark Pandas를 통해 분산 Spark DataFrame을 단일 노드 Pandas DataFrame으로 변환하여 처리합니다.

2.3.3 Lab 01 워크플로우 (Scikit-learn 버전)

Lab 01은 다음과 같은 단계로 진행됩니다.

1단계: 비즈니스 이해 (Business Understanding)

* **목표:** 주어진 주택 가격 데이터를 기반으로 주택 가격을 예측하는 모델을 구축합니다. * **평가 지표:** RMSSE (Root Mean Square Scaled Error)를 사용하여 예측 모델의 성능을 평가합니다.

2단계: 환경 설정 및 데이터 로드

1. 스키마 및 볼륨 생성:

- lab01 스키마와 input, output 볼륨을 생성합니다. 이는 데이터 저장 및 관리를 위한 기본 환경을 설정하는 과정입니다.

```

1 %sql
2 CREATE CATALOG IF NOT EXISTS unity_catalog_demo;
3 USE CATALOG unity_catalog_demo;
4 CREATE SCHEMA IF NOT EXISTS lab01;
5 USE SCHEMA lab01;
6 CREATE VOLUME IF NOT EXISTS input;
7 CREATE VOLUME IF NOT EXISTS output;
8

```

Listing 2.2: 스키마 및 볼륨 생성 코드

2. CSV 파일 다운로드:

- 공개 GitHub 저장소에서 주택 가격 CSV 파일을 다운로드하여 input 볼륨에 저장합니다.

```

1 %sh
2 wget -O /Volumes/unity_catalog_demo/lab01/input/housing.csv https://raw
  .githubusercontent.com/apache/spark/master/examples/src/main/
  resources/housing.csv

```

3

Listing 2.3: CSV 파일 다운로드 코드

3. 파일 복사(선택 사항):

- dbutils.fs.cp 명령을 사용하여 input 볼륨에서 output 볼륨으로 파일을 복사합니다. 이는 파일 이동을 보여주는 예시입니다.

```
1 dbutils.fs.cp("/Volumes/unity_catalog_demo/lab01/input/housing.csv", "/
  Volumes/unity_catalog_demo/lab01/output/housing.csv")
2
```

Listing 2.4: 파일 복사 코드

4. 데이터 경로 설정:

- 데이터 파일 경로를 변수로 설정하여 이후 코드에서 쉽게 참조할 수 있도록 합니다.

```
1 housing_path = "/Volumes/unity_catalog_demo/lab01/output/housing.csv"
2
```

Listing 2.5: 데이터 경로 변수 설정 코드

5. 데이터 로드 및 확인:

- CSV 파일을 Spark DataFrame으로 읽어들이고, display() 명령으로 데이터를 확인합니다. df.describe().show()를 추가하여 통계 요약도 볼 수도 있습니다.

```
1 df = spark.read.csv(housing_path, header=True, inferSchema=True)
2 display(df)
3
```

Listing 2.6: 데이터 로드 및 확인 코드

3단계: 데이터 전처리 및 탐색 (Data Preparation & Exploration)

1. 데이터프레임 확장:

- 기존 데이터프레임에 date (Unix 타임스탬프 변환)와 zipcode (문자열 캐스팅) 컬럼을 추가합니다.

```
1 from pyspark.sql.functions import from_unixtime, col
2 df_augmented = df.withColumn("date", from_unixtime(col("date"))) \
3                     .withColumn("zipcode", col("zipcode").cast("string"))
4 display(df_augmented)
5
```

Listing 2.7: 데이터프레임 컬럼 추가 코드

2. Delta 테이블로 저장:

- 전처리된 데이터를 table_housing이라는 Delta 테이블로 저장합니다. Delta Lake는 데이터브릭스의 기본 형식이며, ACID 트랜잭션, 스키마 진화 등의 기능을 제공합니다.

```
1 df_augmented.write.format("delta").mode("overwrite").saveAsTable("
  table_housing")
```


2

Listing 2.8: Delta 테이블 저장 코드

3. 테이블 탐색:

- DESCRIBE EXTENDED table_housing 명령으로 테이블의 상세 정보를 확인합니다. provider가 delta인지 확인합니다.

```

1 %sql
2 DESCRIBE EXTENDED table_housing;
3

```

Listing 2.9: Delta 테이블 상세 정보 확인 코드

4. 기본 쿼리 및 시각화:

- 집 코드별 가격 추세 등 기본적인 SQL 쿼리를 실행하여 데이터를 이해합니다.
- 가격과 주거 면적(sqft_living) 간의 상관관계를 시각화합니다.
- **시각화 방법:** 쿼리 결과 셀 옆의 + 버튼을 클릭하고 Visualization을 선택합니다. Scatter plot을 선택하고 X축을 sqft_living, Y축을 price로 설정하면 주거 면적이 넓을수록 가격이 높아지는 경향을 확인할 수 있습니다.

```

1 %sql
2 SELECT price, sqft_living FROM table_housing;
3

```

Listing 2.10: 가격과 주거 면적 상관관계 쿼리

4단계: 데이터 모델링 (Data Modeling)**1. ANSI SQL 모드 비활성화:**

- Pandas와 Spark의 호환성을 위해 spark.sql.ansi.enabled 설정을 false로 변경합니다.

```

1 spark.conf.set("spark.sql.ansi.enabled", False)
2

```

Listing 2.11: ANSI SQL 모드 설정 코드

2. 특징 엔지니어링 (Feature Engineering):

- zipcode 컬럼을 LabelEncoder를 사용하여 고유한 숫자 값으로 인코딩합니다.
- 예측 모델에 사용할 특징(features)을 조합합니다.
- Spark DataFrame을 Pandas DataFrame으로 변환하여 Scikit-learn 라이브러리를 사용하고, 다시 Spark DataFrame으로 변환하는 과정을 거칩니다.

```

1 import pyspark.pandas as ps
2 import pandas as pd
3 from sklearn.preprocessing import LabelEncoder
4 from pyspark.ml.feature import VectorAssembler
5
6 # Spark 을DataFrame PySpark Pandas 으로DataFrame 변환
7 psdf = ps.DataFrame(spark.table("table_housing"))

```

```

8
9 # Pandas 으로DataFrame 변환하여 Scikit-learn LabelEncoder 사용
10 pdf = psdf.to_pandas()
11 le = LabelEncoder()
12 pdf['zipcode_encoded'] = le.fit_transform(pdf['zipcode'])
13
14 # 다시 PySpark Pandas 으로DataFrame 변환
15 psdf_encoded = ps.DataFrame(pdf)
16
17 # 특징 조합 (는VectorAssembler Spark 이지만ML, 여기서는 개념 설명)
18 # 실제 Scikit-에서는learn features 리스트를 직접 사용
19 features = ['bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot', 'floors
    ', 'waterfront', 'view', 'condition', 'grade', 'sqft_above', '
    sqft_basement', 'yr_built', 'yr_renovated', 'zipcode_encoded', 'lat
    ', 'long', 'sqft_living15', 'sqft_lot15']
20 psdf_final = psdf_encoded[features + ['price']]
21
22 # 최종 Spark 으로DataFrame 변환
23 df_final = psdf_final.to_spark()
24 display(df_final)
25

```

Listing 2.12: 특징 엔지니어링 코드

3. 훈련/테스트 데이터 분할:

- randomSplit() 함수를 사용하여 데이터를 훈련 세트(80%)와 테스트 세트(20%)로 나눕니다. 재현성을 위해 seed 값을 설정합니다.

```

1 train_df, test_df = df_final.randomSplit([0.8, 0.2], seed=123)
2

```

Listing 2.13: 훈련/테스트 데이터 분할 코드

4. 선형 회귀 모델 학습 및 평가:

- Scikit-learn의 LinearRegression 모델을 사용하여 훈련 데이터에 모델을 학습시킵니다.
- 모델의 계수(coefficients)와 절편(intercept)을 출력하고, mean_squared_error 및 r2_score를 사용하여 모델 성능을 평가합니다.

```

1 from sklearn.linear_model import LinearRegression
2 from sklearn.metrics import mean_squared_error, r2_score
3 import numpy as np
4
5 # Spark 을DataFrame Pandas 으로DataFrame 변환
6 train_pdf = train_df.toPandas()
7 test_pdf = test_df.toPandas()
8
9 X_train = train_pdf[features]
10 y_train = train_pdf['price']
11 X_test = test_pdf[features]
12 y_test = test_pdf['price']
13

```

```

14 # 모델 학습
15 model = LinearRegression()
16 model.fit(X_train, y_train)
17
18 # 모델 파라미터 출력
19 print(f"Coefficients: {model.coef_}")
20 print(f"Intercept: {model.intercept_}")
21
22 # 예측
23 predictions = model.predict(X_test)
24
25 # 모델 평가
26 rmse = np.sqrt(mean_squared_error(y_test, predictions))
27 r2 = r2_score(y_test, predictions)
28
29 print(f"RMSE: {rmse}")
30 print(f"R2 Score: {r2}")
31

```

Listing 2.14: 선형 회귀 모델 학습 및 평가 코드

5단계: 예측 및 모델 분석 (Prediction & Model Analysis)

1. 예측 수행:

- 학습된 모델을 사용하여 테스트 데이터에 대한 예측을 수행하고, 예측 결과를 데이터프레임에 추가합니다.

```

1 # 예측 결과를 Pandas 에 DataFrame 추가
2 test_pdf['prediction'] = predictions
3
4 # 다시 PySpark Pandas 으로부터 DataFrame 변환
5 psdf_predictions = ps.DataFrame(test_pdf)
6
7 # 최종 Spark 으로부터 DataFrame 변환
8 df_predictions = psdf_predictions.to_spark()
9 display(df_predictions)
10

```

Listing 2.15: 예측 수행 코드

2. 예측 결과 Delta 테이블로 저장:

- 예측 결과를 table_predictions라는 Delta 테이블로 저장하여 영구적으로 보존합니다.

```

1 df_predictions.write.format("delta").mode("overwrite").saveAsTable("
    table_predictions")
2

```

Listing 2.16: 예측 결과 Delta 테이블 저장 코드

3. RMSSE 평가 및 시각화:

- 예측값과 실제값 간의 잔차(residual errors)를 계산하고, RMSSE를 기준으로 오차 분포를 시각화합니다.
- **시각화 방법:** 쿼리 결과 셀 옆의 + 버튼을 클릭하고 Visualization을 선택합니다.
 - * **히스토그램:** Histogram을 선택하고 X축을 residual_error로 설정합니다. Number of bins를 조정하여 오차 분포를 확인합니다. 대부분의 오차가 0 근처에 집중되어 있음을 볼 수 있습니다.
 - * **파이 차트:** Pie chart를 선택하고 X축을 RMSSE_multiple, Y축을 count로 설정합니다. 이를 통해 RMSSE 1 이내에 들어오는 예측의 비율(예: 80%)과 RMSSE 2 이내에 들어오는 예측의 비율(예: 96%)을 확인할 수 있습니다.

```

1 %sql
2 WITH errors AS (
3     SELECT
4         price,
5         prediction,
6         (price - prediction) AS residual_error,
7         ABS(price - prediction) / (SELECT STDDEV(price) FROM
8             table_predictions) AS RMSSE_multiple
9     FROM table_predictions
10 )
11 SELECT
12     CASE
13         WHEN RMSSE_multiple <= 1 THEN 'Within RMSSE 1'
14         WHEN RMSSE_multiple <= 2 THEN 'Within RMSSE 2'
15         ELSE 'Outside RMSSE 2'
16     END AS RMSSE_category,
17     COUNT(*) AS count
18 FROM errors
19 GROUP BY RMSSE_category;

```

Listing 2.17: RMSSE 평가 및 시각화 쿼리

4. 모델 성능 해석:

- 주택 가격의 범위가 매우 넓음(예: 5만 달러에서 5백만 달러)을 고려할 때, RMSSE가 작은 범위(예: 1 또는 2 이내)에 집중되어 있다면 모델이 가격을 상당히 정확하게 예측했다고 볼 수 있습니다.

2.4 체크리스트

이 체크리스트는 강의 내용을 학습하고 과제를 수행하며, 최종 문서를 검토하는 데 도움이 됩니다.

2.4.1 학습 및 이해 점검

ACID, BASE, CAP 이론의 개념과 차이를 명확히 이해했는가?

IaaS, PaaS, SaaS의 정의와 예시를 설명할 수 있는가?

레이크하우스 아키텍처의 장점을 설명할 수 있는가?

데이터 모델링의 3단계(개념적, 논리적, 물리적)를 설명하고 각 단계의 목적을 이해했는가?
 NoSQL 데이터 모델링의 특징(비정규화, 임베딩 vs. 참조)을 이해했는가?
 메타데이터의 중요성과 유형(비즈니스, 기술, 운영)을 설명할 수 있는가?
 OLTP와 OLAP 시스템의 주요 차이점을 설명할 수 있는가?
 스타 스키마와 스노우플레이크 스키마의 구조와 장단점을 비교할 수 있는가?
 인본, 김볼, 데이터 볼트 모델링 기법의 특징과 차이점을 이해했는가?
 브론즈, 실버, 골드 데이터 레이어의 역할과 목적을 설명할 수 있는가?
 데이터 형식(텍스트 vs. 바이너리, 행 기반 vs. 컬럼 기반) 선택의 중요성을 이해했는가?
 델타 레이크의 주요 기능(ACID, 스키마 진화, 시간 여행)을 설명할 수 있는가?
 데이터 압축의 중요성과 유형(손실 vs. 무손실)을 이해했는가?
 데이터 프로파일링의 목적과 주요 기법을 설명할 수 있는가?

2.4.2 Lab 01 실습 점검

Lab 01 노트북(scikitlearn 버전)을 데이터브릭스 워크스페이스에 성공적으로 가져왔는가?
 스키마 및 볼륨 생성 셀을 성공적으로 실행했는가?
 CSV 파일 다운로드 및 복사 셀을 성공적으로 실행했는가?
 Spark DataFrame으로 데이터를 로드하고 `display()`로 확인했는가?
`date` 및 `zipcode` 컬럼을 추가하고 데이터프레임을 확장했는가?
 확장된 데이터를 `table_housing` Delta 테이블로 저장했는가?
`DESCRIBE EXTENDED` 명령으로 `table_housing`의 Delta 형식을 확인했는가?
 가격과 주거 면적(`sqft_living`) 간의 상관관계를 산점도(scatter plot)로 시각화했는가?
`spark.sql.ansi.enabled` 설정을 `False`로 변경했는가?
`zipcode` 컬럼을 `LabelEncoder`로 인코딩하고 특징을 조합했는가?
 데이터를 훈련 세트와 테스트 세트로 `randomSplit()`을 사용하여 분할했는가?
 Scikit-learn `LinearRegression` 모델을 훈련 데이터에 학습시켰는가?
 모델의 `RMSE`와 `R2 Score`를 확인했는가?
 학습된 모델로 테스트 데이터에 대한 예측을 수행하고 `prediction` 컬럼을 추가했는가?
 예측 결과를 `table_predictions` Delta 테이블로 저장했는가?
 잔차 오차(`residual_error`)의 분포를 히스토그램으로 시각화했는가?
`RMSSE 1` 및 `RMSSE 2` 이내에 들어오는 예측의 비율을 파이 차트로 시각화했는가?
 모델 성능 평가 결과를 올바르게 해석할 수 있는가?

2.5 FAQ (자주 묻는 질문)

Q: 과제에서 AI 어시스턴트(예: Databricks AI Assistant)를 사용해도 되나요?

A: 네, 사용해도 됩니다. 이 과정의 목적은 개념을 학습하는 것이며, AI 어시스턴트와 코파일럿은 현대 시대에 피할 수 없는 도구입니다. 올바른 질문을 통해 이점을 얻는 것은 좋습니다. 다만, 단순히 답을 얻는 것을 넘어, 먼저 스스로 시도하여 개념을 이해하고 '근육 기억'을 형성하는 것이 중요합니다. 그래야 정보가 더 오래 기억됩니다.

Q: 과제 2.9에서 '데이터베이스 생성'이 무엇을 의미하나요? 스키마와 데이터베이스의 차이는 무엇인가요?

A: 과제 2.9에서 '데이터베이스 생성'은 사실상 '스키마 생성'을 의미합니다. 데이터브릭스 유니티 카탈로그(Unity Catalog) 환경에서 '스키마(Schema)'와 '데이터베이스(Database)'는 상호 교환 가능한 용어입니다. 제공된 코드는 카탈로그를 생성하고, 그 안에 스키마를 생성하며, 해당 스키마를 활성 스키마로 설정합니다. 테이블은 이 스키마 안에 생성됩니다.

Q: 과제에서 데이터베이스 이름을 `_assignment_one` 대신 `username_assignment_one`으로 해야 한다고 되어 있는데, 어떻게 해야 하나요?

A: 현재는 자신의 환경에서 작업하므로, `_assignment_one`과 같은 이름으로 진행해도 됩니다. `username_assignment_one`은 과거에 모든 사용자가 공유 환경을 사용할 때 충돌을 피하기 위한 지침이었습니다. 강사가 과제 지침을 수정할 예정이므로, 현재 제공된 코드의 이름으로 따라가세요.

Q: 제 과제 2.9 내용이 강사님이 보여주신 것과 다릅니다. CSV 파일을 테이블로 파싱하라는 내용이 있는데, 더 이상 하지 않나요?

A: 네, 더 이상 CSV 파일을 테이블로 파싱할 필요가 없습니다. 강사가 지침을 업데이트하고 다시 업로드했지만, 다운로드한 버전이 이전 버전일 수 있습니다. CSV 테이블 대신 델타 테이블만 생성하면 됩니다. 최신 버전을 다시 다운로드하여 확인하거나, CSV 관련 단계를 건너뛰고 델타 테이블 생성 부분만 진행하세요.

Q: 데이터 자산(data assets)은 어디서 찾을 수 있나요? 온라인에서 직접 찾아야 하나요?

A: 아니요, 데이터 자산은 Google Drive의 `assignment 01` 폴더 안에 있습니다. `assignment 01` 폴더에는 과제 노트북과 `data` 디렉토리가 있습니다. `data` 디렉토리 안에 `blog.json`, `people-10m` (parquet 파일), `names` 데이터셋이 있습니다. 이 파일들을 다운로드하여 데이터브릭스 볼륨에 업로드해야 합니다.

Q: 데이터브릭스 볼륨에 데이터를 업로드하는 구체적인 절차는 무엇인가요?

A:

1. 데이터브릭스 워크스페이스에서 Catalog로 이동합니다.
2. 과제용으로 생성한 카탈로그와 스키마를 선택합니다.
3. Volumes 탭에서 Create Volume을 클릭하여 관리형 볼륨을 생성합니다.
4. 생성된 볼륨 안에 `people-10m`, `names`와 같은 디렉토리를 생성합니다.

5. 각 디렉토리로 이동하여 Upload to this volume을 클릭하고, 로컬에 다운로드한 해당 데이터 파일(예: blog.json, parquet 파일)을 업로드합니다.

Q: Pandas DataFrame이 Spark DataFrame보다 느린 이유는 무엇인가요?

A: Pandas DataFrame은 기본적으로 단일 노드(single node)에서 작동하도록 설계되었습니다. 즉, 데이터를 하나의 머신 메모리 내에서 처리합니다. 반면 Spark DataFrame은 분산 엔진으로, 여러 워커 노드에 작업을 분할하여 병렬로 처리합니다. 따라서 데이터 세트가 단일 머신의 메모리를 초과하거나 대규모일 경우, Spark DataFrame이 Pandas DataFrame보다 훨씬 빠르게 작업을 완료할 수 있습니다. Spark DataFrame을 Pandas DataFrame으로 변환할 때 느려지는 것은, 분산된 Spark 데이터를 하나의 드라이버 노드로 '수집(collect)'하여 Pandas가 처리할 수 있는 단일 객체로 만들기 때문입니다.

Q: Lab 노트북을 어떻게 공부해야 하나요? 모든 단계를 암기해야 하나요?

A: 모든 단계를 암기할 필요는 없습니다. 중요한 것은 '의도'와 '목적'을 이해하는 것입니다. 어떤 ML 프로젝트든 공통적으로 거치는 7~8단계의 스캐폴딩(scaffolding)을 이해하고, 각 단계에서 어떤 종류의 작업(예: 데이터 요약, 훈련/테스트 분할)이 필요한지 아는 것이 중요합니다. 구체적인 명령어(syntax)는 사용하는 언어나 플랫폼에 따라 달라지지만, 개념은 동일합니다. 또한, Lab 내용은 과제와 높은 상관관계를 가지므로, Lab에 익숙해지면 과제 수행에 큰 도움이 될 것입니다.

2.6 빠르게 훑어보기 (1페이지 요약)

데이터 모델링의 핵심

데이터 모델링은 방대한 데이터를 비즈니스 관점에서 조직화하고, 효율적인 저장 및 검색을 위해 구조화하는 과정입니다.

- **3단계:** 개념적(비즈니스), 논리적(구조), 물리적(구현) 모델로 점진적 구체화.
- **이점:** 데이터 가치 극대화, 시각화, 공통 어휘, 조기 오류 수정.

NoSQL 모델링

관계형 DB와 달리 유연한 스키마와 비정규화를 선호하며, 조인 대신 데이터 임베딩을 통해 읽기 성능을 최적화합니다.

- **유형:** 키-값, 빅 테이블, 문서, 그래프 DB 등.
- **특징:** 질문 중심 사고, 비정규화, 와이드 테이블.

메타데이터의 중요성

'데이터에 대한 데이터'로, 데이터의 의미, 출처, 사용법 등을 설명하여 데이터 이해, 관리, 활용을 돕습니다.

- **유형:** 비즈니스, 기술, 운영 메타데이터.
- **관리:** 데이터 카탈로그를 통해 메타데이터를 저장하고 검색합니다.

OLTP vs. OLAP

OLTP: 실시간 트랜잭션 처리, 정규화된 ER 모델, 빠른 읽기/쓰기. **OLAP:** 대규모 데이터 분석, 비정규화된 차원 모델(스타/스노우플레이크), 복잡한 쿼리.

데이터 웨어하우스 모델링

- **인본:** 탐다운, 3NF 정규화, 단일 진실의 원천.
- **킴볼:** 바텀업, 차원 모델(스타/스노우플레이크), 보고/분석 최적화.
- **데이터 볼트:** 허브, 링크, 위성으로 변경 관리 및 감사에 강점.

데이터 아키텍처 레이어

- **브론즈:** 원본 데이터 수집 (최소한의 변경).
- **실버:** 데이터 정제, 변환, 비즈니스 규칙 적용 (데이터 컨포먼스).
- **골드:** 분석 및 보고에 최적화된 집계 데이터 (스타 스키마, 데이터 마트).

데이터 형식 및 압축

형식: 텍스트(CSV, JSON) vs. 바이너리(Parquet, Avro), 행 기반 vs. 컬럼 기반. 분석에는 컬럼 기반 바이너리 형식이 유리. **델타 레이크:** Parquet + ACID 트랜잭션, 스키마 진화, 시간 여행. **압축:** 디스크 공간 절약, I/O 감소. 무손실 압축(데이터베이스)과 손실 압축(이미지) 구분.

데이터 프로파일링

데이터 품질(정확성, 신뢰성)을 분석하여 문제점을 식별하고 해결합니다.

- **기법:** 구조적/콘텐츠/관계 발견, 상관관계 분석, 시각화.
- **도구:** Spark describe(), summary().

Lab 01: 주택 가격 예측

데이터브릭스에서 Scikit-learn을 활용한 ML 워크플로우 실습.

1. 데이터 로드, 전처리 (Delta 테이블 저장).
2. 데이터 탐색, 시각화 (가격-면적 상관관계).
3. 특징 엔지니어링, 훈련/테스트 분할.
4. 선형 회귀 모델 학습, 예측.
5. 모델 평가 (RMSSE, R2), 잔차 시각화.

Chapter 3

Lecture 3

3.1 개요

이번 강의는 데이터 엔지니어링의 핵심인 데이터 파이프라인에 초점을 맞춥니다. 데이터 파이프라인은 데이터를 한 지점에서 다른 지점으로 이동시키거나, 여러 소스에서 데이터를 수집하고 변환하여 다양한 목적지로 저장하는 과정을 자동화하는 시스템입니다. 특히 ETL(Extract, Transform, Load) 및 ELT(Extract, Load, Transform)와 같은 특정 유형의 파이프라인을 포함하여, 배치(Batch) 및 스트리밍(Streaming) 처리 방식의 차이점과 필요성을 깊이 있게 탐구합니다.

강의는 이전 수업에서 다룬 개념들을 복습하는 것으로 시작하여, 스트리밍 데이터 처리의 필요성과 기본 개념을 소개합니다. 이후 배치와 스트리밍 처리의 장단점을 비교하고, Lambda 및 Kappa 아키텍처와 같은 데이터 파이프라인 설계 패턴을 상세히 설명합니다. 또한, 데이터 파이프라인에서 발생할 수 있는 실패, 트레이드오프, 그리고 모범 사례에 대해서도 논의합니다.

강의의 후반부에는 Lab 02 실습을 통해 실제 네트워크 로그 데이터를 파싱하고 분석하는 과정을 직접 경험합니다. 이를 통해 이론적 지식을 실제 Databricks 환경에서 어떻게 적용하는지 배울 수 있습니다. 마지막으로, 과제 2에 대한 안내와 함께 질의응답 시간을 가집니다.

3.2 용어 정리

데이터 엔지니어링 분야에서 자주 사용되는 핵심 용어들을 정리했습니다. 각 용어의 쉬운 설명과 함께 원어 및 비고를 통해 이해를 돕습니다.

용어	쉬운 설명	원어	비고
3차 정규형	데이터 중복을 줄이고 무결성을 높이는 관계형 DB 설계 원칙	Third Normal Form (3NF)	관계형 데이터베이스에 주로 적용
비정규화	쿼리 성능 향상을 위해 데이터를 중복 저장하거나 통합하는 과정	Denormalization	NoSQL/비관계형 데이터베이스에 주로 적용
ETL	데이터 추출, 변환, 적재의 세 단계로 이루어진 데이터 처리 과정	Extract, Transform, Load	전통적인 배치 처리 방식
ELT	데이터 추출, 적재 후 변환하는 과정	Extract, Load, Transform	클라우드 환경에서 유연한 처리 가능
스타 스키마	중앙의 팩트 테이블과 주변의 차원 테이블로 구성된 데이터 웨어하우스 모델	Star Schema	단순하고 쿼리 성능이 좋음
스노우플레이크 스키마	스타 스키마에서 차원 테이블이 추가로 정규화된 형태	Snowflake Schema	데이터 중복이 적고 복잡도가 높음
키-값 저장소	고유한 키와 해당 키에 연결된 값으로 데이터를 저장하는 방식	Key-Value Store	S3, Redis 등. 단순하고 확장성이 뛰어남
문서 저장소	JSON, XML 등 문서 형태로 데이터를 저장하는 방식	Document Store	MongoDB, CouchDB 등. 유연한 스키마
컬럼형 저장소	데이터를 행이 아닌 컬럼 단위로 저장하여 분석 쿼리 성능을 높이는 방식	Columnar Data Store	Cassandra, DynamoDB 등. OLAP에 적합
Parquet	컬럼형 바이너리 데이터 저장 형식	Parquet	Apache Parquet. 효율적인 압축 및 쿼리 성능
Delta Lake	데이터 레이크에 ACID 트랜잭션, 스키마 관리 등을 제공하는 오픈 소스 저장 계층	Delta Lake	Databricks에서 개발. 배치/스트리밍 통합
메타데이터	데이터에 대한 데이터. 데이터의 맥락과 구조를 설명	Metadata	스키마, 트랜잭션 로그 등
뷰	쿼리 정의를 저장하고, 참조 시마다 쿼리를 실행하여 결과를 생성하는 가상 테이블	View (Virtual Table)	실제 데이터를 저장하지 않음
구체화된 뷰	쿼리 결과를 미리 계산하여 저장해두는 뷰	Materialized View	쿼리 성능 향상. 데이터가 오래될 수 있음
데이터 파이프라인	데이터의 흐름을 자동화하고 관리하는 일련의 과정	Data Pipeline	데이터 통합 및 엔지니어링의 핵심
배치 처리	일정량의 데이터를 모아 한 번에 처리하는 방식	Batch Processing	보고서, 청구서 처리 등
스트리밍 처리	데이터가 생성되는 즉시 실시간으로 처리하는 방식	Streaming Processing	사기 탐지, 실시간 대시보드 등
마이크로 배치	스트리밍 데이터를 작은 배치 단위로 나누어 처리하는 방식	Microbatch	Spark Structured Streaming의 기본 모드
체크포인트	스트리밍 처리 중 마지막으로 성공적으로 처리된 지점을 기록	Checkpoint	장애 발생 시 복구 지점
워터마크	스트리밍 데이터에서 지연 도착 데이터를 처리하는 기준 시간	Watermark	너무 늦게 도착한 데이터는 무시
Lambda 아키텍처	배치 레이어와 스트리밍 레이어를 분리하여 운영하는 데이터 처리 아키텍처	Lambda Architecture	초기 스트리밍 시스템에서 많이 사용
Kappa 아키텍처	모든 데이터를 스트리밍으로 처리하는 단일 레이어 아키텍처	Kappa Architecture	Lambda의 복잡성을 줄인 현대적 방식
Kafka	분산 스트리밍 플랫폼	Kafka	고성능 메시지 큐 및 스트리밍 처리
Flink	진정한 실시간 스트리밍 처리 프레임워크	Flink	매우 낮은 지연 시간, 복잡 이벤트 처리 (CEP)
Spark Structured Streaming	Apache Spark 기반의 스트리밍 처리 API	Spark Structured Streaming	배치/스트리밍 통합 API, Lakehouse의 기반
Unity Catalog	Databricks의 통합 데이터 거버넌스 솔루션	Unity Catalog	메타데이터 관리, 접근 제어, 데이터 계보
Photon	Databricks의 고성능 Spark 엔진	Photon	C++로 재작성되어 빠른 쿼리 성능 제공
UDF	사용자가 직접 정의하여 재사용할 수 있는 함수	User-Defined Function	데이터 변환 로직을 캡슐화

3.3 핵심 개념 및 원리

3.3.1 데이터베이스 정규화 및 비정규화

개념

정규화(Normalization)는 관계형 데이터베이스의 중복을 줄이고 데이터 무결성을 높이는 과정입니다. 반대로 **비정규화(Denormalization)**는 쿼리 성능 향상을 위해 의도적으로 데이터를 중복시키거나 통합하는 과정입니다.

3차 정규형 (Third Normal Form, 3NF)

- **한 줄 핵심 요약:** 데이터 중복을 최소화하고 데이터의 일관성을 유지하기 위한 관계형 데이터베이스 설계 규칙입니다.
- **직관적 비유:** 도서관에서 책 정보를 관리할 때, 모든 책의 저자 정보(이름, 생년월일, 국적)를 각 책 레코드마다 반복해서 적는 대신, 저자 정보를 별도의 목록으로 만들고 책 레코드에서는 저자 ID만 참조하는 것과 같습니다. 이렇게 하면 저자 정보가 바뀌어도 한 곳만 수정하면 됩니다.
- **기술적 설명:**
 - * **목표:** 데이터 중복을 줄이고 데이터 무결성(Data Integrity)을 보장합니다.
 - * **적용:** 주로 관계형 데이터베이스(Relational Databases)에서 사용됩니다.
 - * **원리:** 테이블을 여러 개의 작은 테이블로 분리하고, 외래 키(Foreign Key)를 사용하여 테이블 간의 관계를 정의합니다.

비정규화 (Denormalization)

- **한 줄 핵심 요약:** 쿼리 성능을 극대화하기 위해 데이터 중복을 허용하고 여러 테이블의 데이터를 하나의 테이블로 결합하는 전략입니다.

- **직관적 비유:** 도서관에서 특정 주제의 책 목록을 자주 찾아야 할 때, 매번 책 정보와 저자 정보를 따로 찾아 결합하는 것이 번거롭다면, 아예 자주 찾는 책 목록에는 저자 이름까지 함께 기록해두는 것과 같습니다. 이렇게 하면 검색은 빨라지지만, 저자 정보가 바뀌면 여러 곳을 수정해야 하는 단점이 있습니다.
- **기술적 설명:**
 - * **목표:** 조인(Join) 작업의 필요성을 줄여 쿼리 성능을 향상시킵니다.
 - * **적용:** NoSQL 또는 비관계형 데이터베이스(Non-Relational Databases)에서 주로 사용됩니다. 조인 기능이 없거나 제한적인 경우에 특히 유용합니다.
 - * **원리:** 여러 테이블의 측정값(measures)을 하나의 테이블로 결합하여 정보의 중복을 허용합니다.

Table 3.1: 정규화 vs 비정규화 비교

특징	정규화	비정규화
목표	데이터 중복 감소, 무결성 보장	쿼리 성능 향상
데이터 중복	최소화	허용 (의도적)
테이블 구조	여러 개의 작은 테이블	통합된 큰 테이블
주요 사용처	관계형 데이터베이스 (OLTP)	NoSQL, 데이터 웨어하우스 (OLAP)
조인 필요성	높음	낮음
데이터 일관성	높음	낮을 수 있음 (중복 데이터 관리 필요)

3.3.2 ETL과 ELT

개념

ETL(Extract, Transform, Load)은 데이터를 추출, 변환, 적재하는 전통적인 방식이며, **ELT(Extract, Load, Transform)**는 데이터를 추출, 적재한 후 변환하는 방식입니다.

ETL (Extract, Transform, Load)

- **한 줄 핵심 요약:** 소스 시스템에서 데이터를 추출(Extract)하여 필요한 형태로 변환(Transform)한 후, 대상 시스템에 적재(Load)하는 과정입니다.
- **직관적 비유:** 해외에서 수입한 원자재(데이터)를 국내 공장(변환 시스템)에서 가공하여 완제품(변환된 데이터)을 만든 후, 창고(대상 시스템)에 보관하는 것과 같습니다.
- **기술적 설명:**
 - * **Extract:** 다양한 소스(데이터베이스, 파일 등)에서 데이터를 가져옵니다.
 - * **Transform:** 데이터 품질 향상, 형식 통일, 집계, 정규화 등 분석에 적합한 형태로 데이터를 변경합니다. 이 단계는 대상 시스템에 적재하기 전에 이루어집니다.
 - * **Load:** 변환된 데이터를 데이터 웨어하우스나 데이터 마트와 같은 대상 시스템에 저장합니다.
 - * **특징:** 주로 전통적인 배치 지향(Batch-oriented) 처리 방식에 사용되며, 들어오는 데이터의 스키마(Schema)가 비교적 잘 알려져 있을 때 효과적입니다.

ELT (Extract, Load, Transform)

- **한 줄 핵심 요약:** 소스 시스템에서 데이터를 추출(Extract)하여 대상 시스템에 먼저 적재(Load)한 후, 대상 시스템 내에서 변환(Transform)하는 과정입니다.

- **직관적 비유:** 해외에서 수입한 원자재(데이터)를 일단 국내 창고(대상 시스템)에 그대로 보관한 후, 필요할 때마다 창고 내에서 가공하여 사용하는 것과 같습니다.
- **기술적 설명:**
 - * **Extract:** 다양한 소스에서 데이터를 가져옵니다.
 - * **Load:** 원본 데이터를 거의 그대로 데이터 레이크(Data Lake)와 같은 대상 시스템에 저장합니다.
 - * **Transform:** 대상 시스템(주로 클라우드 기반의 확장 가능한 스토리지 및 컴퓨팅 자원) 내에서 데이터를 변환합니다. 이 방식은 클라우드 환경의 탄력적인 자원을 활용하여 대규모 데이터 처리 및 유연한 스키마 변경에 유리합니다.
 - * **특징:** 들어오는 데이터의 스키마가 덜 알려져 있거나, 나중에 다양한 방식으로 데이터를 분석하고 싶을 때 유용합니다. 원본 데이터를 보존하면서 필요에 따라 여러 변환을 적용할 수 있습니다.

데이터 웨어하우스 및 데이터 마트 스키마

데이터 웨어하우스와 데이터 마트는 분석 및 보고를 위해 데이터를 저장하는 시스템입니다. 이들은 주로 두 가지 스키마를 사용합니다.

- **스타 스키마 (Star Schema):**
 - * 중앙에 하나의 팩트 테이블(Fact Table)이 있고, 그 주위를 여러 차원 테이블(Dimension Table)이 둘러싸는 형태입니다.
 - * 팩트 테이블은 측정값(예: 판매량, 금액)을 포함하고, 차원 테이블은 측정값의 맥락(예: 시간, 제품, 고객)을 제공합니다.
 - * 단순하고 이해하기 쉬우며, 쿼리 성능이 좋습니다.
- **스노우플레이크 스키마 (Snowflake Schema):**
 - * 스타 스키마와 유사하지만, 차원 테이블이 추가로 정규화되어 여러 계층의 차원 테이블을 가질 수 있습니다.
 - * 데이터 중복이 적지만, 조인(Join)이 더 많이 필요하여 쿼리 복잡도가 높아지고 성능에 영향을 줄 수 있습니다.

3.3.3 데이터 저장소 유형

데이터는 다양한 방식으로 저장될 수 있으며, 각 방식은 특정 사용 사례에 최적화되어 있습니다.

키-값 저장소 (Key-Value Store)

- **한 줄 핵심 요약:** 고유한 키(Key)와 해당 키에 연결된 값(Value)으로 데이터를 저장하는 가장 단순한 형태의 데이터 저장소입니다.
- **직관적 비유:** 사물함에 물건을 보관할 때, 사물함 번호(키)와 그 안에 들어있는 물건(값)을 매칭하여 저장하는 것과 같습니다. 번호를 알면 즉시 물건을 찾을 수 있습니다.
- **기술적 설명:**
 - * **예시:** JSON 파일, Amazon S3 (객체 스토리지).
 - * **특징:** 매우 단순한 저장 방식이지만, 높은 확장성(Scalability)과 빠른 접근 속도(Fast Access)를 제공합니다. 값은 어떤 형태의 객체(Object)도 될 수 있습니다.

문서 저장소 (Document Store)

- **한 줄 핵심 요약:** 데이터를 JSON, XML, BSON과 같은 문서(Document) 형태로 저장하는 데이터 저장소입니다.
- **직관적 비유:** 서류 캐비닛에 각 서류철(문서)을 통째로 보관하는 것과 같습니다. 각 서류철은 다양한 정보를 포함할 수 있으며, 필요에 따라 서류철의 내용을 유연하게 변경할 수 있습니다.
- **기술적 설명:**
 - * **예시:** MongoDB, CouchDB.
 - * **특징:** 키-값 저장소보다 더 복잡한 구조의 데이터를 저장할 수 있으며, 스키마가 유연하여 데이터 모델 변경에 용이합니다.

컬럼형 데이터 저장소 (Columnar Data Store)

- **한 줄 핵심 요약:** 데이터를 행(Row) 단위가 아닌 컬럼(Column) 단위로 저장하여 분석 쿼리 성능을 최적화하는 데이터 저장소입니다.
- **직관적 비유:** 일반적인 스프레드시트가 행별로 데이터를 저장한다면, 컬럼형 저장소는 각 열(컬럼)의 데이터를 한데 모아 저장하는 방식입니다. 특정 열의 데이터만 필요할 때 전체 행을 읽을 필요 없이 해당 열만 빠르게 읽을 수 있습니다.
- **기술적 설명:**
 - * **예시:** Cassandra, DynamoDB.
 - * **특징:** 관계형 데이터베이스가 행 단위로 데이터를 저장하는 것과 달리, 컬럼 단위 저장은 특정 컬럼에 대한 집계(Aggregation) 쿼리나 분석 작업에서 뛰어난 성능을 발휘합니다.
 - * **왜 필요한가?:** 데이터 분석 시에는 특정 몇몇 컬럼의 데이터만 필요한 경우가 많습니다. 예를 들어, '총 판매액'을 계산할 때는 '판매 금액' 컬럼만 필요하고, '고객 이름'이나 '주소' 컬럼은 필요하지 않습니다. 컬럼형 저장소는 이처럼 필요한 컬럼만 효율적으로 읽어올 수 있어 I/O(입출력) 비용을 크게 줄이고 쿼리 속도를 향상시킵니다.

3.3.4 데이터 형식: 바이너리 vs 텍스트, Parquet

바이너리 형식 (Binary Formats) 선호 이유

- **한 줄 핵심 요약:** 바이너리 형식은 텍스트 형식보다 데이터 압축률이 높고 읽기/쓰기 속도가 빠르며, 전반적인 성능이 우수합니다.
- **직관적 비유:** 책을 보관할 때, 모든 내용을 손으로 직접 쓰는 것(텍스트)보다 인쇄된 책(바이너리)이 훨씬 작고 가벼우며, 읽기도 빠르고 보관도 효율적인 것과 같습니다.
- **기술적 설명:**
 - * **압축률:** 바이너리 형식은 데이터를 더 효율적으로 압축할 수 있어 저장 공간을 절약합니다.
 - * **성능:** 데이터 파싱(Parsing) 과정이 텍스트보다 단순하여 읽기 및 쓰기 작업이 더 빠릅니다.
 - * **데이터 타입:** 숫자, 날짜 등 다양한 데이터 타입을 텍스트 변환 없이 원시 형태로 저장하여 정확성과 효율성을 높입니다.

Parquet 형식

- **한 줄 핵심 요약:** Parquet은 컬럼형(Columnar) 바이너리 데이터 저장 형식으로, 높은 압축률과 빠른 쿼리 성능을 제공하여 빅데이터 분석에 매우 적합합니다.

- **직관적 비유:** 도서관에서 책을 보관할 때, 모든 책을 한 줄로 쭉 늘어놓는 대신, 같은 종류의 책(예: 소설, 과학, 역사)끼리 모아서 별도의 서가에 보관하는 것과 같습니다. 특정 종류의 책만 찾을 때 훨씬 빠르게 접근할 수 있습니다.
- **기술적 설명:**
 - * **형식:** 컬럼형 바이너리 형식입니다.
 - * **장점:**
 - **효율적인 압축:** 컬럼별로 데이터 타입이 유사하여 높은 압축률을 달성합니다.
 - **빠른 쿼리 성능:** 필요한 컬럼만 읽어올 수 있어 I/O 비용을 줄이고 쿼리 속도를 향상시킵니다.
 - **스키마 진화(Schema Evolution):** 스키마 변경에 유연하게 대응할 수 있습니다.

3.3.5 Delta Lake 형식

개념

Delta Lake는 데이터 레이크에 ACID 트랜잭션, 스키마 관리, 시간 여행(Time Travel) 등의 기능을 추가하여 데이터의 신뢰성과 성능을 높이는 오픈 소스 저장 계층입니다.

- **한 줄 핵심 요약:** Databricks에서 제공하는 오픈 소스 테이블 형식으로, 데이터 레이크에 데이터 웨어하우스의 안정성과 신뢰성을 부여합니다.
- **직관적 비유:** 일반적인 데이터 레이크가 정돈되지 않은 창고라면, Delta Lake는 창고에 재고 관리 시스템, 보안 시스템, 버전 관리 시스템을 도입하여 물건을 더 안전하고 효율적으로 관리할 수 있게 해주는 것과 같습니다.
- **기술적 설명:**
 - * **제공처:** Databricks에서 개발한 오픈 소스 테이블 형식입니다.
 - * **주요 속성 (4가지):**
 1. **ACID 트랜잭션 (Atomicity, Consistency, Isolation, Durability):** 데이터의 안정성과 일관성을 보장합니다. 여러 사용자가 동시에 데이터를 읽고 써도 데이터 손상이나 불일치가 발생하지 않습니다.
 2. **스키마 진화 (Schema Evolution):** 데이터 스키마 변경에 유연하게 대응할 수 있습니다. 새로운 컬럼 추가, 기존 컬럼 타입 변경 등을 쉽게 처리합니다.
 3. **데이터 이력 (History) / 시간 여행 (Time Travel):** 데이터의 모든 변경 이력을 기록하여 특정 시점의 데이터 상태로 되돌아가거나(롤백), 과거 데이터를 조회할 수 있습니다.
 4. **배치 및 스트리밍 모드 통합 (Unifying Batch and Streaming Modes):** 배치 처리와 스트리밍 처리를 동일한 테이블에서 일관된 방식으로 수행할 수 있습니다.
 - * **추가 기능:**
 - **세분화된 삽입 및 삭제 (Fine-grained Inserts and Deletes):** 데이터베이스 내의 데이터를 매우 세분화된 수준에서 쉽게 업데이트하거나 삭제할 수 있습니다.
 - **데이터 버전 관리 (Versioning):** 데이터의 여러 버전을 관리하여 이전 버전으로 돌아가거나 앞으로 이동할 수 있습니다.

3.3.6 메타데이터 (Metadata)

개념

메타데이터는 '데이터에 대한 데이터'를 의미하며, 데이터의 맥락과 구조를 이해하는 데 필요한 정보를 제공합니다.

- **한 줄 핵심 요약:** 데이터 자체의 내용이 아니라, 데이터가 무엇인지, 어떻게 구성되어 있는지, 언제 생성되었는지 등을 설명하는 정보입니다.
- **직관적 비유:** 책의 내용(데이터)이 아니라, 책의 제목, 저자, 출판일, 페이지 수, 목차(메타데이터)와 같습니다. 이 정보들을 통해 책의 내용을 파악하고 관리할 수 있습니다.
- **기술적 설명:**
 - * **역할:** 데이터의 컨텍스트(Context)를 제공하여 데이터를 이해하고 활용하는 데 도움을 줍니다.
 - * **예시:**
 - **스키마 (Schemas):** 테이블의 컬럼 이름, 데이터 타입, 제약 조건 등 데이터의 구조를 정의합니다.
 - **트랜잭션 로그 (Transaction Logs):** 데이터베이스에서 발생한 모든 변경 사항을 기록합니다.
 - **Delta Lake의 로그:** Delta Lake 테이블의 모든 변경 이력과 버전을 관리하는 데 사용됩니다.

3.3.7 테이블 유형

데이터베이스 시스템에서는 다양한 목적과 수명 주기를 가진 테이블 유형을 제공합니다.

영구 테이블 (Permanent Tables)

- **한 줄 핵심 요약:** 데이터의 영속성(Persistence)을 지원하며, 명시적으로 삭제하기 전까지 데이터베이스에 계속 저장됩니다.
- **직관적 비유:** 은행 계좌 정보처럼 한 번 생성되면 계속 유지되고, 필요할 때마다 조회하거나 업데이트할 수 있는 데이터입니다.

임시 테이블 (Temporary Tables)

- **한 줄 핵심 요약:** 특정 세션(Session) 또는 트랜잭션(Transaction) 동안만 존재하며, 세션이 종료되면 자동으로 삭제되는 테이블입니다.
- **직관적 비유:** 웹사이트에서 장바구니에 담은 상품 목록처럼, 웹 브라우저를 닫으면 사라지는 임시 정보와 같습니다.
- **유형:**
 - * **로컬 임시 테이블 (Local Temporary Tables):** 현재 세션 동안만 존재합니다.
 - * **글로벌 임시 테이블 (Global Temporary Tables):** 동일한 클러스터 내의 여러 세션에서 공유될 수 있지만, 여전히 영구적이지는 않습니다.

뷰 (Views) / 가상 테이블 (Virtual Tables)

- **한 줄 핵심 요약:** 쿼리 정의에 의해 생성되는 가상 테이블로, 실제 데이터를 저장하지 않고 참조될 때마다 쿼리를 실행하여 결과를 생성합니다.

- **직관적 비유:** 특정 보고서를 만들 때 필요한 데이터를 매번 여러 테이블에서 가져와 조합하는 대신, 그 조합 과정을 미리 정의해둔 '레시피'와 같습니다. 레시피 자체는 음식이 아니지만, 레시피를 사용하면 항상 신선한 음식을 만들 수 있습니다.
- **기술적 설명:**
 - * **정의:** 뷰는 쿼리(Query)에 의해 정의됩니다.
 - * **동작:** 뷰를 참조할 때마다 기본 쿼리가 실행되어 최신 결과를 반환합니다.

구체화된 뷰 (Materialized Views)

- **한 줄 핵심 요약:** 뷰와 유사하지만, 쿼리 결과를 미리 계산하여 실제 데이터로 저장해두는 테이블입니다.
- **직관적 비유:** 자주 만드는 보고서의 결과(음식)를 미리 만들어 냉장고에 보관해두는 것과 같습니다. 필요할 때 즉시 꺼내 쓸 수 있어 빠르지만, 시간이 지나면 신선도가 떨어질 수 있습니다.
- **기술적 설명:**
 - * **장점:** 쿼리 결과를 캐시(Cache)하여 성능을 크게 향상시킵니다. 집계(Aggregation) 함수나 조인(Join)이 많은 BI(Business Intelligence) 보고서에 유용합니다.
 - * **단점:** 기본 데이터가 변경되어도 구체화된 뷰는 자동으로 업데이트되지 않을 수 있으므로, 주기적인 새로 고침(Refresh)이 필요합니다. 그렇지 않으면 오래된(Stale) 데이터를 반환할 수 있습니다.

스트리밍 테이블 (Streaming Tables)

- **한 줄 핵심 요약:** 스트리밍 파이프라인을 위해 특별히 설계된 테이블로, 새로운 이벤트나 데이터가 들어올 때마다 자동으로 업데이트됩니다.
- **직관적 비유:** 주식 시장의 실시간 시세판처럼, 새로운 거래 정보가 들어올 때마다 화면이 자동으로 갱신되어 항상 최신 정보를 보여주는 것과 같습니다.
- **기술적 설명:**
 - * **특징:** 스트리밍 파이프라인의 일부로, 데이터가 지속적으로 유입됨에 따라 테이블이 자동으로 최신 상태를 유지합니다.

3.3.8 관리형 테이블 (Managed Tables) vs 외부 테이블 (External Tables)

Databricks와 같은 플랫폼에서는 데이터와 메타데이터를 관리하는 방식에 따라 테이블을 두 가지로 분류합니다.

Table 3.2: 관리형 테이블 vs 외부 테이블 비교

특징	관리형 테이블 (Managed Tables)	외부 테이블 (External Tables)
데이터 위치	웨어하우스 경로 내에서 플랫폼이 관리	Databricks 외부에서 생성 및 관리
데이터/메타데이터 관리	데이터베이스(플랫폼)가 모두 관리	데이터베이스는 메타데이터만 관리, 데이터는 외부 파일 시스템에 존재
테이블 삭제 시 동작	메타데이터와 실제 데이터 모두 삭제	메타데이터만 삭제, 외부 파일은 유지
공유 용이성	상대적으로 공유하기 어려움	외부 파일이므로 공유하기 쉬움
데이터 손실 위험	테이블 삭제 시 데이터도 손실되므로 높음	테이블 삭제 시 데이터는 유지되므로 낮음
생성 코드	형식(Format)과 위치(Location)를 지정하지 않으면 관리형으로 간주 (더 간단)	형식과 위치를 명시적으로 지정해야 함

3.3.9 데이터 파이프라인 개요

개념

데이터 파이프라인은 데이터 통합 및 엔지니어링의 핵심 요소로, 데이터를 한 지점에서 다른 지점으로 이동시키거나, 여러 소스에서 수집, 변환하여 다양한 목적지로 저장하는 과정을 자동화하는 시스템입니다.

정의 및 역할

- **한 줄 핵심 요약:** 데이터를 소스에서 목적지까지 이동, 변환, 저장하는 일련의 자동화된 과정입니다.
- **직관적 비유:** 공장에서 원자재(데이터)를 들여와 여러 공정(변환)을 거쳐 완제품(처리된 데이터)을 만들고, 이를 최종 소비자(분석 시스템, 애플리케이션)에게 전달하는 컨베이어 벨트 시스템과 같습니다.
- **기술적 설명:**
 - * **단순한 형태:** 데이터를 A 지점에서 B 지점으로 단순히 이동시키는 것일 수 있습니다.
 - * **복잡한 형태:** 여러 데이터 소스에서 데이터를 수집(Ingest)하고, 여러 변환(Transformation)을 거쳐, 여러 출력 위치(Output Locations) 또는 목적지(Destinations)에 데이터를 저장하는 복잡한 과정일 수 있습니다.
 - * **ETL 파이프라인:** 데이터 파이프라인의 특별한 경우로, 일반적으로 데이터를 수집, 변환한 후 분석 또는 애플리케이션, 머신러닝/AI 시스템에서 소비할 수 있는 형태로 출력합니다. 비즈니스 요구 사항과 의사 결정을 지원하도록 설계됩니다.

Figure 3.1: 데이터 파이프라인과 ETL 파이프라인의 관계 (벤 다이어그램)

위 그림은 데이터 파이프라인이 더 넓은 개념이며, ETL 파이프라인은 데이터 파이프라인의 특정 유형임을 보여주는 벤 다이어그램을 나타냅니다.

데이터 파이프라인의 생명 주기

데이터 파이프라인은 소프트웨어 애플리케이션과 유사하게 관리됩니다.

1. **설계 (Design):** 비즈니스 요구 사항과 데이터 흐름을 정의합니다.
2. **구현 (Implement):** 정의된 설계에 따라 코드를 작성하고 시스템을 구축합니다.
3. **테스트 (Test):** 파이프라인이 예상대로 작동하고 데이터가 정확하게 처리되는지 검증합니다.
4. **배포 (Deploy):** 운영 환경에 파이프라인을 배포합니다.
5. **모니터링 (Monitor):** 파이프라인의 성능, 오류, 데이터 품질 등을 지속적으로 감시합니다.
6. **변경 처리 및 업데이트 (Handle Changes and Update):** 데이터 소스, 스키마, 비즈니스 로직 등의 변경 사항에 대응하여 파이프라인을 업데이트하고 재배포합니다.

자동화 및 트리거

- **자동화 (Automation):** 파이프라인이 수동 개입 없이 자체적으로 실행될 수 있도록 자동화하는 것이 중요합니다. 이는 주요 장점 중 하나입니다.
- **트리거 (Triggers):** 파이프라인이 언제 실행될지 결정하는 요소입니다.
 - * **파일 기반 (File-based):** 파일 시스템에 새로운 파일이 나타날 때.

- * 스케줄 기반 (Schedule-based): Cron 스케줄과 같이 정해진 시간에 주기적으로 실행.
- * 수동 (Manual): 사용자가 직접 실행.

데이터 파이프라인의 장점

- IT 팀 부하 감소: 자동화를 통해 IT 팀의 수동 작업을 줄여줍니다.
- 데이터 소비 및 전달 자동화: 사용자에게 데이터가 더 빠르고 효율적으로 전달됩니다.
- 클라우드 기반 동적 자원 활용: 데이터 볼륨이 증가할 때 클라우드의 탄력적인 자원(Elasticity)을 활용하여 부하를 처리할 수 있습니다.
- 실시간 분석 및 의사 결정 지원: 비즈니스에 중요한 실시간 분석 및 의사 결정을 가능하게 합니다.

데이터 파이프라인의 과제

- 데이터 문제: 스키마 변경, 데이터 품질 저하 등 데이터 자체의 문제가 파이프라인 오류를 유발할 수 있습니다. 이러한 문제를 해결하는 데 시간이 걸리고 서비스 중단(Outages)을 초래할 수 있습니다.
- 프레임워크 의존성: 파이프라인이 구축된 프레임워크에 강하게 의존하는 경향이 있습니다. 프레임워크 변경은 파이프라인에 문제를 일으킬 수 있습니다.

3.3.10 데이터 파이프라인 유형

테이블 유형과 마찬가지로, 데이터 파이프라인도 다양한 처리 방식과 목적에 따라 여러 유형으로 나뉩니다.

배치 파이프라인 (Batch Pipelines)

- 한 줄 핵심 요약: 일정량의 데이터를 모아 한 번에 처리하는 방식입니다.
- 직관적 비유: 세탁물을 모아서 한 번에 세탁하는 것과 같습니다. 일정량이 모일 때까지 기다렸다가 처리하므로 효율적이지만, 즉각적인 처리는 어렵습니다. **사용 사례:** 보고서 생성, 청구서 처리, 과거 데이터 분석(Historical Analysis) 등.

스트리밍 파이프라인 (Streaming Pipelines) / 실시간 파이프라인 (Real-time Pipelines)

- 한 줄 핵심 요약: 데이터가 생성되는 즉시 지속적으로 수집하고 처리하는 방식입니다.
- 직관적 비유: 수도꼭지에서 물이 나오는 즉시 컵에 받아 마시는 것과 같습니다. 데이터가 들어오는 대로 즉시 처리하므로 최신 정보를 얻을 수 있습니다.
- 사용 사례: 사기 탐지(Fraud Detection), 실시간 대시보드, 추천 시스템, IoT(사물 인터넷) 장치 이벤트 처리 등.

ETL 및 ELT 파이프라인

- ETL (Extract, Transform, Load): 전통적인 배치 지향 처리 방식으로, 데이터를 추출, 변환한 후 적재합니다. 들어오는 데이터의 스키마가 잘 알려져 있을 때 적합합니다.
- ELT (Extract, Load, Transform): 데이터를 추출, 적재한 후 변환합니다. 들어오는 데이터의 스키마가 덜 알려져 있거나 유연한 분석이 필요할 때 적합합니다.

기타 파이프라인 유형

- **데이터 복제 (Data Replication):** 데이터를 다른 위치로 복사하여 백업 또는 고가용성을 제공합니다.
- **머신러닝 파이프라인 (Machine Learning Pipelines):** 머신러닝 모델 생성 과정을 포함합니다. (추후 강의에서 다룰 예정)
- **워크플로우 및 오케스트레이션 파이프라인 (Workflow and Orchestration Pipelines):** 데이터 소스와 목적지를 변경할 수 있도록 더 유연하고 구성 가능한 파이프라인입니다.

3.3.11 스트리밍 데이터 파이프라인의 필요성 및 활용

개념

스트리밍 데이터 파이프라인은 데이터가 생성되는 즉시 처리하여 **실시간 인사이트**를 제공하고, **빠른 의사 결정**을 지원하는 데 필수적입니다.

왜 스트리밍 데이터 파이프라인이 중요한가?

- **속도 (Speed):** 모든 것은 속도에 관한 것입니다. 들어오는 데이터를 가능한 한 빨리 분석하여 인사이트를 제공해야 합니다.
- **실시간 처리 (Real-time Processing):** 데이터 수집, 분석, 저장에 모두 실시간으로 이루어져야 합니다.
- **대규모 데이터 처리 (Handling Large Amounts of Data):** 대량의 데이터를 효율적으로 처리할 수 있어야 합니다.
- **시계열 데이터 (Time Series Data):** 타임스탬프가 찍히고 시간 순서로 정렬되어 들어오는 데이터에 특히 잘 작동합니다.

스트리밍이 필요한 상황

- **센서 데이터 (Sensor Data):** IoT 장치에서 실시간으로 수집되는 온도, 습도, 위치 정보 등.
- **광고 서빙 (Ad Serving):** 사용자 행동에 기반한 실시간 광고 추천.
- **보안 애플리케이션 (Security Applications):** 이상 징후 탐지, 침입 감지.
- **클릭스트림 데이터 (Clickstream Data):** 웹사이트 사용자 행동 분석.
- **결제 거래 (Payment Transactions):** 실시간 사기 탐지.

배치 작업을 스트리밍으로 구현하는 방법

- 흥미롭게도, 배치 작업도 스트리밍 파이프라인으로 구현할 수 있습니다.
- **원리:** 스트리밍 파이프라인의 트리거(Trigger) 주기를 길게 설정(예: 24시간마다 한 번)하면, 이는 사실상 배치 처리와 동일하게 작동합니다.
- **장점:** 스트리밍 프레임워크가 제공하는 강력한 기능(예: 체크포인트, 오류 복구, exactly-once 처리)을 배치 작업에도 활용할 수 있습니다.

비즈니스 요구 사항에 집중

- 데이터 파이프라인을 설계할 때 가장 중요한 것은 IT 부서에 편리한 것보다 비즈니스 요구 사항에 집중하는 것입니다.
- 고려 사항:
 - * 데이터 접근 속도 (Speed of Access)
 - * 데이터 정확성 (Accuracy of Data)
 - * 어떤 데이터가 필요한가?
 - * 비즈니스가 어떤 질문에 대한 답변을 필요로 하는가?

3.3.12 스트리밍 핵심 개념

스트리밍 데이터 파이프라인을 이해하고 설계하는 데 필요한 주요 개념들입니다.

- 소스 (Source), 처리 (Processing), 싱크 (Sink):
 - * 모든 데이터 파이프라인은 데이터를 가져오는 소스, 데이터를 변환하거나 분석하는 처리 구성 요소, 그리고 처리된 데이터를 쓰는 싱크로 구성됩니다.
- 파일 기반 스트리밍 (File-based Streaming) vs 이벤트 기반 스트리밍 (Event-based Streaming):
 - * 파일 기반: 데이터가 파일에서 읽히거나 파일로 쓰여집니다. (예: 새로운 파일이 특정 디렉토리에 도착하면 처리)
 - * 이벤트 기반: 데이터가 이벤트 형태로 소비되거나 생성됩니다. (예: 메시지 큐에서 이벤트를 읽거나 이벤트를 발행)
- 마이크로 배치 (Microbatch) vs 연속 스트리밍 (Continuous Streaming):
 - * 마이크로 배치: 스트리밍 데이터를 작은 덩어리(배치)로 나누어 주기적으로 처리합니다. (예: Spark Structured Streaming의 기본 모드)
 - * 연속 스트리밍: 들어오는 스트림을 끊임없이 처리합니다. (예: Flink)
- 처리 트리거 (Processing Trigger):
 - * 마이크로 배치를 시작하는 데 사용되는 메커니즘입니다. (예: 1초마다, 5초마다)
- 출력 모드 (Output Modes):
 - * 스트림에 데이터를 쓸 때 데이터를 처리하는 방식입니다.
 - * Append (추가): 새로운 데이터만 추가합니다.
 - * Overwrite (덮어쓰기): 기존 데이터를 모두 삭제하고 새로운 데이터로 덮어씁니다.
 - * Update (업데이트): 기존 데이터를 업데이트하거나 새로운 데이터를 추가합니다.
- 체크포인트 (Checkpoint):
 - * 데이터가 올바르게 기록된 마지막 지점을 기록합니다.
 - * 장애 발생 시 체크포인트 지점으로 돌아가 복구하고, 그 이후의 데이터만 재처리하여 데이터 손실이나 중복을 방지합니다.
- 윈도우 (Window):
 - * 집계(Aggregation) 함수(예: 합계, 개수, 평균)에 사용되는 시간 간격입니다.
 - * 특정 시간 범위 내의 데이터를 그룹화하여 집계합니다. (예: 지난 5분간의 평균 온도)
- 워터마크 (Watermark):
 - * 지연 도착 데이터(Late Arriving Data)를 처리하기 위한 기준 시간입니다.

- * 워터마크를 넘어 너무 늦게 도착한 데이터는 오래된(Stale) 것으로 간주되어 무시될 수 있습니다. 워터마크 내의 데이터는 허용되고 처리됩니다.
- 스트림 작업 (Stream Operations):
 - * 스트리밍 파이프라인 내에서 발생하는 모든 작업(필터링, 변환, 집계 등)을 의미합니다.
- 인스트림 분석 (In-stream Analytics):
 - * 스트림에서 데이터를 읽어 즉시 처리하며, 데이터를 디스크나 파일에 쓰지 않습니다.
 - * 데이터를 디스크에 쓰고 다시 읽는 과정을 생략하므로 더 빠릅니다. 주로 메모리 내에서 처리됩니다.

3.3.13 데이터 처리 속도 스펙트럼

모든 사용 사례가 실시간 처리를 요구하는 것은 아니며, 비즈니스 요구 사항에 따라 적절한 처리 속도를 선택하는 것이 중요합니다.

Table 3.3: 데이터 처리 속도 스펙트럼 및 사용 사례

처리 속도	특징	사용 사례
배치 (Batch)	데이터를 모아 주기적으로 한 번에 처리	계량 및 청구, 임시 분석, 비즈니스 인텔리전스 (BI)
준실시간 (Near Real-time)	데이터를 몇 분 이내에 처리하여 사용 가능	모바일/IoT 데이터 처리, 로그 수집, 광고 서빙, 클릭스트림 분석
실시간 (Real-time)	데이터를 몇 초 또는 1초 미만에 처리하여 즉시 반응	신용카드 사기 탐지, 금융 자산 거래, 멀티플레이어 게임, ML 추론, 환자 모니터링

주의

중요: 모든 사용 사례에 실시간 처리가 필요한 것은 아닙니다. 불필요하게 실시간 시스템을 구축하면 비용과 복잡성이 증가하므로, 사용 사례에 맞는 적절한 처리 속도를 선택해야 합니다.

3.3.14 Lambda vs Kappa 아키텍처

데이터 파이프라인을 설계하는 두 가지 주요 아키텍처 패턴입니다.

Lambda 아키텍처 (Lambda Architecture)

- 한 줄 핵심 요약: 배치 처리와 스트리밍 처리를 병렬로 수행하는 두 개의 분리된 레이어를 사용하여 데이터의 속도와 신뢰성을 모두 확보하려는 아키텍처입니다.
- 직관적 비유: 중요한 문서를 보관할 때, 원본 문서는 안전한 금고(배치 레이어)에 보관하고, 급하게 필요한 정보는 복사본을 만들어 빠르게 전달(스트리밍 레이어)하는 것과 같습니다. 두 정보는 나중에 합쳐져 최종 보고서가 됩니다.
- 기술적 설명:
 - * 구성:
 1. 배치 레이어 (Batch Layer): 대량의 데이터를 배치 모드로 처리하여 정확하고 신뢰성 있는 결과를 생성합니다. (더 신뢰할 수 있다고 간주됨)
 2. 스트리밍 레이어 (Streaming Layer) / 속도 레이어 (Speed Layer): 들어오는 데이터를 실시간 또는 준실시간으로 처리하여 빠른 결과를 제공합니다.
 3. 서빙 레이어 (Serving Layer): 배치 레이어와 스트리밍 레이어의 결과를 결합하여 사용자에게 제공합니다.
 - * 장점: 배치 처리의 신뢰성과 스트리밍 처리의 속도를 모두 활용할 수 있습니다.

* 단점:

- **데이터 처리 중복:** 동일한 데이터를 두 가지 방식으로 처리해야 하므로 복잡성이 증가합니다.
- **결과 일치 문제:** 배치 레이어와 스트리밍 레이어의 결과를 결합하고 일치시키는 과정 (Reconciliation)이 어렵습니다.
- **유지보수:** 두 개의 독립적인 코드베이스와 시스템을 유지보수해야 합니다.

Kappa 아키텍처 (Kappa Architecture)

- **한 줄 핵심 요약:** 모든 데이터를 스트리밍으로 처리하는 단일 레이어를 사용하여 Lambda 아키텍처의 복잡성을 줄이고 시스템을 단순화한 아키텍처입니다.
- **직관적 비유:** 모든 문서를 디지털화하여 하나의 클라우드 시스템에 저장하고, 필요할 때마다 이 시스템에서 실시간으로 정보를 검색하거나 보고서를 생성하는 것과 같습니다. 원본과 복사본을 따로 관리할 필요가 없습니다.
- **기술적 설명:**
 - * **구성:** 단일 스트리밍 레이어 (Single Streaming Layer)만 존재합니다. 이 스트리밍 레이어가 마이크로 배치 (Microbatches) 또는 연속 스트리밍 방식으로 배치 처리와 스트리밍 처리를 모두 지원합니다.
 - * **장점:**
 - **단순성:** 하나의 시스템만 유지보수하면 되므로 아키텍처가 훨씬 단순합니다.
 - **확장성 (Scalability):** 단일 시스템이므로 확장성이 더 좋습니다.
 - **일관성 (Consistency):** 모든 데이터가 동일한 스트리밍 레이어를 통해 처리되므로 데이터 일관성을 유지하기 쉽습니다. 특히 **Exactly-Once Processing** (정확히 한 번 처리)을 지원하여 동일한 데이터가 두 번 처리되는 문제를 방지합니다.
 - * **단점:** 스트리밍 기술의 성숙도와 강력한 스트리밍 엔진이 필요합니다.

Table 3.4: Lambda vs Kappa 아키텍처 비교

특징	Lambda 아키텍처	Kappa 아키텍처
레이어 수	배치 레이어 + 스트리밍 레이어 (2개)	스트리밍 레이어 (1개)
처리 방식	배치 (정확성) + 스트리밍 (속도)	모든 데이터를 스트리밍으로 처리 (배치도 스트리밍의 일종으로 간주)
복잡성	높음 (두 개의 시스템, 데이터 중복, 결과 일치 문제)	낮음 (단일 시스템, 단일 데이터 흐름)
데이터 일관성	배치/스트리밍 결과 일치 문제 발생 가능	Exactly-Once 처리로 높은 일관성 유지
유지보수	두 개의 코드베이스 관리 필요	하나의 코드베이스 관리
등장 시기	초기 스트리밍 기술이 미성숙했을 때 등장	스트리밍 기술이 성숙한 이후 등장
선호도	현재는 Kappa 아키텍처가 더 선호됨	현대적이고 단순하여 더 선호됨

주의

AWS Lambda와 혼동 주의: AWS Lambda는 서버리스 컴퓨팅 서비스의 이름이며, 여기서 다루는 Lambda 아키텍처와는 다른 개념입니다. Lambda 및 Kappa는 특정 제품이 아닌 데이터 파이프라인 설계 **아키텍처 패턴**을 지칭하는 산업 용어입니다.

3.3.15 스트리밍 처리 프레임워크

다양한 스트리밍 처리 프레임워크가 있으며, 각각 고유한 특징과 사용 사례를 가집니다.

Table 3.5: 주요 스트리밍 처리 프레임워크 비교

프레임워크	처리 모델	지연 시간	내결함성	사용 편의성	주요 특징/사용 사례
Kafka Streams	이벤트 기반	낮음	높음 (RocksDB 상태 관리)	중간 (Java 라이브러리)	Kafka 생태계 통합, 상태 저장 처리
Apache Flink	이벤트 기반 (진정한 스트리밍)	매우 낮음	Exactly-Once	어려움 (고급 시스템)	복잡 이벤트 처리 (CEP), 초저지연
Spark Structured Streaming	마이크로 배치 (연속 모드 지원)	중간 (수백 ms)	Exactly-Once (체크포인트)	매우 좋음 (통합 API)	배치/스트리밍 통합, Lakehouse 기반
Apache Storm	이벤트 기반	매우 낮음	At-Least-Once	어려움 (레거시)	초기 스트리밍 프레임워크, 현재는 레거시
Apache Samza	이벤트 기반	낮음	높음	중간	LinkedIn 개발, YARN/Kafka 통합
KSQL DB	이벤트 기반 (Kafka SQL)	낮음	높음 (Kafka 상속)	매우 좋음 (SQL)	Kafka 위에 SQL 레이어, 연속 SQL 쿼리
AWS Kinesis	이벤트 기반	낮음	높음	좋음	AWS 클라우드 네이티브, 고성능 스트리밍
Google Cloud Dataflow (Beam)	이벤트 기반	낮음	높음	좋음	Google 클라우드 네이티브, Apache Beam 기반
Azure Stream Analytics	이벤트 기반	낮음	높음	좋음	Azure 클라우드 네이티브

파일 기반 vs 이벤트 기반 스트리밍

– 파일 기반 스트리밍 (File-based Streaming):

- * 데이터가 디스크에 먼저 기록된 후, 다른 프로세스가 이를 읽어 처리하는 방식입니다.
- * 특징: 디스크 I/O가 발생하므로 느립니다.

– 이벤트 기반 스트리밍 (Event-based Streaming):

- * 데이터가 생성되는 즉시 호스트에서 직접 가져와 처리하는 방식입니다.
- * 특징: 디스크 I/O를 최소화하므로 빠릅니다.
- * 1세대 (메시지 브로커): ActiveMQ, RabbitMQ와 같은 메시지 지향 미들웨어 (Message Oriented Middleware)를 사용했습니다.
- * 2세대 (분산 스트리밍 플랫폼): Kafka, Kinesis와 같은 플랫폼이 사실상의 표준이 되었습니다. 이들은 고성능, 영속성, 대용량 트래픽 처리를 지원하며 스트리밍에 특화되어 있습니다.

Kafka의 주요 개념

- 프로듀서 (Producer): 데이터를 Kafka 클러스터로 보내는 주체입니다.
- 컨슈머 (Consumer): Kafka 클러스터에서 데이터를 읽어가는 주체입니다.
- 토픽 (Topic): 데이터 스트림을 분류하는 논리적인 채널입니다. 프로듀서는 특정 토픽에 데이터를 쓰고, 컨슈머는 특정 토픽을 구독하여 데이터를 읽습니다.
- 파티션 (Partition): 토픽은 여러 파티션으로 나뉘어 데이터를 병렬로 처리하고 확장성을 높입니다.

3.3.16 Databricks 엔드투엔드 스트리밍 아키텍처

Databricks는 클라우드 기반의 통합 데이터 및 AI 플랫폼으로, 다양한 데이터 소스부터 최종 애플리케이션까지의 엔드투엔드 데이터 흐름을 지원합니다.

Figure 3.2: Databricks 기반 엔드투엔드 스트리밍 아키텍처 (개념도)

위 그림은 Databricks 플랫폼을 중심으로 한 데이터 흐름을 보여줍니다. 다양한 데이터 소스에서 데이터가 수집되어 클라우드 스토리지 또는 메시지 버스에 랜딩되고, *Delta Lake*, *Unity Catalog*, *Photon*, *Structured Streaming*을 통해 처리된 후, *Databricks SQL*, *ML*, *BI/AI* 앱으로 소비되는 과정을 나타냅니다.

데이터 소스 (Data Sources)

- 데이터 웨어하우스, 온프레미스 시스템, SaaS 애플리케이션, 머신/애플리케이션 로그, 이벤트, IoT 데이터 등 다양한 소스에서 데이터가 발생합니다.

- Databricks는 클라우드 전용 플랫폼이지만, 온프레미스 시스템의 데이터를 클라우드로 마이그레이션하는 추세가 강합니다.

데이터 랜딩 (Data Landing)

- **파일 스토리지 (File Storage):** 클라우드 기반 스토리지 (예: AWS S3, Azure Data Lake Storage (ADLS))에 파일 형태로 저장됩니다.
- **메시지 버스 (Message Buses):** Kafka, Kinesis, Event Hubs와 같은 분산 메시지 시스템을 통해 실시간 이벤트 스트림이 수집됩니다.

데이터 처리 계층 (Data Processing Layer)

- **Delta Lake:** 클라우드 스토리지에 저장된 데이터에 ACID 트랜잭션, 스키마 진화, 시간 여행 등의 기능을 추가하여 데이터의 신뢰성과 분석 준비 상태를 보장합니다. 데이터는 여전히 오픈 포맷으로 저장됩니다.
- **Unity Catalog:** 메타데이터 관리, 데이터 거버넌스, 접근 제어, 데이터 계보(Lineage), 감사 로그(Audit Logs) 등을 제공하는 통합 카탈로그입니다. 행/컬럼 수준 보안도 지원합니다.
- **Photon:** Spark 엔진을 C++로 재작성한 고성능 쿼리 엔진입니다. JVM 기반 Spark의 비효율성을 개선하여 더 빠른 데이터 처리와 벡터화된 연산을 제공합니다.
- **Structured Streaming / DLT (Lakehouse Declarative Pipelines, LDP):** Apache Spark 기반의 스트리밍 처리 API로, 배치 및 스트리밍 처리를 통합합니다. DLT(이전 명칭)는 Structured Streaming 위에 구축된 선언적 파이프라인 프레임워크입니다. 이 모든 것은 오픈 소스 기반입니다.

데이터 소비 (Data Consumption)

- **Databricks SQL:** 쿼레이션된 데이터에 대해 실시간 분석, 대시보드, 보고서를 생성할 수 있는 SQL 인터페이스를 제공합니다. NoSQL 데이터에 대해서도 SQL 쿼리를 실행할 수 있습니다.
- **머신러닝 (Machine Learning):** Databricks 플랫폼은 ML 런타임(Runtime)을 제공하여 일반적인 ML 패키지들이 사전 설치 및 검증되어 있습니다. 데이터 과학자들이 종속성 문제 없이 쉽게 ML 모델을 개발하고 배포할 수 있도록 돕습니다.
- **엔터프라이즈 애플리케이션 (Enterprise Applications):** Tableau, PowerBI, Looker와 같은 BI 도구와 통합될 수 있으며, 실시간 AI 앱이나 운영 앱(예: 사기 패턴 감지 시 알림)과도 연동됩니다.

3.3.17 Databricks Jobs 기능

Databricks Jobs는 데이터 파이프라인의 워크플로우 오케스트레이션, 안정성 및 관찰 가능성을 위한 다양한 기능을 제공합니다.

- **파일 도착 트리거 (File Arrival Triggers):** 새로운 파일이 도착하면 파이프라인을 자동으로 시작합니다. 파일 도착 시간을 예측하기 어려울 때 컴퓨팅 자원 낭비를 줄일 수 있습니다.
- **조건부 작업 (Conditional Tasks):** 워크플로우 내에서 이전 작업의 결과에 따라 다른 경로를 선택할 수 있습니다. (예: 이전 작업이 성공하면 A 경로, 실패하면 B 경로)
- **서버리스 컴퓨팅 (Serverless Compute):** 컴퓨팅 자원을 직접 관리할 필요 없이, Databricks가 자동으로 자원을 프로비저닝하고 스케일링합니다. (무료 에디션에서 사용 가능)

- **작업 수준 매개변수 (Job Level Parameters):** 작업에 매개변수를 전달하여 동일한 코드를 다른 소스나 목적지에 대해 재사용할 수 있습니다. (예: 500 개의 파이프라인 대신 하나의 파이프라인에 500 개의 매개변수)
 - **웹훅 (Webhooks):** 파이프라인 상태 변경(성공, 실패 등) 시 Slack, 이메일, PagerDuty 등으로 알림을 보냅니다.
 - **작업 큐 (Job Queues):** 동시 실행 가능한 작업 수를 제한하거나, 새로운 트리거를 큐에 넣어 순차적으로 처리합니다.
 - **연속 실행 (Continuous Execution) 및 작업 루핑 (Task Looping):** 마이크로 배치 또는 연속 모드로 파이프라인을 지속적으로 실행하거나, 특정 작업을 여러 번 반복 실행할 수 있습니다.
 - **테이블 트리거 (Table Triggers):** 특정 테이블에 새로운 데이터가 채워지면 다른 파이프라인을 트리거합니다.
 - **시각적 모니터링 (Visual Monitoring):** 파이프라인의 실행 상태, 소요 시간, 실패 여부 등을 대시보드에서 시각적으로 확인할 수 있습니다.
 - **시스템 테이블의 작업 데이터 (Jobs Data in System Tables):** 작업 실행에 대한 메타데이터(실행 시간, ID, 트리거 사용자, 소요 시간 등)가 시스템 테이블에 저장되어 나중에 분석할 수 있습니다. (예: 어제 작업이 1시간 걸렸는데 평소에는 30분 걸렸다면 원인 분석)
 - **모듈형 워크플로우 (Modular Workflows):** 다른 작업을 호출하거나 다른 워크플로우를 포함하여 모듈화된 파이프라인을 구성할 수 있습니다.
 - **실행 시간 알림 (Duration Alerts):** 작업 실행 시간이 예상보다 길어지면 알림을 보내거나 작업을 중단하여 불필요한 컴퓨팅 비용을 방지합니다.
 - **간트 시각화 (Gantt Visualization):** 작업의 진행 상황과 소요 시간을 간트 차트 형태로 시각적으로 보여줍니다.
 - **SQL 작업 (SQL Tasks):** SQL 쿼리를 직접 작업으로 실행할 수 있습니다.
 - **Databricks Asset Bundles (DAB):** CI/CD(지속적 통합/지속적 배포)를 지원하여 코드 프로모션(개발 환경에서 운영 환경으로)을 자동화합니다. (추후 강의에서 다룰 예정)
- 이러한 기능들은 데이터 엔지니어의 작업을 훨씬 효율적이고 안정적으로 만들어줍니다.

3.3.18 스트리밍 파이프라인 설계 고려사항 및 트레이드오프

스트리밍 파이프라인을 설계할 때는 다양한 요구 사항과 제약 사항 사이에서 균형을 찾아야 합니다. Spark는 성능, 비용, 품질을 조절할 수 있는 유연성을 제공하지만, 모든 것을 동시에 얻을 수는 없습니다 (CAP 이론의 변형).

반복적인 설계 과정

1. **목표 및 요구 사항 이해:** 스트리밍 사용 사례의 목표와 요구 사항(예: 데이터의 중요성)을 명확히 이해합니다.
2. **전략 수립:** 각 목표를 달성하기 위한 전략을 세웁니다.
3. **자원 정의:** 각 목표에 필요한 컴퓨팅, 스토리지 등의 자원을 정의합니다.
4. **비용 계산:** 필요한 자원의 비용을 계산합니다.
5. **트레이드오프 재검토:** 기대치와 실제 비용/성능 사이의 균형을 맞추기 위해 트레이드오프를 재검토합니다.

주요 고려 사항

- 확장성 (Scalability):
 - * 데이터 볼륨 (Data Volume): 얼마나 많은 데이터가 유입될 것인가?
 - * TPS (Throughput Per Second): 초당 처리량은 얼마인가? 높은 처리량을 위해서는 그에 맞는 컴퓨팅 자원 설계가 필요합니다.
- 성능 (Performance):
 - * 종단 간 지연 시간 (End-to-End Latency): 데이터가 들어와서 처리되고 결과가 나올 때까지의 시간은 얼마인가?
- 처리 요구 사항 (Processing Needs):
 - * 데이터 정제 수준: 데이터가 얼마나 깨끗한가? 정제, 변환, 정규화, 큐레이션 등 얼마나 많은 작업이 필요한가?
 - * 작업 복잡성: 몇 번의 홉(Hops), 싱크(Sinks), 조인(Joins)이 필요한가? 상태 저장 처리(Stateful Processing)가 필요한가? 재처리 로직(Reprocessing Logic)이 필요한가?
- 품질 (Quality):
 - * 데이터 정확성: 데이터가 올바른가? 데이터 유실(No Drops)이나 중복(No Duplicates)이 없는가?
 - * 멱등성 (Idempotency): 동일한 파이프라인을 여러 번 실행해도 결과가 동일한가? (중복 파일 처리 방지)
- 신뢰성 및 가용성 (Reliability and Availability):
 - * 장애 복구 (Failure Recovery): 장애 발생 시 어떻게 복구할 것인가?
 - * 재해 복구 (Disaster Recovery, DR): 전체 리전(Region)이 다운될 경우 어떻게 대응할 것인가? (매우 복잡한 사용 사례)

자원 고려 사항

- 컴퓨팅 (Compute):
 - * 필요한 컴퓨팅 자원의 양과 유형 (VM 수, 유형, 풀). 서버리스 환경에서는 이 부분이 추상화되지만, 직접 클러스터를 관리할 때는 중요합니다.
- 스토리지 (Storage):
 - * 데이터를 어디에 저장할 것인가? (웜 스토리지, 콜드 스토리지). 추가 서비스 통합, API 호출 및 요금 제한.
- 인력 (People):
 - * 파이프라인 개발, 관리, 유지보수에 필요한 인력의 기술 수준.

실제 시나리오: 네트워크 위협 탐지 시스템 비용 문제

- 문제: 한 대규모 회사에서 네트워크 위협 탐지 시스템을 Databricks로 구축했습니다. 이벤트 허브에서 데이터를 수집하고 모델을 적용하여 위협 패턴을 예측했습니다. 하지만 스토리지 비용이 70%, VM(컴퓨팅) 비용이 30%로 비정상적으로 높게 나왔습니다. 일반적으로 컴퓨팅 비용이 더 높아야 합니다.
- 원인 분석:
 - * 액세스 계층 (Access Tier): 활성 데이터(Active Data)를 웜 스토리지(Warm Storage)에 보관해야 하는데, 자주 접근하는 데이터를 콜드 스토리지(Cool Storage)나 아카이브 스토리지(Glacier)에 보관하여 접근 비용이 매우 높아졌습니다. 콜드 스토리지에서 데이터를 자주 읽으면 비용이

급증합니다.

- * **복제 방식 (Replication):** 재해 복구를 위해 지리적 중복 스토리지 (Geo-redundant Storage)를 사용했는데, 이는 로컬 중복 스토리지 (Local Redundant Storage)보다 훨씬 비쌉니다. 비즈니스 중요도에 따라 신중하게 선택해야 합니다.
- * **높은 송신 비용 (High Egress Cost):** 데이터를 클라우드 외부로 전송할 때 발생하는 비용이 높았습니다.
- * **작은 파일 문제 (Small Files Problem):** 낮은 스트리밍 볼륨으로 인해 매우 작은 파일들이 많이 생성되었고, 이는 컴퓨팅 경로에서 높은 오버헤드를 유발했습니다. (컴팩션 (Compaction) 필요)
- **교훈:** 스토리지 계층, 복제 전략, 파일 크기 관리 등 세부적인 비용 요소를 면밀히 검토해야 합니다.

데이터 재처리 (Reprocessing) 및 따라잡기 (Catch-up)

시스템 장애 등으로 인해 데이터 처리가 지연되었을 때, 누락된 데이터를 처리하는 방법입니다.

- **옵션 1: 대규모 클러스터 확장 (Scale Up a Large Cluster):**
 - * 기존 클러스터를 확장하여 처리량을 늘리고, 스로틀 (Throttle)을 조정하여 백로그 (Backlog)를 처리합니다.
 - * **단점:** 백로그 처리 시간이 허용될 때만 가능하며, 체크포인트에 영향을 줄 수 있습니다.
- **옵션 2: 두 번째 클러스터 사용 (Spin Up a Second Cluster):**
 - * 새로운 클러스터를 시작하고, 오프셋 (Offset)을 조정하여 누락된 데이터부터 처리하도록 합니다.
 - * 기존 클러스터는 계속해서 이전 데이터를 처리하고, 두 클러스터가 데이터를 따라잡으면 이전 클러스터를 종료합니다.
 - * **단점:** 오프셋을 수동으로 변경하는 것은 위험할 수 있습니다.

로직 변경 (Changing Logic)

비즈니스 로직이 변경될 때 파이프라인을 업데이트하는 방법입니다.

- **방법:** 시간 제약이 없다면, 별도의 작업을 시작하여 변경된 로직으로 데이터를 재계산하고 따라잡도록 합니다. 완료되면 기존 작업을 중단하고 싱크 (Sink)를 새로운 작업으로 전환합니다.

멀티테넌시 (Multi-tenancy)

여러 고객 또는 테넌트 (Tenant)의 데이터를 하나의 시스템에서 처리하는 방법입니다.

- **문제:** 온프레미스에서는 각 고객마다 별도의 컴퓨팅 자원을 할당했지만, 클라우드에서는 매우 비효율적입니다.
- **해결책:**
 - * **파티셔닝 (Partitioning):** 고객별로 데이터를 파티셔닝하여 격리합니다.
 - * **스키마/카탈로그 격리 (Schema/Catalog Isolation):** 고객별로 스키마나 카탈로그를 분리하여 논리적 격리를 제공합니다.
 - * **권한 관리 (Grants):** Unity Catalog와 같은 도구를 사용하여 고객별로 데이터 접근 권한을 세분화합니다.

중복 제거 (Deduplication) 및 워터마킹 (Watermarking)

- 워터마킹: 지연 도착 데이터(Late Arriving Data)를 처리하기 위한 유예 기간을 설정합니다. (예: 버스가 10분 더 기다려주는 것과 같음)
- 중복 제거: 워터마크 내에서 중복 데이터를 제거합니다.

3.3.19 스트리밍 파이프라인 모범 사례

- **SLA 이해 (Understand SLAs):** 서비스 수준 협약(Service Level Agreements)을 명확히 이해하여 파이프라인이 충족해야 할 성능 및 가용성 기준을 파악합니다.
- **비용 추정 (Ballpark Cost Estimate):** 프로젝트가 중단될 정도로 비용이 높아지지 않도록 초기 단계에서 대략적인 비용을 추정합니다.
- **항상 체크포인트 사용 (Always Use Checkpoint):** 장애 발생 시 데이터 손실 없이 복구할 수 있도록 체크포인트를 활용합니다.
- **처리 트리거 간격 도입 (Introduce Process Trigger Interval):** 비즈니스 요구 사항에 따라 적절한 트리거 간격을 설정하여 컴퓨팅 자원을 최적화합니다.
- **쓰기 최적화 (Optimize Your Writes):** 데이터를 효율적으로 저장하고, 작은 파일 문제를 방지하기 위해 컴팩션(Compaction) 등을 고려합니다.
- **용량 계획 (Plan for Capacity):** 데이터 유입량에 따라 파이프라인이 확장될 수 있도록 미리 용량을 계획합니다. (예: 모든 데이터를 한곳에 모았다가(MUX) 나중에 고객 유형별로 분리(DEMUX)하는 전략)

3.4 실습: 네트워크 로그 분석 (Lab 02)

이번 실습에서는 Databricks 환경에서 실제 네트워크 로그 데이터를 분석하는 과정을 다룹니다. 목표는 비정형 로그 데이터를 구조화하고, 참조 데이터를 결합하여 의미 있는 인사이트를 도출하는 것입니다.

3.4.1 실습 목표 및 환경 설정

실습 목표

- Databricks에서 공개 데이터셋을 로드하고 탐색합니다.
- 정규 표현식(Regular Expression)을 사용하여 비정형 텍스트 로그를 구조화된 데이터로 파싱합니다.
- PySpark 및 SQL 컨텍스트에서 사용자 정의 함수(UDF)를 생성하고 활용합니다.
- 외부 참조 데이터를 가져와 로그 데이터와 조인(Join)하여 데이터를 풍부하게 만듭니다.
- 변환된 데이터를 Delta 테이블에 저장하고, Spark의 실행 계획(Explain Plan)을 분석합니다.

환경 설정

1. **Lab 02 노트북 다운로드:** Canvas에서 lab02 network analysis 파일을 다운로드합니다.
2. **Databricks Workspace로 импорт:** Databricks Workspace의 홈 폴더 또는 원하는 위치로 이동하여, 우클릭 또는 상단 메뉴의 'Import'를 통해 다운로드한 zip 파일을 업로드합니다.

3. **컴퓨팅 연결:** 노트북을 열고 서버리스 컴퓨팅(Serverless Compute)에 연결합니다.
4. **초기 설정 셀 실행:** 첫 번째 셀을 실행하여 카탈로그, 스키마(schema), 볼륨을 생성합니다. 이 셀은 멍등성(Idempotent)을 가지므로 여러 번 실행해도 문제가 없습니다.
5. **카탈로그 확인:** 카탈로그 탐색기에서 CSI 103 카탈로그 아래에 Lab02 스키마가 생성되었는지 확인합니다. 초기에는 테이블이 없지만, 볼륨은 생성되어 있을 것입니다.

```

1 # 예시: 초기 설정 셀 내용
2 # 이 코드는 Databricks 환경에서 카탈로그, 스키마, 볼륨을 설정합니다.
3 # 실제 노트북에서는 더 많은 변수 설정이 포함될 수 있습니다.
4
5 catalog_name = "CSI103" # 사용자 카탈로그 이름
6 schema_name = "lab02"   # 생성할 스키마 이름
7
8 spark.sql(f"CREATE CATALOG IF NOT EXISTS {catalog_name}")
9 spark.sql(f"USE CATALOG {catalog_name}")
10 spark.sql(f"CREATE SCHEMA IF NOT EXISTS {schema_name}")
11 spark.sql(f"USE SCHEMA {schema_name}")
12
13 # 볼륨 생성 예시()
14 spark.sql(f"CREATE VOLUME IF NOT EXISTS {schema_name}.input_data")
15 spark.sql(f"CREATE VOLUME IF NOT EXISTS {schema_name}.output_data")
16
17 print(f현재" 카탈로그: {spark.catalog.currentCatalog()}")
18 print(f현재" 스키마: {spark.catalog.currentDatabase()}")

```

3.4.2 데이터 로드 및 탐색

공개 데이터셋 탐색

Databricks는 다양한 샘플 데이터셋을 공개적으로 마운트(Mount)하여 제공합니다.

- `dbutils.fs.ls("/databricks-datasets/")` 명령을 사용하여 사용 가능한 데이터셋 목록을 확인합니다.
- 이번 실습에서는 `/databricks-datasets/log_files/` 경로의 Apache 로그 파일을 사용합니다.

```

1 # 공개 데이터셋 목록 확인
2 dbutils.fs.ls("/databricks-datasets/")
3
4 # 사용할 로그 파일 경로 정의 및 탐색
5 log_files_path = "/databricks-datasets/log_files/"
6 dbutils.fs.ls(log_files_path)

```

로그 파일 내용 확인

- `dbutils.fs.head(log_files_path + "part-00000")` 명령으로 로그 파일의 첫 몇 줄을 확인합니다.
- 로그는 사용자 ID, IP 주소, 타임스탬프, 엔드포인트 요청, HTTP 응답 코드 등으로 구성된 공백으로 구분된 텍스트 파일임을 알 수 있습니다.

- 예시 로그 형식: 10.0.0.1 - user1 [10/Oct/2023:10:00:00 +0000] "GET /api/data HTTP/1.1" 200 1234

```
1 dbutils.fs.head(log_files_path + "part-00000")
```

원시 로그 데이터프레임 생성

- `spark.read.text(log_files_path)`를 사용하여 로그 파일을 텍스트 데이터프레임으로 읽어옵니다. 이 데이터프레임은 각 로그 라인을 하나의 문자열 컬럼으로 가집니다.
- `display(raw_log_files_df)` 명령으로 데이터프레임 내용을 확인합니다. `display`는 Spark 작업을 트리거하는 액션(Action)입니다.

```
1 raw_log_files_df = spark.read.text(log_files_path)
2 display(raw_log_files_df)
```

3.4.3 정규 표현식을 이용한 로그 파싱

개념

로그 데이터는 종종 비정형 텍스트 형태로 제공됩니다. 이를 분석 가능한 구조화된 데이터로 변환하기 위해 정규 표현식(Regular Expression)을 사용합니다.

로그 파싱 함수 정의

- `pyspark.sql.functions.regexp_extract` 함수를 사용하여 로그 문자열에서 IP 주소, 사용자 ID, 타임스탬프, 요청 메서드, 엔드포인트, 응답 코드, 콘텐츠 크기 등을 추출합니다.
- 정규 표현식은 복잡할 수 있으며, 시행착오를 통해 패턴을 완성하거나 ChatGPT와 같은 도구의 도움을 받을 수 있습니다. 중요한 것은 생성된 정규 표현식의 각 부분이 무엇을 의미하는지 이해하는 것입니다.

```
1 from pyspark.sql.functions import regexp_extract, col
2
3 # Apache 로그 파일 파싱을 위한 정규 표현식
4 # 이 정규 표현식은 IP, 사용자, 타임스탬프, 메서드, 엔드포인트, 프로토콜, 응답 코드,
  # 콘텐츠를 추출합니다.
5 log_pattern = r'^(\S+) (\S+) (\S+) \[([^\w:/]+\s[+-]\d{4})\] "(\S+) (\S+)
  (?:\s(\S+))?" (\d{3}) (\S+)$'
6
7 parsed_df = raw_log_files_df.select(
8     regexp_extract(col("value"), log_pattern, 1).alias("ip_address"),
9     regexp_extract(col("value"), log_pattern, 3).alias("user_id"),
10    regexp_extract(col("value"), log_pattern, 4).alias("timestamp_str"),
11    regexp_extract(col("value"), log_pattern, 5).alias("method"),
12    regexp_extract(col("value"), log_pattern, 6).alias("endpoint"),
13    regexp_extract(col("value"), log_pattern, 8).alias("response_code"),
14    regexp_extract(col("value"), log_pattern, 9).alias("content_size")
15 )
16
17 display(parsed_df)
```

파싱된 데이터프레임 확인

- `parsed_df.count()`를 통해 파싱된 레코드 수를 확인합니다. 원본과 동일한 100,000개 레코드가 파싱되었는지 확인합니다.
- `parsed_df.createOrReplaceTempView("log_data")` 명령으로 파싱된 데이터프레임을 SQL 뷰로 등록합니다. 이는 현재 Spark 세션 내에서만 유효합니다.

3.4.4 사용자 정의 함수(UDF) 생성 및 활용

개념

사용자 정의 함수(UDF)는 Spark에서 제공하는 기본 함수로는 처리하기 어려운 복잡하거나 특정 로직을 직접 정의하여 데이터프레임이나 SQL 쿼리에서 재사용할 수 있도록 하는 기능입니다.

타임스탬프 파싱 문제

- 로그에서 추출한 `timestamp_str`은 `[10/Oct/2023:10:00:00 +0000]`와 같은 특정 형식의 문자열입니다.
- `CAST(timestamp_str AS TIMESTAMP)`와 같이 일반적인 형변환으로는 올바르게 파싱되지 않고 `NULL`이 반환됩니다. 이는 Spark가 해당 문자열 형식을 자동으로 인식하지 못하기 때문입니다.

```
1 SELECT CAST(timestamp_str AS TIMESTAMP) FROM log_data LIMIT 5;
```

이 쿼리는 `NULL` 값을 반환하거나 오류를 발생시킬 것입니다.

PySpark UDF 생성 및 사용

- **정의:** Python 함수를 정의하고, `pyspark.sql.functions.udf` 데코레이터를 사용하여 Spark UDF로 등록합니다. 이때 반환 타입(Return Type)을 명시해야 합니다.
- **활용:** 데이터프레임 API의 `select` 함수 내에서 UDF를 호출하여 `timestamp_str`을 Unix 타임스탬프(Epoch Time)로 변환합니다.

```
1 from pyspark.sql.functions import udf
2 from pyspark.sql.types import FloatType
3 from datetime import datetime
4
5 # 타임스탬프 문자열을 파싱하는 Python 함수 정의
6 def parse_timestamp_udf(timestamp_str):
7     try:
8         # 예시: "10/Oct/2023:10:00:00 +0000" -> datetime 객체
9         dt_object = datetime.strptime(timestamp_str, "%d/%b/%Y:%H:%M:%S %z")
10
11         return float(dt_object.timestamp()) # Unix 타임스탬프로 반환
12     except ValueError:
13         return None
14
15 # Python 함수를 Spark 로UDF 등록 반환( 타입 명시 필수)
16 # FloatType()은 반환값이 부동 소수점 숫자임을 나타냅니다.
17 parse_state_udf = udf(parse_timestamp_udf, FloatType())
```

```

17
18 # 데이터프레임 에서API UDF 사용
19 parsed_df.select(parse_state_udf(col("timestamp_str")).alias("
    unix_timestamp")).display()

```

SQL UDF 생성 및 사용

- 정의: spark.udf.register를 사용하여 Python 함수를 SQL 컨텍스트에서 사용할 수 있는 UDF 로 등록합니다.
- 활용: spark.sql을 통해 SQL 쿼리를 실행할 때, 등록된 UDF를 일반 SQL 함수처럼 호출할 수 있습니다.

```

1 # Python 함수를 SQL 로UDF 등록
2 # 첫 번째 인자는 에서SQL 사용할 UDF 이름, 두 번째는 Python 함수, 세 번째는 반환 타
  입
3 spark.udf.register("parse_state", parse_timestamp_udf, FloatType())
4
5 # Spark SQL 쿼리에서 UDF 사용
6 clean_logs_df = spark.sql(f"""
7     SELECT
8         ip_address,
9         user_id,
10        FROM_UNIXTIME(parse_state(timestamp_str), 'yyyy-MM-dd HH:mm:ss') AS
    event_time,
11        method,
12        endpoint,
13        CAST(response_code AS INT) AS response_code,
14        CAST(content_size AS INT) AS content_size
15    FROM log_data
16 """)
17 display(clean_logs_df)

```

위 코드에서는 `FROM_UNIXTIME` 함수를 사용하여 *Unix* 타임스탬프를 사람이 읽기 쉬운 `yyyy-MM-dd HH:mm:ss` 형식으로 변환했습니다.

3.4.5 데이터 정제 및 변환

데이터프레임 컬럼 변환 및 이름 변경

- `content_size`와 `response_code` 컬럼을 정수형(INT)으로 변환합니다.
- `timestamp_str`을 UDF를 통해 `event_time`으로 변환하고, 필요한 경우 `FROM_UNIXTIME`과 같은 Spark SQL 함수를 사용하여 날짜/시간 형식을 지정합니다.
- `selectExpr` 또는 `select` 함수를 사용하여 컬럼 이름을 변경 (Aliasing)하고 필요한 컬럼만 선택합니다.

```

1 from pyspark.sql.functions import expr
2
3 # 을selectExpr 사용하여 SQL 표현식으로 컬럼 변환 및 이름 변경
4 clean_logs_df_v2 = parsed_df.selectExpr(
5     "ip_address",

```



```

6     "user_id",
7     "FROM_UNIXTIME(parse_state(timestamp_str), 'yyyy-MM-dd HH:mm:ss') AS
event_time",
8     "method",
9     "endpoint",
10    "CAST(response_code AS INT) AS response_code",
11    "CAST(content_size AS INT) AS content_size"
12 )
13 display(clean_logs_df_v2)

```

3.4.6 참조 데이터 조인

개념

주요 데이터셋만으로는 모든 정보의 의미를 파악하기 어려울 때가 많습니다. 이때 **참조 데이터 (Reference Data)**를 조인하여 데이터를 풍부하게 만들고, 더 깊은 분석을 가능하게 합니다.

HTTP 상태 코드 참조 데이터 로드

- GitHub에서 HTTP 상태 코드 목록을 JSON 파일로 가져와 Databricks 볼륨에 저장합니다.
- `spark.read.json(path, multiLine=True)`를 사용하여 JSON 파일을 읽어옵니다. `multiLine=True`는 JSON 파일이 여러 줄에 걸쳐 있을 때 필요합니다.

```

1 # 에서GitHub HTTP 상태 코드 JSON 파일 다운로드 및 볼륨에 저장 예시()
2 # 실제 노트북에서는 dbutils.fs.cp 또는 wget 명령을 사용할 수 있습니다.
3 # dbutils.fs.cp("dbfs:/Volumes/csi103/lab02/input_data/status_codes.json",
   "dbfs:/Volumes/csi103/lab02/output_data/status_codes.json")
4
5 status_codes_df = spark.read.json(
6     "dbfs:/databricks-datasets/definitive-guide/data/activity-status-codes.
   json",
7     multiLine=True # JSON 파일이 여러 줄에 걸쳐 있을 경우 필요
8 )
9 display(status_codes_df)

```

로그 데이터와 상태 코드 조인

- `clean_logs_df` (로그 데이터)와 `status_codes_df` (상태 코드 참조 데이터)를 조인합니다.
- 조인 조건은 `response_code` 컬럼입니다. 이때, 두 데이터프레임의 조인 키 컬럼의 데이터 타입이 일치해야 합니다. `clean_logs_df`의 `response_code`는 INT형이고, `status_codes_df`의 `code`는 STRING형이므로, 조인 전에 INT형 `response_code`를 STRING형으로 형변환해야 합니다.

```

1 from pyspark.sql.functions import col
2
3 # 조인 키의 데이터 타입을 일치시키기 위해 response_code 으로STRING 형변환
4 network_df = clean_logs_df.join(
5     status_codes_df,
6     clean_logs_df.response_code.cast("string") == status_codes_df.code,
7     "left" # 왼쪽 조인 로그( 데이터의 모든 레코드를 유지)

```

```

8 )
9 display(network_df)

```

조인된 데이터 필터링

- 조인된 `network_df`에서 특정 조건을 만족하는 레코드만 필터링합니다.
- **예시:** 응답 코드가 200(성공)이고, `content_size`가 100보다 큰 요청만 필터링합니다. 이는 애플리케이션에서 성공했지만 처리량이 많은 요청을 분석하는 데 유용할 수 있습니다.

```

1 filtered_network_df = network_df.filter(
2     (col("response_code") == 200) & (col("content_size") > 100)
3 )
4 display(filtered_network_df)

```

3.4.7 Delta 테이블로 데이터 저장

개념

모든 변환 및 정제 작업이 완료되면, 처리된 데이터를 **Delta 테이블**에 저장하여 후속 분석이나 머신러닝 모델 학습에 활용할 수 있도록 합니다.

Delta 테이블에 쓰기

- `filtered_network_df.write.format("delta").mode("overwrite").saveAsTable("network_filter")` 명령을 사용하여 필터링된 데이터를 `network_filter`라는 Delta 테이블에 저장합니다.
- `mode("overwrite")`는 테이블이 이미 존재하면 덮어쓰도록 지시합니다.
- `saveAsTable()`은 Databricks 카탈로그에 테이블을 등록합니다.

```

1 filtered_network_df.write.format("delta").mode("overwrite").saveAsTable("
    network_filter")

```

저장된 테이블 확인

- `SELECT * FROM network_filter` 쿼리를 실행하여 저장된 데이터를 확인합니다.
- `SELECT COUNT(*) FROM network_filter`를 통해 저장된 레코드 수를 확인합니다. 이는 필터링된 데이터프레임의 레코드 수와 일치해야 합니다.
- Databricks 카탈로그 탐색기에서 lab02 스키마 아래에 `network_filter` 테이블이 생성되었는지 확인할 수 있습니다.

3.4.8 Spark 실행 계획 분석

개념

`df.explain()` 명령은 Spark가 데이터프레임 작업을 어떻게 실행할지 계획하는 물리적 실행 계획(Physical Plan)을 보여줍니다. 이를 통해 쿼리 성능 병목 현상을 이해하고 최적화할 수 있습니다.

- `filtered_network_df.explain()` 명령을 실행하여 Spark의 실행 계획을 확인합니다.
- 실행 계획 해석:
 - * 계획은 일반적으로 아래에서 위로 읽습니다.
 - * **Scan**: 데이터를 읽어오는 단계입니다.
 - * **RowToColumnar/ColumnarToRow**: Spark는 내부적으로 컬럼형 형식으로 작업하지만, 사용자 입력/출력 또는 특정 함수 호출 시 행 형식으로 변환할 수 있습니다. 이러한 직렬화/역직렬화 과정은 비용이 많이 들 수 있습니다.
 - * **Filter**: 데이터를 필터링하는 단계입니다.
 - * **Project**: 컬럼을 선택하거나 변환하는 단계입니다.
 - * **Write**: 최종 결과를 쓰는 단계입니다.
- 최적화 팁: 실행 계획에서 비용이 많이 드는 단계를 식별하고, 해당 단계를 최적화하여 전체 파이프라인 성능을 향상시킬 수 있습니다. (예: 불필요한 직렬화/역직렬화 방지, 필터링을 최대한 일찍 적용)

```
1 filtered_network_df.explain()
```

3.5 FAQ (자주 묻는 질문)

- Q: Lambda 아키텍처와 AWS Lambda 서비스는 같은 것인가요?
- A: 아닙니다. Lambda 아키텍처는 배치 및 스트리밍 처리를 병렬로 수행하는 데이터 파이프라인 설계 패턴을 의미합니다. 반면 AWS Lambda는 Amazon Web Services에서 제공하는 서버리스 컴퓨팅 서비스의 이름입니다. 이름은 같지만 개념은 완전히 다릅니다.
- Q: Kappa 아키텍처가 Lambda보다 항상 좋은가요?
- A: 일반적으로 Kappa 아키텍처가 더 단순하고 유지보수가 용이하며, Exactly-Once 처리와 같은 이점을 제공하여 현대적인 스트리밍 시스템에서 선호됩니다. 하지만 Lambda 아키텍처는 스트리밍 기술이 미성숙했던 시기에 배치 처리의 신뢰성을 확보하기 위한 현실적인 대안이었습니다. 현재는 스트리밍 기술의 발전으로 Kappa가 더 유리한 경우가 많습니다.
- Q: 배치 처리도 스트리밍 파이프라인으로 구현할 수 있다는 것이 무슨 의미인가요?
- A: Spark Structured Streaming과 같은 프레임워크는 배치와 스트리밍을 통합된 API로 처리합니다. 배치 작업을 스트리밍 파이프라인으로 설계하되, 트리거 간격(예: 24시간마다)을 길게 설정하면 됩니다. 이렇게 하면 스트리밍 프레임워크가 제공하는 체크포인트, Exactly-Once 처리, 오류 복구 등의 이점을 배치 작업에서도 활용할 수 있습니다.
- Q: UDF(사용자 정의 함수)는 언제 사용해야 하나요?
- A: Spark가 기본적으로 제공하는 함수로는 처리하기 어려운 복잡한 데이터 변환 로직이 필요할 때 UDF를 사용합니다. 예를 들어, 특정 형식의 타임스탬프 문자열을 파싱하거나, 사용자 지정 비즈니스 로직을 적용해야 할 때 유용합니다. 하지만 UDF는 Spark의 최적화 엔진을 완전히 활용하지 못할 수 있으므로, 가능한 경우 내장 함수를 사용하는 것이 성능에 더 좋습니다.
- Q: 데이터 파이프라인 설계 시 가장 중요한 고려 사항은 무엇인가요?
- A: 가장 중요한 것은 비즈니스 요구 사항에 집중하는 것입니다. 데이터 접근 속도, 정확성, 필요한 데이터 유형, 비즈니스가 해결하고자 하는 질문 등을 명확히 이해해야 합니다. IT 편의성만을 고려하여 불필요하게 복잡하거나 과도한 성능의 파이프라인을 구축하는 것을 피해야 합니다.

- **Q: Databricks에서 스토리지 비용이 컴퓨팅 비용보다 높게 나오는 경우는 왜 발생하나요?**
- **A:** 이는 비정상적인 상황이며, 주로 스토리지 계층(예: 자주 접근하는 데이터를 콜드 스토리지에 보관), 데이터 복제 전략(예: 불필요한 고비용 지리적 중복), 작은 파일 문제(많은 작은 파일이 메타 데이터 및 API 호출 비용 증가) 등으로 인해 발생할 수 있습니다. 스토리지 접근 비용과 API 호출 비용을 면밀히 분석하고 최적화해야 합니다.

3.6 빠르게 훑어보기 (1페이지 요약)

데이터 파이프라인 핵심 요약

데이터 파이프라인: 데이터의 흐름을 자동화하는 시스템. 수집 → 변환 → 적재. **ETL vs ELT:**

- **ETL:** 추출 → 변환 → 적재 (전통적, 스키마 명확 시)
- **ELT:** 추출 → 적재 → 변환 (클라우드, 스키마 유연 시)

배치 vs 스트리밍:

- **배치:** 데이터 모아 한 번에 처리 (보고서, 청구)
- **스트리밍:** 실시간으로 지속 처리 (사기 탐지, IoT)

Lambda vs Kappa 아키텍처:

- **Lambda:** 배치 + 스트리밍 레이어 분리. 복잡성, 일치 문제.
- **Kappa:** 단일 스트리밍 레이어로 모두 처리. 단순성, Exactly-Once. **현대적 선호.**

스트리밍 핵심 개념: 소스, 처리, 싱크, 체크포인트, 워터마크, 마이크로 배치, UDF. **Databricks 아키텍처:** Delta Lake (ACID, 시간 여행), Unity Catalog (거버넌스), Photon (고성능 Spark), Structured Streaming (통합 API). **설계 고려사항:** 비즈니스 요구 사항 최우선! 확장성, 성능, 품질, 신뢰성, 비용 트레이드오프. **Lab 02:** 비정형 로그 → 정규 표현식 파싱 → UDF로 타임스탬프 변환 → 참조 데이터 조인 → Delta 테이블 저장.

데이터 저장소 및 형식

정규화/비정규화:

- **정규화:** 중복 감소, 무결성 (관계형 DB)
- **비정규화:** 쿼리 성능 향상 (NoSQL, DW)

저장소 유형:

- **키-값:** 단순, 확장성 (S3)
- **문서:** 유연한 스키마 (MongoDB)
- **컬럼형:** 분석 쿼리 최적화 (Cassandra)

데이터 형식:

- **바이너리:** 압축, 빠름 (Parquet)
- **Delta Lake:** 데이터 레이크에 ACID, 스키마 진화, 시간 여행.

주의 사항

비용 최적화: 스토리지 계층, 복제, 작은 파일 문제 등 비용 요소를 면밀히 검토. **UDF 사용:** 성능 저하 가능성 있으므로, 내장 함수 우선 고려. **Lambda vs AWS Lambda:** 다른 개념!

실습 요약 (Lab 02)**네트워크 로그 분석:**

1. `spark.read.text()` 로 원시 로그 로드.
2. `regexp_extract` 로 IP, 타임스탬프, 엔드포인트 등 파싱.
3. `udf` 로 타임스탬프 문자열을 `float` Unix 타임스탬프로 변환.
4. HTTP 상태 코드 참조 데이터와 `join` (타입 캐스팅 유의).
5. `filter` 로 특정 조건의 데이터만 선택.
6. `df.write.format("delta").saveAsTable()` 로 Delta 테이블에 저장.
7. `df.explain()` 으로 Spark 실행 계획 확인 및 최적화.

3.7 체크리스트

이 문서를 학습하거나 과제를 제출하기 전에 다음 항목들을 확인하세요.

3.7.1 학습 점검표

- 데이터 파이프라인 기본 개념: 데이터 파이프라인이 무엇인지, 왜 필요한지 설명할 수 있는가?
- ETL/ELT: 두 방식의 차이점과 각각의 장단점, 적합한 사용 사례를 이해하는가?
- 배치/스트리밍: 배치와 스트리밍 처리의 차이점과 각각의 사용 사례를 구분할 수 있는가?
- Lambda/Kappa 아키텍처: 두 아키텍처의 구조, 장단점, 그리고 Kappa가 더 선호되는 이유를 설명할 수 있는가?
- 스트리밍 핵심 개념: 체크포인트, 워터마크, 마이크로 배치 등의 개념을 이해하고 설명할 수 있는가?
- Databricks 에코시스템: Delta Lake, Unity Catalog, Photon, Structured Streaming의 역할을 이해하는가?
- UDF: 사용자 정의 함수(UDF)를 언제, 왜 사용해야 하는지 이해하는가?
- 트레이드오프: 파이프라인 설계 시 성능, 비용, 품질 사이의 트레이드오프를 고려할 수 있는가?

3.7.2 실습 (Lab 02) 점검표

- 환경 설정: Databricks Workspace에 Lab 02 노트북을 성공적으로 импорт하고 컴퓨팅에 연결했는가?
- 데이터 로드: 공개 데이터셋에서 네트워크 로그 파일을 성공적으로 로드했는가?
- 로그 파싱: 정규 표현식을 사용하여 비정형 로그 데이터를 구조화된 데이터프레임으로 파싱했는가?
- UDF 활용: 타임스탬프 문자열을 올바르게 파싱하기 위해 UDF를 생성하고 활용했는가?
- 데이터 조인: HTTP 상태 코드 참조 데이터를 가져와 로그 데이터와 성공적으로 조인했는가? (데이터 타입 일치 확인)
- Delta 테이블 저장: 처리된 데이터를 Delta 테이블에 성공적으로 저장했는가?
- 실행 계획 분석: `df.explain()` 을 사용하여 Spark의 실행 계획을 확인하고 기본적인 내용을 이해했는가?

3.7.3 문서 작성 및 제출 점검표

- **완전성:** 제공된 강의 자료의 모든 핵심 내용이 문서에 포함되었는가?
 - **초심자 친화성:** 비전공자나 처음 배우는 사람도 이해할 수 있도록 충분한 설명, 예시, 비유가 포함되었는가?
 - **LaTeX 형식:** 모든 LaTeX 가드레일 규칙(특수문자 이스케이프, 캡션, 라벨 등)을 준수했는가?
 - **가독성:** 짧은 문장, 짧은 단락, 불릿/번호 목록, 박스, 공백 등을 활용하여 가독성이 높은가?
 - **레이아웃:** A4 규격에 맞게 표와 그림이 배치되었으며, 섹션 시작 전
-

를 사용했는가?

- 출처/파일명/페이지 언급 금지: 문서 내에 원본 자료의 출처, 파일명, 페이지 번호 언급이 없는가?

Chapter 4

Lecture 4

개요

본 문서는 CSCI103 Lecture 4 강의 자료를 바탕으로 데이터 변환(Data Transformations)에 대한 핵심 개념을 다룹니다. 빅데이터 설계 패턴, 데이터 규정 준수, Spark 내부 동작 원리, Delta Lake 기능, 사용자 정의 함수(UDF), CRUD 작업, 스키마 진화, 변경 데이터 캡처(CDC) 및 서서히 변하는 차원(SCD), 그리고 Autoloader를 활용한 데이터 로딩 패턴까지 폭넓게 설명합니다. 이 문서는 비전공자나 처음 배우는 사람도 완벽히 이해할 수 있도록 각 개념의 필요성부터 실제 적용 방법까지 상세한 설명과 예시를 제공하여 시험 대비 및 실무 활용에 도움을 주는 것을 목표로 합니다.

4.1 용어 정리

다음 표는 본 강의에서 다루는 주요 용어들을 쉽게 설명하고 원어를 병기하여 이해를 돕습니다.

용어	쉬운 설명	원어	비고
스트리밍 (파일 기반)	파일이 도착하는 대로 처리하는 방식	File-based Streaming	
스트리밍 (이벤트 기반)	이벤트가 발생할 때마다 처리하는 방식	Event-based Streaming	
체크포인팅	장애 발생 시 마지막 저장 지점으로 복구하는 기능	Checkpointing	
워터마킹	너무 오래된 데이터를 폐기하는 기준 설정	Watermarking	데이터 지연 처리
람다 아키텍처	배치 처리와 실시간 스트리밍을 결합한 데이터 처리 방식	Lambda Architecture	
카파 아키텍처	모든 데이터를 단일 스트리밍 파이프라인으로 처리하는 방식	Kappa Architecture	
레이크하우스	데이터 웨어하우스와 데이터 레이크의 장점을 결합한 아키텍처	Lakehouse	
관리형 테이블	데이터와 메타데이터 모두 프레임워크가 관리하는 테이블	Managed Tables	
외부 테이블	데이터는 외부 저장소에, 메타데이터만 프레임워크가 관리하는 테이블	External Tables	
메타데이터	데이터에 대한 데이터 (설명, 위치, 소유자 등)	Metadata	데이터 거버넌스
구체화된 뷰	미리 계산되어 저장된 뷰 (빠르지만 오래될 수 있음)	Materialized View	
설계 패턴	재사용 가능한 문제 해결 템플릿	Design Patterns	
CQRS	쓰기(Command)와 읽기(Query) 책임을 분리하는 설계 패턴	Command Query Responsibility Segregation	
폴리글랏 영속성	데이터 유형에 따라 최적의 데이터베이스를 선택하는 전략	Polyglot Persistence	"적재적소의 도구"
데이터 레이크 패턴	원본 형태의 대량 데이터를 저장하는 패턴	Data Lake Pattern	
마이크로 배치 처리	스트리밍 데이터를 작은 배치 단위로 처리하는 방식	Microbatch Processing	준실시간 처리
CAP 이론	분산 시스템에서 일관성, 가용성, 분할 내성 중 두 가지만 선택 가능	CAP Theorem	
데이터 메시	분산된 데이터 도메인을 제품처럼 관리하고 공유하는 아키텍처	Data Mesh	
에이전트 아키텍처	LLM을 활용하여 도구와 에이전트를 조율하는 패턴	Agent Architecture	최신 AI 활용
커넥터 패턴	다양한 데이터 소스에 일관된 방식으로 접근하는 패턴	Connector Pattern	브릿지 패턴
메달리온 아키텍처	데이터를 원본, 정제, 집계 단계로 나누어 처리하는 파이프라인	Medallion Architecture	밀티홉 패턴
시간 여행	데이터의 특정 시점 또는 버전으로 돌아가는 기능	Time Travel	Delta Lake 기능
제로 카피 클론	실제 데이터 복사 없이 메타데이터만 복사하여 데이터 복제	Zero Copy Clone	효율적인 복제
GDPR	유럽 연합의 일반 데이터 보호 규정	General Data Protection Regulation	개인 정보 보호
CCPA	캘리포니아 소비자 개인 정보 보호법	California Consumer Privacy Act	개인 정보 보호
삭제할 권리	소비가자 자신의 데이터를 시스템에서 삭제하도록 요구할 권리	Right of Erasure	
데이터 이동성 권리	소비가자 자신의 데이터를 다른 서비스로 이전하도록 요구할 권리	Right of Portability	
가명화/익명화	개인 식별 정보를 제거하거나 대체하여 프라이버시 보호	Pseudonymization/Anonymization	
Spark 작업/단계/태스크	Spark가 쿼리를 실행하는 논리적/물리적 단위 계층	Jobs, Stages, Tasks	병렬 처리 단위
테이블 파티셔닝	대규모 테이블을 관리하기 쉬운 작은 조각으로 나누는 방식	Table Partitioning	물리적 저장
Spark 파티셔닝	Spark가 데이터를 노드에 분산하여 처리하는 방식	Spark Partitioning	논리적 처리
파티션 가지치기	쿼리 시 불필요한 파티션 스캔을 건너뛰는 최적화	Partition Pruning	쿼리 성능 향상
데이터 셔플링	Spark 노드 간에 데이터를 재분배하는 작업 (비용 발생)	Data Shuffling	조인, 그룹화 시 발생
Z-오더링	관련 데이터를 물리적으로 가깝게 저장하여 쿼리 속도 향상	Z-Ordering	Delta Lake 최적화
브로드캐스트 해시 조인	작은 테이블을 모든 워커 노드에 복사하여 조인 속도 향상	Broadcast Hash Join	
셔플 해시 조인	한쪽 테이블이 다른 테이블보다 작고 메모리에 적합할 때 사용	Shuffle Hash Join	
정렬 병합 조인	모든 크기의 테이블에 적용 가능한 기본 조인 방식 (느릴 수 있음)	Sort Merge Join	
사용자 정의 함수 (UDF)	사용자가 직접 정의하는 함수 (한 행 입력, 한 행 출력)	User-Defined Function	
벡터화된 UDF	Pandas UDF를 사용하여 Spark와 Python 간 직렬화 오버헤드 감소	Vectorized UDF	성능 향상
사용자 정의 집계 함수 (UAF)	여러 행을 입력받아 단일 집계 값을 반환하는 함수	User-Defined Aggregate Function	SUM, COUNT 등
사용자 정의 변환 함수 (UDTF)	여러 행을 입력받아 여러 행을 출력하는 함수	User-Defined Transformative Function	EXPLODE, PIVOT 등
CRUD	데이터에 대한 기본 작업 (생성, 읽기, 업데이트, 삭제)	Create, Read, Update, Delete	
Upsert	데이터가 존재하면 업데이트하고, 없으면 삽입하는 작업	Upsert	MERGE 명령
스키마 추론	데이터로부터 스키마를 자동으로 감지하는 기능	Schema Inference	
스키마 진화	데이터 스키마 변경(새 컬럼 추가 등)을 허용하는 기능	Schema Evolution	mergeSchema=true
스키마 강제	정의된 스키마와 다른 데이터가 들어오면 오류를 발생시키는 기능	Schema Enforcement	
변경 데이터 캡처 (CDC)	원본 시스템에서 변경된 데이터를 감지하고 기록하는 기술	Change Data Capture	소스 측면
서서히 변하는 차원 (SCD)	차원 테이블의 변경 이력을 관리하는 방법	Slowly Changing Dimensions	타겟 측면
Autoloader	클라우드 저장소의 새로운 데이터를 점진적으로 처리하는 기능	Autoloader	스트리밍 파일 처리

4.2 핵심 개념 및 원리

4.2.1 지난 강의 복습

이번 강의는 지난 강의에서 다룬 핵심 개념들을 빠르게 복습하며 시작합니다. 이는 새로운 내용을 이해하기 위한 중요한 기반이 됩니다.

– 두 가지 스트리밍 유형:

- * **파일 기반 (File-based):** 파일 시스템에 새로운 파일이 도착하는 것을 감지하여 처리합니다.
- * **이벤트 기반 (Event-based):** 실시간으로 발생하는 개별 이벤트를 즉시 처리합니다.

– 체크포인팅 (Checkpointing):

- * **한 줄 요약:** 장애 발생 시 데이터 처리의 연속성을 보장하기 위해 마지막으로 성공한 상태를 저장

하는 기능입니다.

- * **직관적 비유:** 게임을 하다가 중간에 저장하는 것과 같습니다. 게임이 멈추거나 꺼져도 마지막 저장 지점부터 다시 시작할 수 있습니다.
- * **기술적 설명:** Spark 스트리밍과 같은 시스템에서 처리 상태(예: 읽은 오프셋)를 주기적으로 저장하여, 시스템 오류 발생 시 마지막 저장 지점부터 복구하여 데이터 손실 없이 처리를 재개할 수 있도록 합니다.

– **워터마킹 (Watermarking):**

- * **한 줄 요약:** 데이터의 지연 도착을 허용하면서도, 너무 오래된 데이터를 처리 파이프라인에서 제외하기 위한 기준을 설정하는 메커니즘입니다.
- * **직관적 비유:** 우편물 처리 시, 너무 오래된 편지는 더 이상 유효하지 않다고 판단하여 버리는 것과 유사합니다. **기술적 설명:** 스트리밍 데이터 처리에서 이벤트 시간(event time)을 기준으로 특정 지연 시간(latency)까지 데이터를 기다려 처리하고, 그 시간을 초과한 데이터는 폐기하여 메모리 사용량을 최적화하고 정확한 집계를 보장합니다.

– **스트리밍 출력 모드 (Streaming Output Modes):**

- * **업데이트 (Update):** 변경된 결과만 출력합니다.
- * **덮어쓰기 (Overwrite):** 매번 전체 출력을 덮어씁니다.
- * **추가 (Append):** 새로운 데이터만 출력 싱크에 추가합니다.

– **처리 트리거 (Processing Trigger):**

- * **한 줄 요약:** 마이크로 배치(microbatch) 처리가 얼마나 자주 실행될지 결정하는 설정입니다.
- * **기술적 설명:** Spark 스트리밍에서 readStream 옵션에 지정되는 간격으로, 이 간격마다 새로운 데이터 배치를 처리하도록 트리거합니다.

– **람다 vs 카파 아키텍처 (Lambda vs Kappa Architecture):**

- * **람다 아키텍처 (Lambda Architecture):** 배치(느린) 처리와 스트리밍(빠른) 처리를 모두 사용하여 데이터를 분석하고, 이 두 결과를 통합하여 제공하는 방식입니다. 실시간 분석과 과거 데이터 분석이 모두 필요할 때 유용합니다.
- * **카파 아키텍처 (Kappa Architecture):** 모든 데이터를 단일 스트리밍 파이프라인으로 처리하는 방식입니다. 주로 실시간 처리 요구사항이 강하고, 배치 처리가 불필요하거나 스트리밍으로 대체 가능한 경우에 사용됩니다. IoT 데이터 처리나 이벤트 기반 아키텍처에 적합합니다.

– **데이터 웨어하우스 + 데이터 레이크 = 레이크하우스 (Data Warehouse + Data Lake = Lakehouse):**

- * **한 줄 요약:** 데이터 웨어하우스의 구조화된 관리 기능과 데이터 레이크의 유연한 대용량 저장 기능을 결합한 새로운 데이터 아키텍처입니다.
- * **기술적 설명:** Delta Lake와 같은 기술을 기반으로, 데이터 레이크에 저장된 원시 데이터에 데이터 웨어하우스의 ACID 트랜잭션, 스키마 적용, 거버넌스 기능을 추가하여 데이터의 신뢰성과 활용성을 높입니다.

– **관리형 vs 외부 테이블 (Managed vs External Tables):**

- * **관리형 테이블 (Managed Tables):** 데이터와 메타데이터(스키마, 위치 등) 모두 Spark나 Hive와 같은 프레임워크에 의해 관리됩니다. 테이블을 삭제하면 데이터도 함께 삭제됩니다.
- * **외부 테이블 (External Tables):** 데이터는 외부 저장소(예: S3, HDFS)에 저장되고, 프레임워크는 메타데이터만 관리합니다. 테이블을 삭제해도 실제 데이터는 삭제되지 않고 외부 저장소에 남아 있습니다.

- * **주요 차이점:** 데이터의 물리적 저장 위치와 프레임워크에 의한 데이터 관리 범위입니다. 외부 테이블은 데이터의 독립성을 유지할 수 있어 다른 시스템과의 공유에 유리합니다.
- **메타데이터 (Metadata):**
 - * **한 줄 요약:** 데이터에 대한 데이터, 즉 데이터의 내용, 구조, 출처, 소유자, 생성일 등을 설명하는 정보입니다.
 - * **기술적 설명:** 메타데이터는 데이터의 발견 가능성(discoverability)과 계보(lineage)를 향상시켜 데이터 거버넌스(governance)를 강화하고, 데이터를 더 효과적으로 제어하고 관리할 수 있게 합니다.
- **구체화된 뷰 (Materialized View):**
 - * **한 줄 요약:** 쿼리 결과를 미리 계산하여 물리적으로 저장해 둔 뷰입니다.
 - * **기술적 설명:** 일반 뷰는 쿼리 시점에 데이터를 계산하지만, 구체화된 뷰는 미리 계산된 결과를 저장하므로 접근 속도가 매우 빠릅니다. 단점은 데이터가 새로고침되지 않으면 오래된(stale) 정보일 수 있다는 점입니다.

4.2.2 빅데이터 설계 패턴 (Big Data Design Patterns)

설계 패턴은 소프트웨어 공학에서 반복적으로 발생하는 문제에 대한 재사용 가능한 해결책입니다. 빅데이터 영역에서도 데이터 엔지니어링의 복잡한 문제를 해결하기 위한 특화된 설계 패턴들이 등장했습니다.

핵심 요약

설계 패턴의 핵심:

- **재사용성 (Reusability):** 검증된 해결책을 반복적으로 적용하여 개발 시간과 노력을 절감합니다.
- **공통 문제 해결 (Common Problem Solving):** 데이터 엔지니어링에서 흔히 발생하는 확장성, 속도, 볼륨, 처리 요구사항 등의 문제를 효과적으로 다룹니다.
- **설계 원칙 촉진 (Promotes Design Principles):** 추상화, 분할 정복, 관심사 분리, 단순성 유지 등의 원칙을 솔루션에 통합합니다.
- **공통 어휘 제공 (Provides Vocabulary):** 팀원 간 문제와 해결책을 효율적으로 논의할 수 있는 공통 언어를 제공합니다.
- **신중한 적용 (Judicious Application):** 패턴을 맹목적으로 적용하기보다는, 당면한 문제에 가장 적합한 패턴을 신중하게 선택해야 합니다.

설계 패턴의 종류 및 적용 영역

빅데이터 설계 패턴은 다양한 데이터 엔지니어링 과제를 해결하기 위해 등장했습니다.

- **모델링 (Modeling):**
 - * **스타 및 스노우플레이크 차원 모델링 (Star and Snowflake Dimensional Modeling):** 팩트 테이블(fact table)이 차원 테이블(dimension table)로 둘러싸인 형태로, 데이터 웨어하우스에서 분석 쿼리 성능을 최적화합니다.
 - * **볼트 차원 모델링 (Vault Dimensional Modeling):** 모든 원시 데이터를 볼트(vault)에 캡처한 후, 이를 기반으로 추가 데이터 처리를 수행합니다.
- **수집 (Ingestion):**

- * **커넥터 (Connectors):** 다양한 데이터 소스에 연결하여 데이터를 수집하는 표준화된 방법을 제공합니다.
- * **배치 vs 스트리밍 파이프라인 (Batch vs Streaming Pipelines):** 람다(Lambda) 및 카파(Kappa) 아키텍처와 같이, 데이터 처리의 속도와 특성에 따라 파이프라인을 설계합니다.
- * **컴퓨팅과 스토리지 분리 (Separation of Compute from Storage):** 컴퓨팅 자원과 스토리지 자원을 독립적으로 확장할 수 있도록 설계하여 효율성과 비용 효과성을 높입니다. 대량의 데이터를 저장하더라도 실제 계산이 적으면 컴퓨팅 비용을 절감할 수 있습니다.
- **변환 (Transformations):**
 - * **스키마 변경 처리 (Handling Schema Changes):** 스키마 온 리드(Schema on Read)와 스키마 온 라이트(Schema on Write)와 같이, 데이터 스키마가 유동적인 경우와 고정적인 경우를 다루는 전략입니다.
 - * **ACID 트랜잭션 (ACID Transactions):** 대규모 데이터에 대한 업데이트, 삭제, 병합 작업에서 데이터의 일관성과 신뢰성을 보장합니다.
 - * **멀티홉 파이프라인 (Multi-hop Pipelines):** 데이터를 여러 단계로 나누어 처리하며, 각 단계에서 다양한 서비스 수준 협약(SLA)과 사용 사례를 지원합니다.
- **영속성 (Persistence):**
 - * **압축된 컬럼형 형식 (Compressed Columnar Formats):** Parquet과 같이 데이터를 컬럼 단위로 압축하여 저장하여 속도와 효율성을 높이고 대량의 데이터를 처리합니다.
 - * **비정규화된 와이드 테이블 (Denormalized Wide Tables):** 조인(join) 작업을 피하기 위해 데이터를 미리 비정규화하여 저장합니다.
 - * **다중 목적지 (Multiple Destinations):** 데이터 파이프라인이 여러 소비자에게 다양한 요구사항에 맞춰 데이터를 분산하여 제공합니다.
- **분석 (Analytics):**
 - * **인스트림 분석 (In-stream Analytics):** 데이터가 도착하는 즉시 실시간으로 분석을 수행합니다.

주요 빅데이터 설계 패턴 상세 설명

람다 아키텍처 (Lambda Architecture)

- **한 줄 요약:** 배치 처리와 실시간 스트리밍 처리를 결합하여, 과거 데이터와 실시간 데이터 모두에 대한 분석을 제공하는 패턴입니다.
- **구성:**
 - * **배치 레이어 (Batch Layer):** 모든 과거 데이터를 처리하여 정확하고 완전한 결과를 생성합니다.
 - * **스피드 레이어 (Speed Layer):** 실시간으로 들어오는 데이터를 처리하여 낮은 지연 시간으로 결과를 제공합니다.
 - * **서빙 레이어 (Serving Layer):** 배치 레이어와 스피드 레이어의 결과를 통합하여 사용자에게 제공합니다.
- **사용 사례:** 실시간 대시보드와 함께 과거 추세 분석이 필요한 경우 (예: 웹사이트 트래픽 분석).

카파 아키텍처 (Kappa Architecture)

- **한 줄 요약:** 모든 데이터를 단일 스트리밍 파이프라인으로 처리하여, 람다 아키텍처의 복잡성을 줄인 패턴입니다.
- **구성:** 모든 데이터 처리(과거 및 실시간)가 하나의 스트리밍 파이프라인에서 이루어집니다.

- **사용 사례:** IoT 데이터 처리, 이벤트 기반 시스템, 실시간 응답이 중요한 애플리케이션.

이벤트 기반 아키텍처 (Event-Driven Architecture)

- **한 줄 요약:** 시스템 구성 요소들이 이벤트의 발생과 소비를 통해 비동기적으로 상호작용하는 패턴입니다.
- **구성:** 이벤트 생산자(Event Producers), 이벤트 스트리밍(Event Streaming), 이벤트 소비자(Event Consumers)로 이루어집니다.
- **사용 사례:** 전자상거래 거래 처리, 실시간 알림 시스템.

CQRS (Command Query Responsibility Segregation)

- **한 줄 요약:** 데이터 쓰기(Command)와 읽기(Query)의 책임을 분리하여 시스템을 설계하는 패턴입니다.
- **설명:**
 - * **쓰기 최적화 (Write Optimization):** 데이터 입력 및 업데이트를 효율적으로 처리하도록 시스템을 설계합니다.
 - * **읽기 최적화 (Query Optimization):** 데이터 조회(쿼리)를 빠르고 효율적으로 처리하도록 시스템을 설계합니다.
- **장점:** 쓰기 및 읽기 부하에 따라 시스템을 독립적으로 확장할 수 있어, 높은 데이터 처리량과 빠른 쿼리 응답 시간을 동시에 달성할 수 있습니다.

폴리글랏 영속성 (Polyglot Persistence)

- **한 줄 요약:** 데이터의 특성과 사용 목적에 따라 가장 적합한 여러 종류의 데이터베이스를 혼합하여 사용하는 패턴입니다.
- **직관적 비유:** "망치만 있으면 모든 것이 못으로 보인다"는 말처럼, 하나의 데이터베이스만 고집하는 것이 아니라, 작업에 맞는 올바른 도구를 선택하는 것과 같습니다.
- **사용 사례:**
 - * **관계형 데이터베이스 (Relational Databases):** 트랜잭션 데이터.
 - * **문서 저장소 (Document Stores):** 비정형 또는 반정형 데이터.
 - * **그래프 데이터베이스 (Graph Databases):** 관계 중심 데이터.
- **장점:** 각 데이터베이스의 강점을 활용하여 시스템의 전반적인 성능과 유연성을 극대화합니다.

데이터 레이크 패턴 (Data Lake Pattern)

- **한 줄 요약:** 대량의 원시 데이터를 어떤 변환도 거치지 않은 원래 형태로 저장하는 패턴입니다.
- **설명:** 데이터를 수집할 때 정확히 어떻게 사용될지 모르는 경우, 모든 데이터를 원본 형태로 저장하여 나중에 다양한 분석 목적으로 재탐색할 수 있도록 합니다.
- **저장:** 정형 및 비정형 데이터를 모두 저장합니다.
- **처리 도구:** Hadoop, Spark 등을 사용하여 저장된 데이터를 처리합니다.
- **장점:** 미래의 알 수 없는 사용 사례에 대비하여 데이터의 잠재적 가치를 보존합니다.

마이크로 배치 처리 (Microbatch Processing)

- **한 줄 요약:** 스트리밍 데이터를 작은 배치 단위로 나누어 주기적으로 처리하는 패턴입니다.

- **설명:** 연속적인 데이터 스트림을 실시간에 가까운(near real-time) 통찰력을 얻기 위해 작은 시간 간격(예: 1초)으로 데이터를 수집하고 처리합니다.
- **특징:**
 - * **정확히 한 번 처리 (Exactly-once Semantics):** 데이터가 중복 처리되지 않도록 보장합니다.
 - * **장애 복원력 (Resilient to Failures):** 처리 중 오류가 발생해도 복구할 수 있습니다.
 - * **인스트림 분석 (In-stream Analytics):** 데이터가 도착하는 즉시 분석을 수행합니다.
 - * **지연 데이터 처리 (Handling Late Arriving Data):** 워터마킹 등을 통해 지연 도착 데이터를 처리합니다.

CAP 이론 (CAP Theorem)

- **한 줄 요약:** 분산 시스템은 일관성(Consistency), 가용성(Availability), 분할 내성(Partition Tolerance) 중 최대 두 가지만 동시에 만족할 수 있다는 이론입니다.
- **세 가지 요소:**
 - * **일관성 (Consistency):** 모든 노드에서 동일한 시점에 동일한 데이터를 볼 수 있습니다.
 - * **가용성 (Availability):** 모든 요청에 대해 항상 응답을 받을 수 있습니다 (오류가 발생하더라도).
 - * **분할 내성 (Partition Tolerance):** 네트워크 분할(노드 간 통신 단절)이 발생해도 시스템이 계속 작동합니다.
- **선택:**
 - * **CP 기반 시스템 (Consistency over Availability):** 일관성을 우선시합니다 (예: 관계형 데이터베이스). 네트워크 분할 시 시스템이 일시적으로 작동을 멈출 수 있습니다.
 - * **AP 기반 시스템 (Availability over Consistency):** 가용성을 우선시합니다 (예: NoSQL 데이터베이스). 네트워크 분할 시에도 계속 작동하지만, 데이터 일관성이 일시적으로 깨질 수 있습니다. 빅데이터 시스템은 대규모 데이터 처리 요구사항 때문에 AP 시스템으로 진화하는 경우가 많습니다.

제로 데이터 손실 / 제로 클론 패턴 (Zero Data Loss / Zero Clone Pattern)

- **한 줄 요약:** 데이터 손실을 최소화하고 데이터 복제를 효율적으로 관리하는 패턴입니다.
- **설명:** 분산 시스템(예: Kafka)에서 데이터를 내부적으로 복제하여, 클러스터의 한 노드가 손실되어도 데이터가 다른 노드에 남아있도록 합니다. 이는 금융 시스템이나 의료 시스템처럼 데이터 손실이 치명적인 경우에 중요합니다.

데이터 메시 (Data Mesh)

- **한 줄 요약:** 대규모 조직에서 데이터를 제품처럼 취급하고, 각 도메인 팀이 데이터 파이프라인을 독립적으로 관리하며 표준화된 방식으로 데이터를 공유하는 아키텍처입니다.
- **특징:**
 - * 각 팀은 자체 데이터 파이프라인을 엔드 투 엔드로 관리합니다.
 - * 조직 전체의 표준화된 도구와 방법을 사용합니다.
 - * 생성된 데이터는 데이터 제품(Data Product)으로 공유되며, 내부 및 외부 사용자에게 소비됩니다.
- **장점:** 데이터 소유권과 책임감을 명확히 하고, 데이터 활용을 가속화합니다.

에이전트 아키텍처 (Agentic Architecture)

- **한 줄 요약:** 대규모 언어 모델(LLM)을 활용하여 다양한 도구와 에이전트의 작업을 조율하여 복잡한 태스크를 수행하는 최신 설계 패턴입니다.
- **설명:** 에이전트가 외부 API나 데이터베이스를 쿼리하여 특정 작업을 수행하고, 여러 에이전트가 협력하여 더 큰 솔루션을 제공합니다. AI와 기존 시스템을 결합하는 흥미로운 방법입니다.

커넥터 패턴 (Connector Pattern)

- **한 줄 요약:** 다양한 데이터 소스에 일관된 방식으로 접근할 수 있도록 추상화된 인터페이스를 제공하는 패턴입니다.
- **소프트웨어 공학 용어:** 브릿지 패턴(Bridge Pattern)이라고도 합니다.
- **예시:** JDBC(Java Database Connectivity) 커넥터는 다양한 관계형 데이터베이스에 동일한 Java API를 통해 접근할 수 있게 합니다. Spark SQL이 JDBC/ODBC 커넥터를 통해 다양한 데이터 저장소와 상호작용하는 것이 좋은 예시입니다.
- **장점:** 소비자는 데이터 소스의 종류에 관계없이 일관된 방식으로 데이터를 처리할 수 있습니다.

멀티홉 / 메달리온 아키텍처 (Multi-hop / Medallion Architecture)

- **한 줄 요약:** 데이터 파이프라인을 여러 논리적 단계(홉)로 나누어 데이터 품질을 점진적으로 향상시키고, 각 단계에서 다양한 사용자 그룹의 요구사항을 충족시키는 패턴입니다.
- **단계:**
 - * **랜딩 영역 (Landing Area) / 브론즈 (Bronze):** 원시 데이터를 저장합니다. 데이터 엔지니어가 주로 사용합니다.
 - * **정제 영역 (Refined Area) / 실버 (Silver):** 원시 데이터를 정제하고 구조화합니다. 데이터 과학자가 주로 사용합니다.
 - * **집계 영역 (Aggregation Area) / 골드 (Gold):** 비즈니스 분석가가 바로 사용할 수 있도록 데이터를 집계하고 최적화합니다.
- **특징:**
 - * **데이터 품질 증가:** 각 홉을 거칠수록 데이터 품질이 향상됩니다.
 - * **데이터 가용성 증가:** 초기 홉일수록 데이터 가용성이 높습니다 (데이터가 빨리 도착).
 - * **설계 원칙 적용:** 분할 정복(divide and conquer) 및 관심사 분리(separation of concerns) 원칙을 적용하여 파이프라인 설계를 단순화합니다.

데이터 이동 최소화 (Minimizing Movement of Data) 데이터 이동은 비용과 시간을 소모하므로, 이를 최소화하는 기술들이 중요합니다.

- 시간 여행 (Time Travel):

- * **한 줄 요약:** 데이터의 특정 과거 시점 또는 버전으로 돌아가 데이터를 조회하거나 복원하는 기능입니다.
- * **기술적 설명:** Delta Lake는 Parquet 형식 위에 변경 이력을 기록하는 저널 파일(journal file)을 관리하여, 데이터의 특정 버전으로 쉽게 되돌아갈 수 있게 합니다. 이는 데이터 재처리나 오류 복구에 매우 유용합니다.

- 제로 카피 클론 (Zero Copy Clone):

- * **한 줄 요약:** 실제 데이터를 복사하지 않고, 메타데이터만 복사하여 데이터의 복제본을 생성하는 기술입니다.

- * **기술적 설명:** 새로운 메타데이터 세트만 생성하고 원본 데이터는 그대로 유지하므로, 저장 공간을 절약하고 복제 속도를 크게 향상시킵니다.
- * **Delta Clone:**
 - 얕은 클론 (Shallow Clone): 제로 카피 클론과 동일하게 메타데이터만 복사합니다.
 - 깊은 클론 (Deep Clone): 메타데이터와 실제 데이터를 모두 복사합니다.
- **Delta Share:**
 - * **한 줄 요약:** 데이터 복사 없이 외부 사용자에게 데이터 접근 권한을 공유하는 방법입니다.
 - * **장점:** 저장 비용, 데이터 접근 비용, 네트워크 전송 비용을 절감하고, 원본 데이터의 일관성을 유지합니다.

4.2.3 데이터 규정 준수 (Data Compliance)

데이터 엔지니어로서 데이터 규정 준수는 매우 중요한 책임입니다. 개인 정보 보호와 관련된 법규를 이해하고 준수해야 합니다.

핵심 요약

데이터 규정 준수의 중요성:

- **소비자 권리 보호:** 개인 데이터에 대한 소비자의 권리를 보호합니다.
- **법적 책임:** 규정 위반 시 상당한 벌금이 부과될 수 있습니다.
- **데이터 관리 책임:** 데이터 관리자로서 데이터를 책임감 있게 관리해야 합니다.
- **설계 고려사항:** 데이터 파이프라인 및 시스템 설계 시 규정 준수 요건을 미리 반영해야 합니다.

주요 규정

- **GDPR (General Data Protection Regulation):**
 - * **한 줄 요약:** 유럽 연합(EU)의 일반 데이터 보호 규정으로, EU 시민의 개인 데이터 보호를 강화합니다.
 - * **적용 범위:** 데이터가 미국에 저장되어 있더라도, EU 시민의 데이터라면 GDPR이 적용됩니다.
- **CCPA (California Consumer Privacy Act):**
 - * **한 줄 요약:** 캘리포니아 소비자 개인 정보 보호법으로, 캘리포니아 주민의 개인 정보 보호를 위한 프레임워크를 제공합니다.

핵심 소비자 권리

- **삭제할 권리 (Right of Erasure) / 잊힐 권리 (Right to be Forgotten):**
 - * **한 줄 요약:** 소비자가 자신의 개인 데이터를 시스템에서 삭제하도록 요구할 수 있는 권리입니다.
 - * **기술적 과제:** 대규모 데이터 레이크에서 특정 사용자의 데이터를 식별하고, 다른 사용자 데이터에 영향을 주지 않으면서 해당 데이터를 완전히 삭제하는 것은 매우 어려운 작업입니다.
- **데이터 이동성 권리 (Right of Portability):**
 - * **한 줄 요약:** 소비자가 자신의 데이터를 시스템에서 추출하여 다른 서비스(경쟁사 포함)로 이전하도록 요구할 수 있는 권리입니다.
 - * **기술적 과제:** 특정 사용자의 데이터를 식별하고, 시스템에서 추출하여 표준화된 형식으로 제공할 수 있어야 합니다.

규정 준수를 위한 기술적 접근

- 세분화된 업데이트 및 삭제 (**Fine-grained Updates and Deletes**): Delta Lake와 같은 기술은 대규모 데이터에서 특정 행에 대한 업데이트 및 삭제를 지원하여 규정 준수를 용이하게 합니다.
- 가명화 / 익명화 (**Pseudonymization / Anonymization**):
 - * **한 줄 요약**: 개인 식별이 가능한 정보를 비식별화된 식별자(예: 토큰, GUID)로 대체하여 프라이버시를 보호하는 방법입니다.
 - * **기술적 설명**: 실제 사용자 정보와 비식별화된 식별자 간의 매핑 정보를 별도로 관리합니다. 사용자의 데이터 삭제 요청 시, 이 매핑 정보만 삭제하면 실제 데이터는 시스템에 남아있더라도 더 이상 특정 사용자를 추적할 수 없게 됩니다.

주의

규정 준수 유의사항: 의료 분야(HIPAA)나 정부 기관(FedRAMP)과 같은 규제 산업에서는 데이터 보호 및 보안 표준이 훨씬 엄격합니다. 데이터 엔지니어는 이러한 추가적인 요구사항을 반드시 인지하고 시스템 설계에 반영해야 합니다. 데이터 분류, 접근 감사 추적 등 추가적인 계층이 필요합니다.

4.3 Spark 내부 동작 및 최적화

Spark는 분산 컴퓨팅 프레임워크로, 대규모 데이터 처리를 위해 복잡한 내부 메커니즘을 가지고 있습니다. Spark의 동작 방식을 이해하는 것은 성능 최적화에 필수적입니다.

4.3.1 Spark 아키텍처: 작업, 단계, 태스크 (Jobs, Stages, Tasks)

Spark는 사용자의 코드를 여러 단계로 나누어 분산 환경에서 병렬로 실행합니다.

- **드라이버 (Driver)**: Spark 애플리케이션의 메인 프로세스로, 작업을 스케줄링하고 워커 노드 (Worker Nodes)에 태스크를 분배합니다.
- **워커 노드 (Worker Nodes)**: 실제 데이터 처리 작업을 수행하는 물리적 또는 가상 머신입니다. 각 워커 노드에는 하나 이상의 실행자(Executor)가 있습니다.
- **실행자 (Executor)**: 워커 노드에서 Spark 태스크를 실행하는 JVM 프로세스입니다.

Figure 4.1: Spark 아키텍처: 드라이버, 워커, 실행자, 작업, 단계, 태스크

- **작업 (Job)**: Spark 액션(예: `collect()`, `write()`)이 트리거될 때 생성됩니다. 하나의 Spark 애플리케이션은 여러 작업을 포함할 수 있습니다.
- **단계 (Stage)**: 작업은 여러 단계로 나뉩니다. 각 단계는 논리적인 작업 단위로, 데이터의 셔플 (shuffle) 발생 여부에 따라 구분됩니다.
 - * **좁은 변환 (Narrow Transformation)**: `map`, `filter`와 같이 각 파티션의 데이터만으로 결과를 계산할 수 있는 변환입니다. 셔플이 발생하지 않습니다.
 - * **넓은 변환 (Wide Transformation) / 셔플 변환 (Shuffle Transformation)**: `groupBy`, `reduceBy`와 같이 다른 파티션의 데이터가 필요한 변환입니다. 데이터 셔플이 발생하며, 이는 네트워크 오버헤드를 유발하여 비용이 많이 드는 작업입니다. 셔플은 최소화하는 것이 좋습니다.
- **태스크 (Task)**: 단계는 다시 여러 태스크로 나뉩니다. 태스크는 Spark에서 실행되는 가장 작은 작업 단위이며, 실행자의 코어(core)에서 병렬로 실행됩니다.

예제

Spark 쿼리 실행 흐름:

1. 사용자가 Python(PySpark) 또는 SQL 코드를 작성합니다.
2. 코드는 Spark 드라이버로 제출됩니다.
3. 드라이버는 코드를 여러 **작업(Jobs)**으로 나눕니다.
4. 각 작업은 Spark 엔진에 의해 최적화 과정을 거쳐 여러 **단계(Stages)**로 분할됩니다. 셔플이 필요한 지점에서 새로운 단계가 시작됩니다.
5. 각 단계는 여러 **태스크(Tasks)**로 분할됩니다.
6. 태스크는 워커 노드의 **실행자(Executors)**에 할당된 코어에서 병렬로 실행됩니다.
7. 태스크는 데이터의 작은 부분을 처리하고, 결과는 다시 드라이버를 거쳐 사용자에게 전달됩니다.

4.3.2 테이블 파티셔닝 vs Spark 파티셔닝

‘파티션’이라는 용어는 Spark 생태계에서 여러 맥락에서 사용되므로 혼동하기 쉽습니다. 테이블 파티셔닝과 Spark 파티셔닝은 서로 다른 개념입니다.

Table 4.1: 테이블 파티셔닝과 Spark 파티셔닝 비교

구분	테이블 파티셔닝 (Table Partitioning)	Spark 파티셔닝 (Spark Partitioning)
정의	대규모 테이블을 디스크 상의 물리적인 작은 조각(파일/디렉토리)으로 나누는 방식	Spark가 데이터를 워커 노드에 분산하여 처리하는 논리적인 방식
목적	데이터 조직화, 쿼리 성능 향상 (디스크 I/O 감소)	병렬 처리, 최적화, 장애 허용
제어 주체	사용자 (DDL 문에서 명시적으로 지정)	Spark 엔진 (데이터, 쿼리, 설정에 따라 동적으로 결정)
예시	PARTITIONED BY (country, date)	Spark 데이터프레임의 내부 파티션 수
최적화 기법	파티션 가지치기 (Partition Pruning): WHERE 절에 파티션 키를 사용하면 해당 파티션만 스캔하여 불필요한 데이터 읽기 방지	데이터 셔플링 (Data Shuffling): 조인, 그룹화 시 데이터 재분배. 최소화하는 것이 성능에 중요
변경 용이성	일단 설정되면 변경하기 어려움 (데이터 재구성 필요)	쿼리 실행 시 동적으로 변경 가능

테이블 파티셔닝 (Table Partitioning)

- **한 줄 요약:** 대규모 테이블을 디스크 상의 물리적인 작은 조각으로 나누어 관리하고 쿼리 성능을 높이는 기법입니다.
- **필요성:** 데이터가 너무 많아 전체 테이블을 스캔하는 데 시간이 오래 걸릴 때, 특정 조건의 데이터만 빠르게 찾기 위해 사용합니다.
- **예시:** 판매 데이터를 country나 date 컬럼으로 파티셔닝하면, 특정 국가나 날짜의 데이터만 조회할 때 전체 데이터를 스캔할 필요 없이 해당 파티션만 읽어 빠르게 결과를 얻을 수 있습니다.

- 파티션 가지치기 (Partition Pruning): WHERE country = 'USA' 와 같은 쿼리가 들어오면, Spark 는 'USA' 파티션만 읽어 불필요한 디스크 I/O를 줄입니다.
- 파티셔닝 키 선택 기준:
 - * WHERE 절에서 자주 사용되는 컬럼을 선택합니다.
 - * 카디널리티(Cardinality, 고유 값의 수)가 너무 높지 않은 컬럼을 선택합니다. (너무 많으면 작은 파일이 너무 많이 생성됨)
 - * 각 파티션에 최소 1GB 이상의 데이터가 있도록 하는 것이 좋습니다.
 - * 최신 권장 사항: 데이터 크기가 1TB 미만인 경우, 수동 파티셔닝을 하지 않는 것이 좋습니다. Spark 엔진의 내장된 최적화 기능이 더 효율적일 수 있습니다. 잘못된 파티셔닝은 오히려 성능을 저하시킬 수 있습니다.

예제

SQL을 이용한 테이블 파티셔닝 예시:

```

1 CREATE TABLE customer (
2     id INT,
3     name STRING,
4     age INT,
5     city STRING,
6     state STRING
7 )
8 PARTITIONED BY (state, city); -- 와 state city 컬럼으로 파티셔닝
9
10 INSERT INTO customer PARTITION (state='CA', city='Los Angeles')
11 VALUES (1, 'Alice', 30); -- 특정 파티션에 데이터 삽입
12
13 SHOW PARTITIONS customer; -- 파티션 목록 조회
14 DESCRIBE FORMATTED customer; -- 테이블 상세 정보 파티션 (포함)

```

Listing 4.1: 테이블 파티셔닝 정의

Spark 파티셔닝 (Spark Partitioning)

- 한 줄 요약: Spark가 데이터를 워커 노드에 분산하여 병렬 처리하기 위한 논리적인 데이터 분할 단위입니다.
- 필요성: Spark의 분산 컴퓨팅 능력을 최대한 활용하여 대규모 데이터를 빠르게 처리하고, 장애 발생 시 복원력을 높이기 위함입니다.
- 제어: Spark는 입력 데이터의 크기, Spark 설정, 데이터 스큐(skew) 등을 기반으로 파티션 수를 동적으로 결정합니다. 사용자가 직접 파티션 수를 조절할 수도 있지만, 신중하게 접근해야 합니다.
- 데이터 셔플링 (Data Shuffling): Spark 파티션 간에 데이터가 재분배되는 과정으로, 조인(JOIN), 그룹화(GROUP BY), 축소(REDUCE BY)와 같은 넓은 변환에서 발생합니다. 셔플은 네트워크 오버헤드를 유발하므로 최소화해야 합니다.

주의**파티셔닝 관련 흔한 실수:**

- **너무 적은 파티션:** 병렬 처리의 이점을 충분히 활용하지 못하고, 일부 워커 노드에 과부하가 걸릴 수 있습니다.
- **너무 많은 파티션:** 작은 파일이 너무 많이 생성되어 파일 접근 오버헤드가 커지고, Spark의 스케줄링 및 관리 비용이 증가하여 오히려 성능이 저하될 수 있습니다. 각 파티션에 최소 1GB의 데이터가 있도록 하는 것이 좋습니다.

4.3.3 쿼리 최적화 과정

Spark를 포함한 대부분의 데이터베이스 시스템은 쿼리를 실행하기 전에 복잡한 최적화 과정을 거칩니다.

1. **논리적 계획 (Logical Plan):** 사용자가 제출한 쿼리(SQL 또는 DataFrame API)는 먼저 추상적인 논리적 계획으로 변환됩니다. 이 계획은 '무엇을 해야 하는가'를 정의합니다.
2. **카탈로그 조회 (Catalog Lookup):** 시스템 카탈로그에서 테이블 스키마, 통계 정보 등을 조회하여 논리적 계획을 구체화합니다.
3. **최적화된 논리적 계획 (Optimized Logical Plan):** Spark 엔진은 다양한 규칙 기반 및 비용 기반 최적화를 적용하여 논리적 계획을 개선합니다 (예: 필터 푸시다운, 프로젝션 푸시다운).
4. **물리적 계획 (Physical Plan):** 최적화된 논리적 계획은 '어떻게 해야 하는가'를 정의하는 하나 이상의 물리적 계획으로 변환됩니다. 각 물리적 계획은 특정 알고리즘(예: 조인 방식)을 나타냅니다.
5. **비용 기반 최적화 (Cost-based Optimizer):** 여러 물리적 계획 중에서 가장 효율적인(가장 낮은 비용이 예상되는) 계획을 선택합니다.
6. **코드 생성 (Code Generation):** 선택된 물리적 계획을 기반으로 실제 실행 가능한 코드를 생성하고, 이를 RDD(Resilient Distributed Datasets) 작업으로 변환하여 워커 노드에서 실행합니다.

4.3.4 Delta Lake 최적화 기능

Delta Lake는 데이터 레이크에 데이터 웨어하우스의 신뢰성을 부여하는 기술로, 다양한 최적화 기능을 제공합니다.

파일 압축 (File Compaction) 및 Z-오더링 (Z-Ordering)

- **파일 압축 (File Compaction) / 빈 패킹 (Bin Packing):**
 - * **한 줄 요약:** 작은 파일들을 더 크고 효율적인 파일로 병합하여 쿼리 성능을 향상시키는 작업입니다.
 - * **필요성:** 빅데이터 시스템에서는 데이터가 작은 파일로 자주 들어오는데, 너무 많은 작은 파일은 파일 접근 오버헤드를 증가시켜 쿼리 속도를 저하시킵니다.
 - * **기술적 설명:** OPTIMIZE 명령을 사용하여 여러 작은 Parquet 파일을 하나의 큰 파일로 병합합니다. 이는 디스크 I/O 횟수를 줄여 쿼리 성능을 크게 개선합니다.
 - * **예시:** `OPTIMIZE table_name;` 또는 `OPTIMIZE table_name WHERE date >= '2023-01-01';`
- **Z-오더링 (Z-Ordering):**
 - * **한 줄 요약:** 여러 컬럼에 걸쳐 관련 데이터를 물리적으로 가깝게 저장하여 다차원 쿼리 성능을 최적화하는 기법입니다.

- * **필요성:** 특정 쿼리가 여러 컬럼(예: `from_ip, to_ip, from_port, to_port`)을 동시에 필터링할 때, 이 컬럼들의 데이터가 디스크에 흩어져 있으면 많은 I/O가 발생합니다.
- * **기술적 설명:** Z-오더링은 다차원 공간 채우기 곡선(multi-dimensional space-filling curve)을 사용하여, 지정된 컬럼들의 유사한 값들을 디스크 상에서 물리적으로 인접하게 배치합니다. 이를 통해 쿼리 시 필요한 데이터를 한 번에 읽을 수 있어 성능이 향상됩니다.
- * **예시:** `OPTIMIZE table_name ZORDER BY (from_ip, to_ip, from_port, to_port);`
- **예측 최적화 (Predictive Optimizations):** Databricks의 최신 기능으로, 클러스터에서 이 기능을 활성화하면 파일 압축과 같은 최적화 작업을 자동으로 수행합니다.

시간 여행 (Time Travel)

- **한 줄 요약:** Delta Lake 테이블의 과거 버전으로 돌아가 데이터를 조회하거나 복원하는 기능입니다.
- **필요성:** 데이터 변환 중 오류가 발생했거나, 과거 특정 시점의 데이터를 분석해야 할 때 유용합니다.
- **기술적 설명:** Delta Lake는 모든 트랜잭션 변경 사항을 기록하는 트랜잭션 로그(transaction log)를 유지합니다. 이를 통해 특정 버전 번호(VERSION AS OF) 또는 타임스탬프(TIMESTAMP AS OF)로 테이블을 복원할 수 있습니다.
- **예시:**

```
1 RESTORE TABLE my_delta_table TO VERSION AS OF 26;
2 RESTORE TABLE my_delta_table TO TIMESTAMP AS OF '2023-10-26 10:00:00';
```

Listing 4.2: Delta Lake 시간 여행 예시

VACUUM

- **한 줄 요약:** Delta Lake 테이블에서 더 이상 필요 없는 오래된 데이터 파일들을 영구적으로 삭제하는 명령입니다.
- **필요성:** 시간 여행 기능으로 인해 Delta Lake는 과거 버전의 데이터를 유지합니다. 이 파일들이 너무 많아지면 저장 공간을 불필요하게 차지하고 쿼리 성능에 영향을 줄 수 있습니다.
- **기술적 설명:** VACUUM 명령은 지정된 보존 기간(retention period)보다 오래된 파일들을 삭제합니다. 기본 보존 기간은 7일입니다.
- **예시:** `VACUUM my_delta_table RETAIN 168 HOURS;` (7일 = 168시간)

CLONE (복제)

- **한 줄 요약:** Delta Lake 테이블의 복제본을 생성하는 기능입니다.
- **유형:**
 - * **얕은 클론 (Shallow Clone):** 메타데이터만 복사하고 실제 데이터 파일은 공유합니다 (제로 카피 클론). 빠르고 저장 공간을 절약합니다.
 - * **깊은 클론 (Deep Clone):** 메타데이터와 실제 데이터를 모두 복사합니다. 재해 복구 시나리오나 ML 작업에서 독립적인 데이터셋이 필요할 때 사용됩니다.
- **예시:**

```
1 CREATE TABLE new_table CLONE old_table; -- 기본값은 깊은 클론
2 CREATE TABLE new_table SHALLOW CLONE old_table; -- 얕은 클론
```


Listing 4.3: Delta Lake 클론 예시

4.3.5 Spark 조인 유형 (Spark Join Types)

Spark는 다양한 조인 전략을 사용하여 분산 환경에서 테이블을 결합합니다. 조인 유형을 이해하는 것은 쿼리 성능 최적화에 중요합니다.

주의

조인에 대한 일반적인 인식: 빅데이터 시스템에서는 조인 작업이 느리고 비용이 많이 들 수 있으므로, 가능한 한 조인을 피하고 데이터를 비정규화된 형태로 저장하는 것이 권장됩니다. 그러나 SQL 환경에서는 조인이 필수적이므로, Spark는 효율적인 조인 전략을 제공합니다.

Table 4.2: Spark 조인 유형 비교

조인 유형	설명	요구사항	성능
브로드캐스트 해시 조인 (Broadcast Hash Join)	작은 테이블을 모든 워커 노드에 복사하여 큰 테이블의 각 파티션과 로컬에서 조인	한쪽 테이블이 매우 작아 모든 워커 노드의 메모리에 들어갈 수 있어야 함	가장 빠름: 셔플 및 정렬 없음
셔플 해시 조인 (Shuffle Hash Join)	한쪽 테이블이 다른 테이블보다 작고, 해당 테이블의 파티션이 메모리에 들어갈 수 있을 때 사용	한쪽 테이블이 다른 테이블보다 3배 이상 작고, 파티션이 메모리에 적합해야 함	중간: 셔플은 발생할 수 있으나 정렬 없음
정렬 병합 조인 (Sort Merge Join)	양쪽 테이블을 조인 키로 정렬한 후 병합하는 방식	모든 크기의 테이블에 적용 가능	느림: 셔플 및 정렬이 필요할 수 있음 (기본값)
카르테시안 조인 (Cartesian Join)	두 테이블의 모든 행을 조합하여 조인 (교차 조인)	조인 조건이 없거나 잘못된 경우 발생	매우 느림: 엄청난 양의 데이터를 생성하므로 피해야 함

4.3.6 사용자 정의 함수 (User-Defined Functions, UDFs)

Spark API에 없는 특정 로직을 구현해야 할 때 사용자가 직접 함수를 정의하여 사용할 수 있습니다.

UDF 유형

– UDF (User-Defined Function):

- * **한 줄 요약:** 한 행을 입력받아 한 행을 출력하는 함수입니다.
- * **기술적 설명:** Spark는 JVM 기반으로 작성되었고, Python UDF는 Python으로 작성됩니다. 따라서 Python UDF를 사용할 때마다 Spark와 Python 프로세스 간에 데이터 직렬화(serialization) 및 역직렬화(deserialization)가 발생하여 성능 오버헤드가 발생합니다.

– 벡터화된 UDF (Vectorized UDF) / Pandas UDF:

- * **한 줄 요약:** Pandas Series 또는 DataFrame을 입력받아 처리하여 성능 오버헤드를 줄인 UDF입니다.

- * **기술적 설명:** @pandas_udf 데코레이터를 사용하여 정의하며, Spark와 Python 간의 데이터 전송을 배치(batch) 단위로 처리하여 직렬화/역직렬화 횟수를 줄여 성능을 향상시킵니다.
- **UAF (User-Defined Aggregate Function):**
 - * **한 줄 요약:** 여러 행을 입력받아 단일 집계 값을 반환하는 함수입니다 (예: SUM, COUNT).
- **UDTF (User-Defined Transformative Function):**
 - * **한 줄 요약:** 여러 행을 입력받아 여러 행을 출력하는 함수입니다 (예: EXPLODE, PIVOT).

UDF 사용 권장 사항

1. **Spark API 우선 사용:** Spark API에 이미 유사한 기능이 있다면, 내장 함수를 사용하는 것이 가장 효율적입니다.
2. **Pandas UDF (벡터화된 UDF):** Spark API에 기능이 없고 사용자 정의 로직이 필요하다면, Pandas UDF를 사용하여 성능 오버헤드를 최소화합니다.
3. **일반 Python UDF:** 최후의 수단으로 사용하며, 성능에 민감하지 않은 경우에만 고려합니다.

주의

UDF 남용의 위험성: UDF는 강력하지만, 버그가 있거나 비효율적으로 작성된 UDF는 전체 파이프라인의 성능을 저하시킬 수 있습니다. 따라서 UDF 사용 시에는 신중하게 테스트하고 최적화해야 합니다.

4.3.7 CRUD 작업 (Create, Read, Update, Delete)

CRUD는 데이터베이스 시스템에서 데이터를 다루는 기본적인 네 가지 작업을 의미합니다.

- **Create (생성):** 카탈로그, 볼륨, 스키마, 테이블 등을 생성하는 작업입니다.
- **Read (읽기):** 파티션 키, Z-오더링 등을 활용하여 데이터를 조회하는 작업입니다.
- **Update (업데이트):**
 - * **전체 덮어쓰기 (Overwrite):** 참조 데이터와 같이 전체 데이터를 새로 고쳐야 할 때 사용합니다.
 - * **추가 (Append):** 트랜잭션 데이터와 같이 새로운 데이터만 추가할 때 사용합니다.
 - * **세분화된 업데이트 (Fine-grained Updates):** Delta Lake와 같은 기술을 통해 특정 조건에 맞는 행만 업데이트할 수 있습니다.
 - * **Upsert (업서트):** 데이터가 존재하면 업데이트하고, 없으면 삽입하는 작업입니다. 전통적으로 여러 단계가 필요했지만, Delta Lake의 MERGE 명령으로 원자적으로(atomicity) 처리할 수 있습니다.
- **Delete (삭제):**
 - * **파티션 삭제:** 특정 파티션 전체를 삭제합니다.
 - * **세분화된 삭제 (Fine-grained Deletes):** Delta Lake를 통해 특정 조건에 맞는 행만 삭제할 수 있습니다. 이는 GDPR과 같은 규정 준수에 필수적입니다.

스키마 진화 (Schema Evolution)

데이터 파이프라인에서 데이터 스키마는 시간이 지남에 따라 변경될 수 있습니다 (예: 새 컬럼 추가, 데이터 타입 변경).

- **스키마 추론 (Schema Inference):** 데이터로부터 스키마를 자동으로 감지하는 기능입니다.

– 스키마 진화 (Schema Evolution):

- * **한 줄 요약:** 데이터 스키마의 변경(새 컬럼 추가, 데이터 타입 변경 등)을 허용하고 자동으로 처리하는 기능입니다.
- * **기술적 설명:** 기본적으로 Spark는 스키마 변경에 대해 엄격하여 오류를 발생시킵니다. 하지만 `mergeSchema=true` 옵션을 사용하면, 호환 가능한 스키마 변경(예: 새 컬럼 추가, `string`에서 `int`로의 타입 변경 등)을 자동으로 처리하여 파이프라인이 중단되지 않도록 합니다.
- * **권장 사항:** 파이프라인의 초기 단계(예: 브론즈 레이어)에서는 스키마 진화를 허용하고, 최종 단계(예: 골드 레이어)에서는 스키마 강제 (Schema Enforcement)를 적용하여 데이터 계약을 명확히 하는 것이 좋습니다.

– 스키마 강제 (Schema Enforcement):

- * **한 줄 요약:** 정의된 스키마와 다른 데이터가 들어오면 오류를 발생시켜 파이프라인을 중단시키는 기능입니다.
- * **필요성:** 데이터 품질을 보장하고, 다운스트림 소비자가 예상치 못한 스키마 변경으로 인해 문제를 겪는 것을 방지합니다.

4.3.8 변경 데이터 캡처 (Change Data Capture, CDC) 및 서서히 변하는 차원 (Slowly Changing Dimensions, SCD)

CDC와 SCD는 데이터 변경 이력을 관리하는 데 사용되는 중요한 개념입니다.

CDC (Change Data Capture)

- **한 줄 요약:** 원본 시스템에서 데이터의 변경 사항(삽입, 업데이트, 삭제)을 감지하고 기록하는 기술입니다.
- **관점:** 주로 소스 측면에서 데이터 변경을 포착하는 것을 의미합니다.
- **기술적 설명:** Delta Lake의 변경 데이터 피드(Change Data Feed) 기능을 활성화하면, 테이블의 버전 간 행 수준 변경 사항을 기록하는 별도의 폴더(`_change_data`)가 생성됩니다. 이 피드는 각 행이 삽입(INSERT), 삭제(DELETE), 업데이트(UPDATE)되었는지에 대한 메타데이터를 포함합니다.
- **예시:**

```
1 ALTER TABLE my_delta_table SET TBLPROPERTIES (delta.enableChangeDataFeed
   = true);
2
3 SELECT * FROM table_changes('my_delta_table', 0, 10); -- 버전 0 부터 10 까지
   의 10 변경 사항 조회
```

Listing 4.4: Delta Lake 변경 데이터 피드 활성화 및 조회

– 변경 데이터 피드 예시:

- * **원본 테이블:** A1 B1, A2 B2, A3 B3
- * **변경 데이터 피드:**
 - A2 B2 (A2 B2가 A2 Z2로 변경됨) -> `pre_image (B2), post_image (Z2), operation (UPDATE)`
 - A3 (A3 B3가 삭제됨) -> `pre_image (B3), post_image (NULL), operation (DELETE)`
 - A4 B4 (A4 B4가 새로 삽입됨) -> `pre_image (NULL), post_image (B4), operation (INSERT)`

SCD (Slowly Changing Dimensions)

- **한 줄 요약:** 데이터 웨어하우스의 차원 테이블에서 시간이 지남에 따라 변경되는 속성(예: 고객 주소)의 이력을 관리하는 방법입니다.
- **관점:** 주로 타겟 측면에서 데이터 변경 이력을 관리하는 것을 의미합니다.
- **필요성:** 고객의 주소와 같은 차원 데이터가 변경될 때, 과거 트랜잭션 데이터가 어떤 주소와 관련되어 있었는지 추적해야 할 필요가 있습니다.

SCD 유형

- **Type 0: 고정 (Fixed):** 변경을 허용하지 않습니다. 데이터가 변경되어도 기록하지 않고 원본 값을 유지합니다.
- **Type 1: 덮어쓰기 (Overwrite):**
 - * **한 줄 요약:** 변경된 데이터를 기존 데이터 위에 덮어씹습니다. 과거 이력은 유지되지 않습니다.
 - * **장점:** 추가 저장 공간이나 복잡한 처리 로직이 필요 없습니다.
 - * **단점:** 과거 데이터를 추적할 수 없습니다.
 - * **예시:** 고객의 주소가 변경되면, 기존 주소 위에 새로운 주소를 덮어씹습니다. 5년 전 주소를 물어보면 알 수 없습니다.
- **Type 2: 새 레코드 추가 (Add New Record):**
 - * **한 줄 요약:** 데이터가 변경될 때마다 새로운 레코드를 추가하여 모든 변경 이력을 유지합니다.
 - * **장점:** 모든 과거 이력을 추적할 수 있습니다.
 - * **단점:** 저장 공간과 처리 오버헤드가 증가합니다.
 - * **일반적인 구현:** `start_date`, `end_date`, `is_current`와 같은 추가 컬럼을 사용하여 각 레코드의 유효 기간과 현재 상태를 나타냅니다.
 - * **예시:** 고객의 주소가 변경되면, 기존 주소 레코드의 `end_date`를 업데이트하고 `is_current` 플래그를 `false`로 설정합니다. 그리고 새로운 주소로 `start_date`와 `is_current`가 `true`인 새 레코드를 삽입합니다.
- **Type 3: 새 컬럼 추가 (Add New Column):** 변경될 때마다 새로운 컬럼을 추가하여 이전 값을 저장합니다. 컬럼 수가 너무 많아질 수 있어 지속 가능하지 않습니다.
- **Type 4: 이력 테이블 분리 (Separate History Table):** 변경 이력을 별도의 이력 테이블에 저장합니다.
- **Type 6: Type 1, 2, 3 결합 (Combined):** Type 1, 2, 3의 요소를 결합한 방식입니다.

예제

SCD Type 1 예시 (덮어쓰기):

- 초기 테이블:
 - * 고객 1: 이름 A, 나이 20
 - * 고객 2: 이름 B, 나이 30
- 변경 데이터:
 - * 고객 1: 이름 AA, 나이 21 (업데이트)
 - * 고객 2: (삭제)
 - * 고객 3: 이름 C, 나이 16 (삽입)

- 최종 테이블 (SCD Type 1):
 - * 고객 1: 이름 AA, 나이 21 (업데이트됨)
 - * 고객 3: 이름 C, 나이 16 (삽입됨)
- 고객 2의 데이터는 완전히 사라집니다.
- SCD Type 2 예시 (새 레코드 추가):**
 - 초기 테이블 (추가 컬럼: `start_date`, `end_date`, `is_current`):
 - * 고객 1: 이름 A, 나이 20, 2023-01-01, 9999-12-31, TRUE
 - * 고객 2: 이름 B, 나이 30, 2023-01-01, 9999-12-31, TRUE
 - 변경 데이터 (위와 동일):
 - * 고객 1: 이름 AA, 나이 21 (업데이트)
 - * 고객 2: (삭제)
 - * 고객 3: 이름 C, 나이 16 (삽입)
 - 최종 테이블 (SCD Type 2):
 - * 고객 1: 이름 A, 나이 20, 2023-01-01, 2023-10-26, FALSE (이전 레코드)
 - * 고객 1: 이름 AA, 나이 21, 2023-10-27, 9999-12-31, TRUE (새 레코드)
 - * 고객 2: 이름 B, 나이 30, 2023-01-01, 2023-10-26, FALSE (삭제된 레코드)
 - * 고객 3: 이름 C, 나이 16, 2023-10-27, 9999-12-31, TRUE (새 레코드)
 - 고객 2의 데이터도 `is_current`가 FALSE로 변경되어 이력이 유지됩니다.

강의명: CSCI103

주차: Lecture 4

교수명: 홍길동 교수

목적: Autoloader를 이용한 데이터 로딩 및 관리, 스키마 진화, 데이터 품질 처리 방법 학습

4.4 Autoloader를 이용한 데이터 로딩

Autoloader는 클라우드 저장소(S3, ADLS, GCS 등)에 도착하는 새로운 데이터를 점진적으로 처리하는 효율적인 방법입니다.

4.4.1 Autoloader 개요

- **한 줄 요약:** 클라우드 저장소에 새로운 파일이 도착하는 것을 자동으로 감지하고, 이를 스트리밍 방식으로 처리하는 기능입니다.
- **특징:**
 - * **증분 처리 (Incrementally Processes):** 새로운 데이터만 처리하여 효율적입니다.
 - * **추가 설정 불필요 (No Additional Setup):** 복잡한 설정 없이 쉽게 사용할 수 있습니다.
 - * **효율성 (Efficient):** 파일 기반 스트리밍이지만, 내부적으로 메타데이터를 추적하여 효율성을 높입니다.

- * **클라우드 저장소 지원:** S3, ADLS, Google Cloud Storage 등 주요 클라우드 저장소를 지원합니다.
- * **내장 기능:** 자동 스케일링, 데이터 품질 검사, 스키마 진화 등의 기능이 내장되어 있습니다.
- **동작 원리:** Autoloader는 체크포인트 위치를 사용하여 어떤 파일이 새로 도착했는지 추적하고, 해당 파일의 메타데이터를 기반으로 데이터를 수집합니다.

4.4.2 데이터 로딩 패턴

Autoloader는 다양한 데이터 로딩 시나리오를 지원합니다.

- 클라우드 저장소에서 데이터 수집:

- * `spark.readStream.format("cloudFiles")`를 사용하여 Autoloader를 활성화합니다.
- * `option("cloudFiles.format", "json")`과 같이 데이터 형식을 지정합니다.
- * `option("cloudFiles.schemaLocation", "/path/to/schema")`를 통해 스키마 저장 위치를 지정하여 스키마 추론 및 진화를 관리합니다.

- 파일 경로 패턴 매칭:

- * 정규 표현식(*, ?)을 사용하여 특정 패턴의 파일만 수집할 수 있습니다.
- * `prefix`를 사용하여 특정 경로 아래의 모든 파일을 수집할 수 있습니다.
- * `binaryFile` 형식을 사용하여 이미지와 같은 바이너리 파일을 수집할 수 있습니다.

4.4.3 스키마 관리 및 데이터 품질

Autoloader는 스키마 변경 및 데이터 품질 문제를 유연하게 처리할 수 있는 다양한 옵션을 제공합니다.

- 스키마 추론 및 진화 (Schema Inference and Evolution):

* 조건:

1. 데이터에 대한 스키마를 명시적으로 제공하지 않습니다.
2. 추론된 스키마를 저장할 위치를 제공합니다 (`cloudFiles.schemaLocation`). 이 경로에 스키마 버전이 저장됩니다.
3. `mergeSchema=true` 옵션을 설정하여 새로운 필드를 감지하고 스키마를 자동으로 업데이트하도록 허용합니다.

- * **장점:** 데이터 소스의 스키마가 변경되어도 파이프라인을 중단 없이 유지할 수 있습니다.

- 데이터 손실 방지 및 잘못된 데이터 처리:

* 구조화된 데이터 형식에서 데이터 손실 방지:

- `schemaEvolutionMode("rescue")`: 스키마와 일치하지 않는 데이터를 `_rescued_data`라는 특수 컬럼에 저장합니다. 이 컬럼이 비어있지 않으면 잘못된 데이터가 있음을 나타내며, 이를 분석하여 데이터 품질 문제를 해결할 수 있습니다.
- `schemaEvolutionMode("failOnNewColumns")`: 새로운 컬럼이 도입되어 스키마와 일치하지 않으면 스트림 처리를 중단시킵니다. 데이터 계약을 엄격하게 지켜야 할 때 유용합니다.

- 스키마 힌트 (Schema Hints):

- * **한 줄 요약:** Autoloader가 스키마를 추론할 때 특정 컬럼의 데이터 타입을 명시적으로 지정하여 추론의 정확도를 높이는 기능입니다.
- * **필요성:** Autoloader가 데이터를 `string`으로 잘못 추론하는 경우(예: `float`나 `long`이어야 할 값), 스키마 힌트를 사용하여 올바른 타입을 강제할 수 있습니다.
- * **예시:** `option("cloudFiles.schemaHints", "signal_strength FLOAT, timestamp LONG, metadata`